DISTURBANCE-AWARE MODEL-BASED AND MODEL-FREE ZERO-SUM PURSUIT-EVASION CONTROL: THEORY, ALGORITHM DEVELOPMENT, AND MATLAB VALIDATION

A Thesis

Presented to

the Faculty of the College of Graduate Studies

Tennessee Technological University

by

Blaine Christopher Swieder

In Partial Fulfillment

of the Requirements of the Degree

Master of Science

Electrical and Computer Engineering

August 2026

ABSTRACT

Autonomous and adversarial multi-agent systems require control decisions that remain effective when the inputs applied to a system differ from the commands generated by the controller. This thesis investigates disturbance-aware control for a discrete-time, two-player, zero-sum pursuit-evasion game and develops a common framework for extending established model-based and model-free solution methods without replacing their original policy iteration, value iteration, or Q-learning structures. The pursuer is represented as the minimizing player, while the evader is represented as the maximizing player within a linear-quadratic dynamic game. The nominal saddle point policies are characterized through a generalized algebraic Riccati equation and an equivalent quadratic action-value representation.

Four established methods are considered, model-based generalized Riccati policy and value iteration, as well as model-free quadratic Q-learning policy and value iteration. Each method is extended to include persistent command-channel offsets affecting both the pursuer and evader. The offsets represent repeatable differences between commanded and realized inputs. Offset estimates are obtained from transition residuals or measured command mismatch, depending on the information structure of the algorithm. Affine compensation is then applied by subtracting the estimated offsets from the commanded control actions. As a result, the resulting residual-bias dynamics directly relate the compensated closed-loop response to the remaining estimation error and establish exact compensation as a verification case.

MATLAB studies evaluate nominal operation, uncompensated command offsets, estimated-offset compensation, and exact compensation under matched system matrices, game weights, disturbances, initial conditions, and numerical criteria. The validation includes policy and value convergence, generalized Riccati and Bellman residuals, closed-loop spectral radius, saddle-point curvature, matrix conditioning, feature-rank requirements, bias-estimation error, state and input trajectories, and cumulative game cost. This thesis does not introduce new base algorithms; moreover, its contribution is the unified disturbance-aware extension and matched evaluation of four established zero-sum control methods within a common pursuit-evasion framework. Pursuit-evasion provides the application interpretation, while nonlinear autonomous ground vehicle and QCar implementation are retained as future extensions rather than completed hardware validation.

TABLE OF CONTENTS

CERTIFICATE OF APPROVAL OF THESIS

Disturbance-Aware Model-Based and Model-Free Zero-Sum Pursuit-Evasion Control: Theory, Algorithm Development, and MATLAB Validation

by

Blaine Christopher Swieder

Graduate Advisory Committee:

Syed Ali Asad Rizvi, Chair — Date

Syed Rafay Hasan — Date

Charles Van Neste — Date

Approved for the Faculty:

Julie Baker, Dean — Date

College of Graduate Studies

DEDICATION

I dedicate this thesis to my mother, Kim Carney, and to my grandmother, Sandra Carney, whose wisdom, guidance, and encouragement helped me persevere through this incredible milestone. Their love, support, prayers, and numerous phone calls formed the foundation upon which this thesis stands.

I also dedicate this work to the faculty, teachers, and peers who supported me through this three-year journey. Their encouragement helped me continue moving forward, even during the hardest moments.

ACKNOWLEDGMENTS

First and foremost, I would like to express my sincerest gratitude to my advisor, Dr. Syed Ali Asad Rizvi, for his mentorship and patience throughout my master's degree. His insight and encouragement have been pivotal in shaping both this research and my growth as a student and engineer.

I am also deeply thankful to the members of my master's committee, Dr. Hasan and Dr. Van Neste, for their time, feedback, and support throughout this process as their perspectives have improved this work and guided me as I continued to refine this thesis.

I gratefully acknowledge Tennessee Technological University and the Department of Electrical and Computer Engineering for providing a conducive environment for research and learning.

Last but not least, I thank everyone who believed in me, prayed for me, and cheered me on through this long but fulfilling journey.

LIST OF FIGURES

List of Tables

List of Algorithms

1. INTRODUCTION

1.1. Background and Motivation

Autonomous and cyberphysical systems are increasingly operating in environments where their performance relies on the decisions of another agent. As a result, in these settings, the control problem is not only to stabilize a plant or track a prescribed reference; rather, the controller must act while another decision maker aims to influence the same state in an opposing direction. Pursuit-Evasion Games provide one of the clearest mathematical representations of this interaction. More specifically, a pursuer attempts to reduce a relative error or drive the state toward a capture condition, while an evader attempts to increase the same cost or prevent the capture condition from being reached. As a result, the pursuit-evasion problem can convert an open-ended adversarial interaction into a controlled two-player game whose state, inputs, objective, and equilibrium conditions are explicitly defined [1], [2].

The linear-quadratic specialization is important in this context since it retains the strategic structure of the game while allowing the equilibrium policies to be computed from matrix equations. To be more precise, a discrete-time linear system with quadratic state and input penalties leads to a two-player zero-sum game in which one player minimizes the performance index while the other maximizes it. The corresponding saddle-point solution is connected to a Game Algebraic Riccati Equation or GARE, and the resulting policies are linear functions of the state. Therefore, this formulation is essential as it gives a transparent model-based baseline whose assumptions, numerical conditions, and closed-loop behavior are able to be examined more directly [1], [3].

On the contrary, the model-based solution is only one side of the control problem. A GARE-based controller requires the system matrices to be known, while many practical systems provide measured input-output or state-transition data without providing an exact analytical model. Model-free quadratic Q-learning addresses this difference by estimating an action-value function from sampled data and recovering the same game policies from the estimated quadratic matrix. Some of the early literature demonstrated that a linear discrete-time zero-sum game could be solved by utilizing model-free Q-learning and that the resulting iterations converge to the nominal Riccati solution under the required excitation and rank conditions [4], [5]. Later work in the literature demonstrates that policy iteration Q-learning methods further develop this idea in the context of data-based two-player zero-sum systems [6].

This distinction between model-based and model-free should not be interpreted as a comparison between an analytical method and an unrelated learning method. Both approaches do solve the same

zero-sum game, use the same quadratic objective, and seek the same pursuer and evader policies. Where these two methods differ is the information that is used during the computation. The model-based methods evaluate or update the value matrix using the known system dynamics, whereas the model-free methods reconstruct the required quadratic relationships from measured transitions. Rizvi et al. organize these ideas through corresponding policy iteration and value iteration formulations for nominal linear quadratic zero-sum games [5]. As a result, this shared structure creates a direct foundation for studying four algorithms under the same problem definition; that is, model-based policy iteration, model-based value iteration, model-free Q-learning policy iteration, and model-free Q-learning value iteration.

In addition, more recent literature aims to refine each part of this framework. Recent model-based literature has developed single-loop policy iteration methods for linear zero-sum games with relaxed initialization requirements [7]. Similarly, recent value iteration research has also established on-policy and off-policy formulations for stochastic zero-sum linear quadratic games with unknown dynamics [8]. Modern learning theory has investigated convergence, last-iterate behavior, and sample complexity for zero-sum linear quadratic games [9]. Therefore, these developments within the literature demonstrate that the linear quadratic zero-sum game is an active area within the current literature rather than solely a historical control benchmark.

Nevertheless, much of the nominal algorithmic literature assumes that the input computed by the controller is the input that enters the plant. This assumption is convenient for the derivation; however, it excludes a common execution-side problem. That is, a persistent difference between a commanded and realized input as such a difference may represent an actuator calibration offset, steering-center shift, longitudinal-command mismatch, sensor-to-command conversion error, or another repeatable implementation bias. Even when the nominal feedback gains stabilize the game, a persistent input offset introduces an affine forcing term into the closed-loop dynamics and may create a nonzero deviation from the nominal trajectory.

This is a critical distinction since nominal convergence of the value matrix does not imply that the realized system will follow the nominal closed-loop response. A policy may satisfy the GARE, the saddle-point curvature conditions, and the closed-loop stability condition while the implemented trajectory still settles away from its nominal equilibrium. More recent fault-tolerant Q-learning literature further demonstrates the current interest in learning-based control under actuator and sensor faults rather than only under ideal input channels [10]. In pursuit-evasion research, the current literature has similarly

moved towards safety, disturbance rejection, sensor limitations, and physical implementation [11]–[13].

Therefore, this thesis direction is intended to preserve the established four algorithm zero-sum framework and extend it with a common persistent command-offset model, bias-estimation procedure, affine compensation law, and MATLAB validation structure. The contribution is not intended to be a novel invention of the underlying policy iteration, value iteration, or Q-learning equations. Instead, the contribution is the systematic development of disturbance-aware versions that remain traceable to the original nominal algorithms while allowing the effect of imperfect command realization to be measured and compensated.

1.2. Problem Statement

This thesis considers a discrete-time two-player zero-sum system in which the pursuer acts through one input channel and the evader acts through another. Thus, the nominal game dynamics can be given as follows

$$x_{k+1} = Ax_k + B_P u_{P,k} + B_E u_{E,k},$$

where $x_k \in \mathbb{R}^n$ denotes the game state, $u_{P,k} \in \mathbb{R}^{m_P}$ denotes the pursuer input, $u_{E,k} \in \mathbb{R}^{m_E}$ denotes the evader input, and A, B_P and B_E are the state and input matrices. The pursuer is the minimizing player, while the evader is the maximizing player.

A representative infinite-horizon quadratic objective is expressed as

$$J = \sum_{k=0}^{\infty} \left(x_k^\top Q x_k + u_{P,k}^\top R_P u_{P,k} - u_{E,k}^\top R_E u_{E,k} \right),$$

where $Q \succeq 0, R_P \succ 0$, and $R_E \succ 0$. Under the nominal saddle-point policies, the two inputs can be written as

$$u_{P,k} = Lx_k, \quad u_{E,k} = Kx_k,$$

where L and K are the pursuer and evader feedback gains. The corresponding closed-loop matrix is $A + B_P L + B_E K$.

The nominal model-based methods obtain the value matrix and the two gains from GARE-based policy iteration or value iteration updates. The nominal model-free methods estimate a quadratic Q-function from

measured data and recover the same policies from the partitions of the estimated action-value matrix. In the algorithms that are used as a starting point for this thesis, these two pairs of methods have different information requirements, but they are directed toward the same saddle-point solution [5].

The practicality that is considered here is that the realized player inputs may differ from their commanded values. The command-channel model is given by

$$u_{P,k}^{\text{act}} = u_{P,k}^{\text{cmd}} + b_P, \quad u_{E,k}^{\text{act}} = u_{E,k}^{\text{cmd}} + b_E$$

where b_P and b_E denote constant or slowly varying offsets over the calibration interval and the following closed-loop run. When no compensation is applied, the realized dynamics then become,

$$x_{k+1} = (A + B_P L + B_E K)x_k + B_P b_P + B_E b_E.$$

As a result, the nominal gains may remain stabilizing while the affine offset changes the steady-state response, the state trajectory, and the accumulated game cost.

The command offsets are estimated according to the information available to each algorithmic class. For the model-based methods, the known system matrices allow a transition residual to be formed from measured states and commanded inputs. For the model-free methods, the offset estimate is obtained from measured command-channel data or from an augmented data regression that does not require the analytical state-transition matrices during the Q-learning update.

Then, the compensated commands are expressed as follows

$$u_{P,k}^{\text{cmd}} = Lx_k - \hat{b}_P, \quad u_{E,k}^{\text{cmd}} = Kx_k - \hat{b}_E.$$

As a result, the compensated closed-loop system then becomes

$$x_{k+1} = (A + B_P L + B_E K)x_k + B_P(b_P - \hat{b}_P) + B_E(b_E - \hat{b}_E).$$

This equation is crucial as it provides the central foundation of this thesis. If the offset estimates are exact, then the nominal closed-loop dynamics are recovered. On the other hand, if the estimates are imperfect, the remaining affine term relies only on the estimation errors rather than on the full command offsets.

The research problem here is consequently not to replace the original GARE or Q-learning algorithms. Instead, it is to determine whether the four established methods are able to be extended under a common disturbance model while their original nominal iteration structures remain identifiable. The extended implementations must also be evaluated using common numerical criteria so that differences caused by the information structure are not confused with differences caused by inconsistent simulation conditions.

1.3. Research Questions and Objectives

There are five key research questions in this thesis that connects the nominal zero-sum solution, the command-offset model, the estimation process, the compensation law, and the MATLAB comparison. These five research questions are as follows:

1) *Nominal Algorithm Reproduction*: Can the model-based policy iteration, model-based value iteration, model-free Q-learning policy iteration, and model-free Q-learning value iteration methods be implemented such that their nominal equations remain traceable to the established zero-sum algorithms from which they are derived?
2) *Persistent Command-Offset Effect*: How do constant pursuer and evader command offsets alter the nominal state response, predicted steady-state deviation, finite-window game cost, and closed-loop trajectory when the feedback gains themselves are unchanged?
3) *Bias Estimation and Compensation*: Can the persistent command offsets be estimated using the information available to the model-based and model-free methods, and can affine input compensation reduce the resulting deviation from nominal behavior?
4) *Matched Four Algorithm Comparison*: How do policy iteration and value iteration compare within the model-based and model-free classes when all four methods use the same game matrices, disturbance definitions, calibration conditions, simulation horizon, and numerical diagnostics?
5) *Pursuit-Evasion and Future Research Directions*: To what extent does the resulting disturbance-aware framework provide a defensible mathematical and computational foundation for later pursuit-evasion studies that involve nonlinear vehicles, QLabs, or physical QCar platforms?

From these research questions, the following five primary research objectives are devised.

- *Objective 1*: Reproduce and document the nominal model-based and model-free policy and value iteration methods without presenting the underlying algorithms as new contributions.

- *Objective 2*: Introduce a common persistent command channel offset model for the pursuer and evader and derive its effect on the realized closed-loop dynamics.
- *Objective 3*: Develop information consistent offset-estimation and affine-compensation procedures for the model-based and model-free implementations.
- *Objective 4*: Establish a shared MATLAB validation protocol using convergence, GARE residual, closed-loop spectral radius, saddle-point curvature, regression rank, conditioning, bias-estimation error, trajectory deviation, steady-state offset, and finite-window cost.
- *Objective 5*: Interpret the resulting zero-sum system as a pursuit-evasion benchmark and identify the precise work that remains before nonlinear vehicle or hardware claims are able to be made.

1.4. Scope, Assumptions, and Delimitations

These are several intentionally restrictive scope decisions. First, the thesis considers one pursuer and one evader. Multi-pursuer and multi-evader games are important within the broader pursuit-evasion literature; however, they introduce coordination, assignment, communication, and scalability questions that would make it more challenging to isolate the disturbance-aware extension of the four algorithms. Therefore, the two-player case is used as the controlled setting in which the model-based and model-free information structures can be compared directly.

Second, the primary algorithmic development is based on discrete-time linear systems and quadratic zero-sum objective. Thus, the linear formulation is not claimed to represent every physical pursuit-evasion system globally. On the contrary, it provides the exact setting in which the GARE, the quadratic Q-function, and the pursuer and evader feedback gains can be compared without ambiguity. Locally linearized nonlinear models may later be connected to this framework, but the mathematical claims in the current thesis are restricted to the linear or locally linearized problem used by each experiment.

Third, the command disturbances are modeled as constant or slowly varying offsets over a calibration batch and the following closed-loop run. Hence, this assumption is intended to represent repeatable execution-side mismatch, not arbitrary high-frequency noise. Time-varying faults, abrupt actuator loss, stochastic drift, and unknown nonlinear dead zones are outside the principal scope. Therefore, these effects are relevant future extensions, but they require estimators and robustness arguments that differ from the persistent affine model being considered here.

Fourth, the model-based methods are allowed to use the analytical system matrices during policy or

value computation and during transition-residual calibration. This distinction is maintained throughout the thesis. At the same time, the model-free designation does not prohibit the usage of measured commanded and realized inputs, since those quantities may be available from the implementation interface even when the state-space matrices are unknown.

Fifth, the thesis focuses on structured quadratic Q-learning as opposed to a broad reinforcement learning algorithm comparison. Deep actor-critic methods, self-play, neural policies, and large-scale multi-agent training are outside of the main scope. Additionally, recent pursuit-evasion research literature demonstrates the usefulness of those methods under nonlinear dynamics and sensing limitations [11], [13]. However, introducing these here would obscure the direct relationship between the learned Q-function and the nominal GARE solution.

Sixth, MATLAB serves as the primary validation environment. This thesis reports numerical convergence, stability, estimation, compensation, and trajectory results and does not claim completed QLabs deployment nor physical QCar validation. The autonomous ground vehicle interpretation remains crucial as an application pathway, however the present work is more theoretical and computational in nature.

Finally, the thesis does not claim that policy and value iteration, GARE theory, or model-free Q-learning are a novel contribution of this work. Rather, the nominal algorithmic equations are established currently within the literature. Therefore, the scope of the contribution is the disturbance-aware extension, the matched implementation across four methods, and the resulting numerical evidence.

1.5. Contributions and Novelty

The first contribution of this thesis is a common disturbance-aware formulation for model-based and model-free zero-sum control. The same pursuer and evader command-offset model is introduced into all four methods, which prevents the comparison from being divided into unrelated experiments. This common formulation makes it possible to identify which differences arise from policy iteration versus value iteration and which differences arise from model knowledge versus sampled-data learning.

The second contribution is the extension of the model-based GARE policy and value iteration implementations with persistent-offset calibration and affine compensation. The nominal policy evaluation, policy improvement, and value-update structure remain the established model-based algorithms. The additional components estimate the command offsets, apply compensation during execution, and quantify the residual deviation when the estimates are imperfect.

Then, the third contribution is the corresponding extension of the model-free quadratic Q-learning policy and value iteration implementations. The Q-learning methods retain the sampled Bellman regression and gain recovery structure of the nominal algorithms. Additionally, the disturbance-aware adds commanded-versus-realized input accounting, offset estimation under the model-free information structure, and compensated closed-loop simulation.

The fourth contribution is a shared numerical validation protocol as each method is evaluated using nominal operation, persistent offsets without compensation, persistent offsets with estimated compensation, and exact compensation as a verification case. The reported diagnostic then distinguish algorithm convergence from game validity and execution performance. More specifically, the MATLAB studies include the value or Q-function iteration history, gain convergence, GARE or Bellman residuals, closed-loop spectral radius, saddle-point curvature, matrix conditioning, feature rank, bias–estimation error, trajectory deviation, predicted steady-state offset, and finite-window game cost.

The fifth contribution is the explicit separation between nominal algorithm validity and disturbance-compensation performance. This separation is crucial since a controller can satisfy the nominal GARE while the realized system experiences an affine input error. Hence, by reporting both the nominal numerical checks and the disturbance-aware trajectory results, the thesis avoids using a single convergence plot as evidence for every aspect of the implementation.

Therefore, the novelty of this thesis lies in the integration and matched evaluation of these components; however, it does not lie in renaming the established algorithms or presenting existing Riccati and Q-learning updates as original. This distinction strengthens the thesis since the reader is able to identify precisely where the literature ends and where the disturbance-aware development begins.

1.6. Connections to the Present Literature

There are six closely connected bodies of literature that support this thesis. The first is linear-quadratic zero-sum dynamic game theory as this literature establishes the minimax problem, quadratic value function, the feedback saddle-point policies, and the GARE conditions that are used by the model-based solution [1], [3]. In addition, it also provides the theoretical connection between a controller acting as the minimizing player and an adversarial disturbance or evader acting as the maximizing player.

The second body of literature relates to model-based policy and value iteration for zero-sum games. The classical iterative structure is developed from policy evaluation and improvement, as well as repeated

Riccati mappings. Recent studies continue to research initialization, single-loop implementation, and convergence of GARE iterations [7]. These results support two model-based algorithms while also demonstrating that their numerical implementation remains an active area of research.

The third body of literature concerns quadratic Q-learning and adaptive dynamic programming. Foundational discrete-time work has established that the zero-sum game policies could be learned without direct knowledge of the system matrices [4]. In addition, output feedback and data-based developments then expanded the available information structures and demonstrated policy iteration learning from measured data [6], [14]. Rizvi et al. specifically provide the direct organization of policy iteration and value iteration methods that are used as the nominal algorithmic foundation for this thesis [5].

The fourth body of literature focuses on current learning and optimization results for zero-sum linear-quadratic games. Recent studies have examined on-policy and off-policy value iteration with unknown dynamics, as well as sample complexity and last-iterate convergence for model-free learning [8], [9]. This is an essential body of literature for the thesis since it positions this thesis within the current development of structured zero-sum learning as opposed to within a generic reinforcement learning comparison.

The fifth body of literature covers actuator faults, persistent input mismatch, estimation, and compensation. Fault-tolerant Q-learning methods increasingly aim to preserve stability and optimality when the realized system contains actuator or sensor faults [10]. This thesis aims to use a narrower constant-offset model, however this literature supports the larger motivation that learning and control algorithms need to be evaluated under imperfect execution channels as opposed to only under nominal equations.

The sixth and final body of literature concentrates on pursuit-evasion games and their modern applications. Pursuit-evasion has been demonstrated to provide the adversarial interpretation of the two-player zero-sum system [2]. Moreover, recent surveys prove that learning-based pursuit-evasion research now spans unmanned systems, navigation, and spacecraft applications [11]. Current research also addresses safety, stability, disturbances, sensing limits, and car-like dynamics [12], [13]. These papers provide the application context while this thesis remains focused on a controlled linear-quadratic foundation.

1.7. Significance

This thesis has both a methodological as well as practical significance. More specifically, its methodology creates a direct comparison among four algorithms that are typically discussed separately. Model-based policy and value iteration, as well as Q-learning policy and value iteration are placed within the

same system, cost, disturbance, and reporting structure. As a result, the comparison does not depend on changing the game definition from one algorithm to another.

Additionally, this thesis is also significant due to its treatment of command realization as part of the control problem. In many numerical studies, the commanded input is inserted directly into the state equation. This convention is useful for analyzing the nominal algorithm; however, it can hide a repeatable implementation mismatch. A persistent offset does not need to stabilize the system in order to matter as it may instead produce a nonzero state deviation, change the game cost, or alter the interpretation of the pursuer and evader policies. The compensation framework makes this effect explicit.

Another point of significance is the separation of information structures. The model-based methods use known dynamics, while the model-free methods recover the game policies from data. Hence, by applying the same command-offset experiment to both classes provides a more meaningful comparison rather than simply comparing their nominal iteration counts. It reveals the additional questions that are introduced by data collection, feature rank, regression conditioning, and offset measurement.

This thesis also intends to be reproducible. As a result, each MATLAB implementation records its parameters, convergence, histories, numerical checks, calibration results, trajectories, and summary metrics. The reproducibility that is provided by this thesis is crucial since it allows for later work to replace the benchmark matrices with a vehicle model without changing the logic of the experiment. More specifically, scalar command offsets can later become acceleration and steering offset vectors, while the nominal, uncompensated, estimated compensation, and exact compensation cases remain unchanged.

Finally, the thesis provides a defensible intermediate contribution. That is, it is more complete than a nominal GARE example since it includes disturbance estimation and compensation across model-based and model-free methods. At the same time, it is useful because it specifies which results are established in MATLAB and which results still require nonlinear simulation, QLabs integration, or physical hardware testing.

1.8. Working Hypotheses

Although this thesis is comparative as opposed to be being purely confirmatory, several working hypotheses guide the algorithm development and numerical experiments. These hypotheses are as follows:

1) *Nominal Consistency Hypothesis*: When the four algorithms are implemented under compatible assumptions and sufficient numerical conditions, the model-based and model-free methods will

approach feedback policies that are associated with the same nominal zero-sum solution.

2) *Persistent-Offset Hypothesis*: Constant pursuer and evader command offsets will introduce a measurable affine deviation from the nominal trajectory even when the nominal closed-loop matrix remains Schur stable.
3) *Exact Compensation Hypothesis*: When the true command offsets are subtracted from the command inputs, the realized trajectory will recover the nominal closed-loop trajectory to numerical precision.
4) *Estimation Compensation Hypothesis*: When the offsets are estimated with little error, affine compensation will reduce the steady-state and trajectory deviation by an amount consistent with the residual bias terms $b_P - \hat{b}_P$ and $b_E - \hat{b}_E$.
5) *Information Structure Hypothesis*: The model-free implementations will require stronger data-rank, excitation, and conditioning diagnostics than the model-based implementations since the quadratic game relationships need to be reconstructed from sampled transitions.
6) *Transfer-Foundation Hypothesis*: A disturbance-aware framework that is verified first on a controlled zero-sum linear system will provide a better defended foundation for later vehicle experiments than transferring a nominal controller directly to a platform without an explicit command-mismatch model.

These hypotheses do not predetermine that one algorithmic class must outperform the other. That is, a model-based method may converge more directly since the matrices are known, while a model-free method could remain valuable because it does not need those matrices in the learning update. Hence, the purpose of the experiments is to document these differences under matched conditions rather than to force a predetermined ranking.

1.9. Thesis Organization

The rest of this thesis is organized as follows. Chapter 2 reviews the literature on zero-sum linear-quadratic games, GARE policy and value iteration, quadratic Q-learning, persistent input bias, disturbance estimation, affine compensation, numerical well-posedness, and current pursuit-evasion control. The chapter aims to separate foundational algorithm sources from recent state-of-the-art developments such that the historical basis and the current research direction remain clearly defined.

Chapter 3 develops the technical background and common disturbance-aware game formulation. Moreover, it defines the discrete-time two-player zero-sum system, joint quadratic Q-function, persistent command-channel offsets, estimation assumptions, compensation law, and common numerical criteria.

Then, Chapter 4 presents the model-based methods. The original policy iteration and value iteration structures are documented first, after which the persistent-offset calibration, affine compensation, MATLAB simulation cases, and numerical diagnostics are added. In addition, the chapter distinguishes the changes to the experiment from the underlying algorithms.

Next, Chapter 5 presents the model-free methods. This chapter develops the quadratic Q-function representation, sampled Bellman equations, feature construction, least-squares regression, policy recovery, and the corresponding policy iteration and value iteration procedures. The same command-offset model and compensation cases are then applied under the model-free information structure.

Chapter 6 reports the MATLAB results and cross-method comparison. It aims to evaluate convergence, game validity, closed-loop stability, regression quality, bias estimation, uncompensated behavior, estimated compensation, exact compensation, and finite-window game cost. In addition, the chapter uses matched tables and figures so that the four algorithms are able to be compared without changing the benchmark definition.

Finally, Chapter 7 concludes this thesis by summarizing the main findings, answering the previously stated research questions, identifying limitations, and to define potential future research directions. Some potential future research directions include nonlinear kinematic bicycle models, vector-valued acceleration and steering offsets, online bias estimation, output feedback implementation, actuator saturation, time delay, QLabs simulation, physical QCar testing, and multi-pursuer or multi-evader extensions.

2. Literature Review

2.1. Introduction

The literature that supports disturbance-aware zero-sum pursuit-evasion control is distributed across several closely related areas as opposed to a single field. The mathematical foundations begins with differential games and linear-quadratic dynamic games, while the computational foundation is divided between model-based Riccati methods and model-free quadratic Q-learning. Then, the disturbance-aware component draws from persistent-input estimation, unknown-input reconstruction, offset-free control, and affine compensation. Finally, pursuit-evasion research provides the adversarial interpretation that links the minimizing and maximizing players to a pursuer and an evader. As a result, the literature review needs to establish not only that each area is active, but also how these areas connect into one coherent thesis problem.

The current direction differs from a broad comparison between classical control and deep reinforcement learning. More specifically, the four algorithms that are considered in this thesis are structurally related. Model-based policy and value iteration use the known system matrices to solve the nominal game, while model-free Q-learning policy and value iteration aim to find the corresponding game policies from sampled transitions without using those matrices directly in the learning update. As a result, the methods differ primarily in their information structure and computational procedure, not in the game being solved. Rizvi et al. provide the immediate algorithmic organization for this four-method structure, while earlier zero-sum Q-learning research establishes the model-free connection between the quadratic action-value function and the Game Algebraic Riccati Equation (GARE) [4], [5].

A second distinction is that the thesis does not treat the nominal algorithms as complete representations of implementation behavior. Most derivations assume that the input computed by a policy is the input that enters the system. Hence, this assumption allows the Riccati and Bellman equations to be written cleanly, but it excludes persistent command-channel mismatch. A constant acceleration bias, steering-center shift, actuator calibration error, or repeated difference between a commanded and realized input introduces an affine term into the closed-loop dynamics. The nominal feedback gains may remain stabilizing while the realized trajectory no longer converges to the nominal equilibrium. This observation, as a result, motivates the inclusion of literature on disturbance models, simultaneous input and state estimation, and offset-free control [15]–[18].

As a result, the literature is organized around six connected questions. First, what conditions define a solvable two-player zero-sum linear-quadratic game? Second, how do model-based policy iteration and value iteration compute the associated saddle solution? Third, how do quadratic Q-learning methods recover equivalent policies from sampled data? Fourth, how can persistent command offsets be represented and estimated without changing the nominal game definition? Fifth, how does pursuit-evasion provide a meaningful interpretation for the minimizing and maximizing players? Sixth, which numerical checks are necessary to distinguish a valid saddle solution from an iteration that merely stops changing?

This intentional organization gives the greatest focus to the literature that directly supports the four algorithms and the disturbance compensation extension. Autonomous ground vehicle and QCar sources remain relevant as a potential application direction, however these papers are not the focus of this review. The present contribution is established first on controlled discrete-time zero-sum systems such that the original algorithms, the added bias model, and the resulting compensation behavior remain mathematically traceable. Nonlinear vehicle dynamics, acceleration and steering bias vectors, QLabs simulation, and physical QCar testing are relegated to potential research directions rather than completed validation.

2.2. Zero-Sum Linear-Quadratic Dynamic Games and the GARE

Differential Game Theory formalizes decision problems in which several players influence the same dynamical system while pursuing different objectives. Isaacs established the classical pursuit-evasion and differential game framework, including the interpretation of opposing players and terminal capture conditions [19]. Later, Basar et al. developed the general theory of dynamic noncooperative games, including zero-sum, Nash, open-loop, and feedback formulations [1]. These foundational works are crucial since the language that is used in this thesis, such as the player, policy, value, saddle-point, and zero-sum objective, all originate from this game-theoretic structure.

The zero-sum case is distinguished by a single performance index that one player attempts to minimize and the other attempts to maximize. For a discrete-time linear system, the nominal dynamics can be represented by the equation

$$x_{k+1} = Ax_k + B_P u_{P,k} + B_E u_{E,k}, \tag{2.1}$$

where x_k denotes the game state, $u_{P,k}$ denotes the minimizing-player input, and $u_{E,k}$ denotes the maximizing-player input. A representative infinite-horizon quadratic cost can be defined by the equation

$$J = \sum_{k=0}^{\infty} \left(x_k^\top Q x_k + u_{P,k}^\top R_P u_{P,k} - u_{E,k}^\top R_E u_{E,k} \right), \tag{2.2}$$

where $Q \succeq 0, R_P \succ 0$, and $R_E \succ 0$. The sign difference between the two input penalties reflects the minimax structure. The pursuer attempts to reduce the state and its own control effort, while the evader acts to increase the same game value within the admissible saddle conditions.

The linear-quadratic structure is crucial, since it produces policies that are linear in the state and a value function that is quadratic. More specifically, if a stabilizing saddle solution exists, then the value can be expressed as

$$V(x_k) = x_k^\top P x_k, \tag{2.3}$$

and the policies that take the form of

$$u_{P,k} = L x_k, \quad u_{E,k} = K x_k. \tag{2.4}$$

The resulting closed-loop matrix is $A + B_P L + B_E K$. Pachter et al. provide a discrete-time treatment of linear-quadratic dynamic games that is especially relevant because the MATLAB algorithms contained in this thesis operate in sampled time as opposed to through continuous-time differential equations [3]. The discrete-time formulation also makes the relationship between the Bellman equation, the GARE, and quadratic Q-learning more apparent.

The Game Algebraic Riccati Equation, or GARE, is the central matrix equation for the nominal model-based problem. Moreover, its block structure contains the interactions among the state, minimizing input, and maximizing input. Unlike the single-player LQR problem, the zero-sum game needs to satisfy both a minimizing-player curvature condition and a maximizing-player curvature condition. The minimizing-player control block must be positive definite, while the appropriate maximizing-player Schur complement must be negative definite. These conditions are not secondary numerical conveniences; rather, they determine whether the stationary point obtained from the joint quadratic form is a saddle point instead of a minimum in both inputs or an indefinite stationary point.

The pursuit-evasion specialization of linear-quadratic games has a long history. Ho established an early analytical result for LQ pursuit-evasion differential games, while Bagchi et al. extended the setting to

stochastic disturbances [20], [21]. Shinar examined solvability conditions for LQ differential games associated with pursuit-evasion problems, demonstrating that an apparently reasonable quadratic formulation does not automatically guarantee the existence of an admissible saddle solution [22]. These results support the decision in this thesis to report curvature and stability checks explicitly rather than infer validity from the existence of a numerical matrix alone.

In addition, more recent research continues to study the distinction between open-loop and feedback equilibria and the behavior of zero-sum LQ games under stochastic or long-horizon conditions. Sun et al. distinguish open-loop and closed-loop saddle points in stochastic LQ differential games, while Sun later develops the two-person stochastic zero-sum case in greater detail [23], [24]. Monti et al. examine feedback and open-loop Nash equilibria for infinite-horizon discrete-time dynamic games, which reinforces the importance of stating which equilibrium concept is implemented by an algorithm [25]. Sun and Yong's more recent long-time analysis further demonstrates that the LQ game remains an active theoretical setting rather than only a historical benchmark [26].

The current learning literature also treats zero-sum LQ games as a demanding benchmark. Wu et al. emphasize that the corresponding optimization problem is nonconvex and nonconcave and develop improved sample-complexity and last-iterate convergence results [9]. This work is crucial for this thesis since it places the model-free algorithms within the current state of research while also clarifying that convergence claims depend on the precise algorithm, information assumptions, and admissible policy region. This thesis does not aim to reproduce that sample-complexity theory. Instead, it uses the established LQ game structure to compare the four algorithms under a shared persistent command-offset model.

Therefore, the Game Algebraic Riccati Equation (GARE) literature establishes three requirements for the remainder of the thesis. First, the nominal problem must be stated as a genuine minimax game rather than two independent controllers. Second, the feedback gains must be associated with a stabilizing value matrix and valid saddle curvature. Third, any disturbance-aware extension should preserve the nominal GARE structure if the thesis claims to extend rather than replace the original algorithms. These requirements provide the basis for the model-based policy iteration and model-based value iteration literature that is in the following section.

2.3. Model-Based Policy and Value Iteration

Policy Iteration and Value Iteration are related dynamic programming procedures, but they should not be treated as interchangeable names for the same computation. Policy iteration alternates between evaluating fixed policies and improving those policies, while value iteration repeatedly updates the value function through a Bellman or Riccati mapping and then obtains the policies from the current value estimate. The two procedures may converge to the same nominal solution under compatible assumptions, but their intermediate matrices, initialization requirements, and numerical behavior may and can differ.

The policy iteration lineage is connected to iterative Riccati methods developed for optimal regulation. Kleinman introduced an iterative Riccati technique that is based on repeated Lyapunov equations, and Hewer developed a discrete-time steady-state gain iteration [27], [28]. Although these studies concern single-player regulation, they establish the basic computational principle that a nonlinear Riccati equation is able to be approached through a sequence of linear matrix equations associated with current policies. Mageirou extended iterative ideas to Riccati game equations, making the link to dynamic games explicit [29].

For a zero-sum game, policy evaluation is performed with the current minimizing and maximizing gains fixed. If L_i and K_i are the policies at iteration i, then the current closed-loop matrix is given by

$$A_i = A + B_P L_i + B_E K_i. \tag{2.5}$$

Provided the fact that A_i is Schur stable, then the associated value matrix can be obtained from a discrete Lyapunov equation. Policy improvement then updates both gains from the blocks of the game quadratic form. The two improvements are coupled because the pursuer gain relies on the evader blocks and the evader gain depends on the pursuer blocks. As a result, this coupling is one reason that a zero-sum policy iteration cannot be reduced to two independent LQR updates.

Abu-Khalaf et al. developed policy iterations for the Hamilton-Jacobi-Isaacs equation in H_∞ state-feedback control with input saturation, demonstrating how the game interpretation connects disturbance attenuation and minimax control [30]. Wu et al. later proposed simultaneous policy updates for learning continuous-time H_∞ control solutions [31]. These studies are relevant since the pursuer-vs-evader interpretation and the controller-versus-disturbance interpretation share the same zero-sum mathematical structure, even when the physical meanings of the two players differ.

Rizvi et al. organize model-based policy iteration and value iteration alongside their model-free counterparts for linear systems [5]. As a result, this organization is the direct starting point for the algorithmic structure in this thesis. The model-based policy iteration code sourced from the author uses known matrices to evaluate a closed-loop policy and then improves the two gains. The model-based value iteration source code repeatedly applies the GARE mapping and then computes the policies associated with the updated value matrix. The present disturbance-aware work retains these roles. The command-offset estimation and compensation procedures are added to the execution and evaluation layers rather than inserted into the nominal Riccati equations without justification.

In addition, recent research has continued to refine zero-sum policy iteration. Zhao et al. propose a single-loop policy iteration method for linear zero-sum games and establish convergence for the resulting GARE sequence [7]. The importance of this study is not that every earlier policy iteration is obsolete; rather, it demonstrates that initialization, loop structure, and convergence proof remain active concerns. Similarly, Li et al. develop a relaxed policy iteration algorithm for nonlinear zero-sum games with application to H_∞ control [32]. Nortmann et al. examine iterative and data-driven algorithms for linear-quadratic discrete-time dynamic games, further demonstrating the current interest in linking model-based game solutions to data-driven computation [33].

On the other hand, value iteration follows different computational logic. Starting from an initial value matrix, the algorithm repeatedly applies a Riccati or Bellman operator. The policies are functions of the current value matrix rather than fixed during a separate evaluation step. Bertsekas provides the broader dynamic programming foundation for this distinction [34]. In zero-sum games, the value update must preserve the minimax block structure and remain within a region where the necessary matrix inverses or linear solvers are well-defined.

Guo et al. formulate on-policy and off-policy value iteration algorithms for stochastic zero-sum dynamic games with unknown dynamics [8]. Their research is crucial, since it treats both model-based convergence and model-free forms within one value iteration setting. Moreover, it also shows that the line between model-based and model-free computation is determined by the information used in the update, not simply by whether the word *value iteration* appears in the algorithm title.

Therefore, the literature suggests several principles for this thesis. First, policy iteration and value iteration should be implemented and reported separately. Second, changes that are made for numerical robustness, such as replacing explicit matrix inversion with a linear solve, should be identified as

implementation choices rather than novel control laws. Third, the original update equations should remain recognizable when the disturbance-aware layer is added. Fourth, convergence histories should report changes in the value matrix and both gains since a small value update does not automatically prove that all policy quantities have converged.

These principles also clarify the role of the model-based algorithms in the final comparison. They are not used merely to generate a single reference matrix, and they provide two established procedures with different iteration structures but the same nominal system knowledge. The disturbance-aware study then asks whether the same command-offset model and compensation logic can be applied without altering what each nominal algorithm does.

2.4. Model-Free Quadratic Q-Learning and Adaptive Dynamic Programming

Model-free control is often discussed as if it only refers to deep neural network policies trained through long episodes; however, that framing is too broad for this thesis. The model-free methods that are considered here are structured quadratic Q-learning algorithms for linear zero-sum systems. The purpose of these model-free methods is to recover the value and policy information that is needed by the game without inserting the analytical state-transition matrices into the learning update.

Watkins et al. established Q-learning as a model-free temporal difference method for estimating optimal action values [35]. In addition, Sutton and Barto provide the broader reinforcement learning framework, including value functions, Bellman equations, exploration, and temporal difference learning [36]. For linear-quadratic control, however, the action-value function has additional structure. Bradtke et al. developed linear least-squares temporal difference methods, and then Lagoudakis et al. later developed least-squares policy iteration [37], [38]. These studies establish the general connection between sampled transitions, linear parameter estimation, and policy improvement.

In a two-player zero-sum linear-quadratic game, the joint state-action vector can be written as

$$\zeta_k = \begin{bmatrix} x_k^\top & u_{P,k}^\top & u_{E,k}^\top \end{bmatrix}^\top, \tag{2.6}$$

and the quadratic action-value function is represented by

$$Q_H(\zeta_k) = \zeta_k^\top H \zeta_k, \tag{2.7}$$

where H is said to be symmetric. The state, pursuer-input, evader-input, and cross-coupling blocks of H contain the information needed to recover the two saddle policies. Thus, the learner estimates a finite set of quadratic coefficients rather than an unrestricted nonlinear function.

Al-Tamimi et al. provide foundational adaptive critic and model-free Q-learning designs for discrete-time zero-sum games with application to H_∞ control [4], [39]. Their work is important since it demonstrates that the GARE solution can be recovered from data through a quadratic Q-function without direct knowledge of the system matrices. Additionally, Lewis et al. place these ideas within adaptive dynamic programming and feedback control more generally [40]. These sources form the conceptual link from nominal Riccati computation to the model-free algorithms in this thesis.

Policy iteration Q-learning evaluates the Q-function associated with current pursuer and evader policies and then improves the policies from the estimated matrix partitions. Luo et al. develop a policy iteration Q-learning method for data-based two-player zero-sum linear discrete-time systems [6]. The method emphasizes that the data matrix must contain sufficient independent quadratic information. In practical terms, the samples must excite the state and both input channels strongly enough that the symmetric parameters of H can be identified.

Finite-horizon and output-feedback extensions broaden this structured learning framework. Liu et al. develop finite-horizon Q-learning for discrete-time zero-sum games and later output-feedback Q-learning for finite-horizon problems [41], [42]. These are relevant studies since they show that the quadratic zero-sum learning structure is not limited to one infinite-horizon full-state formulation. Lian et al. consider off-policy inverse Q-learning for unknown antagonistic systems, while Guo et al. develop Q-learning for stochastic zero-sum games [43], [44].

Recent studies extend similar ideas toward nonlinear, switched, delayed, and safety-constrained systems. Wang et al. propose neural Q-learning for discrete-time nonlinear zero-sum games with an adjustable convergence rate [45]. Additionally, Wang et al. also develop a parallel off-policy Q-learning method for discrete-time Markov jump systems [46]. Jiang et al. address delayed two-player zero-sum games through data-based Q-learning, and Liu et al. study safe reinforcement learning for nonlinear zero-sum games with unknown state constraints and asymmetric input constraints [47], [48]. These papers represent recent developments, but they also help to define the boundary of the present work. The thesis does not introduce neural approximators, Markov jumps, unknown safety sets, or delayed dynamics into the four baseline algorithms. Instead, it develops a controlled persistent-offset extension whose effect can be isolated.

The literature also distinguishes structured zero-sum Q-learning from policy gradient optimization. Zhang et al. establish convergence results for policy optimization in zero-sum LQ games, while Wu et al. later improve sample-complexity and last-iterate guarantees [9], [49]. These papers are essential state-of-the-art references; however, they do not replace the quadratic Q-learning algorithms used as the thesis starting point. In addition, they demonstrate that zero-sum LQ learning can be approached through different computational routes, each with different theoretical assumptions.

There are several numerical requirements that follow directly from the model-free literature. The feature matrix needs to have sufficient rank to identify the symmetric Q-function parameters. The regression matrix must remain adequately conditioned; moreover, the behavior policies need to provide excitation without driving the system outside the region in which the data are meaningful. The estimated game matrix must satisfy the same minimizing and maximizing curvature requirements used by the model-based solution. Finally, a small Bellman residual should be interpreted together with gain error, rank, conditioning, and closed-loop behavior rather than used as a stand-alone proof of policy recovery.

The present thesis uses these principles to keep the model-free comparison disciplined. The model-free label refers to the absence of the analytical transition matrices from the Q-learning update. It does not imply that no measurements, no prior structural assumptions, or no commanded-versus-realized input data are available. This is an important distinction when persistent command offsets are introduced, since the policy may be learned without the matrices while the bias estimate may still use measured actuator-channel information.

2.5. Persistent Command Bias, Unknown-Input Estimation, and Affine Compensation

The disturbance-aware contribution starts with a simple but consequential observation. That is, a control command a realized control input are not always identical. A persistent mismatch may be caused by actuator calibration, steering-center error, longitudinal response offset, conversion scaling, or another repeatable implementation effect. If the mismatch remains approximately constant over a calibration batch and the following run, then it can be represented as an additive command-channel bias.

For the two-player game, the realized inputs are modeled as

$$u_{P,k}^{\text{act}} = u_{P,k}^{\text{cmd}} + b_P, \quad u_{E,k}^{\text{act}} = u_{E,k}^{\text{cmd}} + b_E. \tag{2.8}$$

When the nominal policies are applied without compensation, the closed-loop dynamics become

$$x_{k+1} = (A + B_P L + B_E K)x_k + B_P b_P + B_E b_E. \tag{2.9}$$

The additional term is affine and persistent. If the nominal closed-loop matrix is Schur stable, then the state need not diverge. Instead, it may approach a nonzero equilibrium determined by the bias and the closed-loop resolvent. This behavior is essential since a nominal stability result can coexist with a meaningful trajectory error.

The offset-free control literature provides a mature foundation for representing this problem. Muske et al. develop disturbance modeling for offset-free linear model predictive control, while Pannochia et al. develop disturbance models that remove steady-state tracking offset [15], [16]. Maeder et al. provide a systematic treatment of linear offset-free MPC, and Jimoh et al. compare different offset-free formulations [50], [51]. Although these studies are primarily written in the MPC setting, the underlying principle is directly relevant. More specifically, persistent mismatch should be represented explicitly rather than treated as random numerical noise.

The unknown-input estimation literature addresses the related problem of reconstructing unmeasured forcing terms. Gillijns et al. derive unbiased minimum-variance input and state estimators for linear discrete-time systems [17]. Later, Yong et al. provide a unified filter for simultaneous input and state estimation [18]. These methods are more general than the batch constant-bias estimator that are used in the present model-based MATLAB benchmark. Nevertheless, they establish the identifiability and estimation principles that justify treating a persistent command offset as an estimable input rather than an arbitrary unexplained deviation.

For the known-model case, the transition residual is expressed by

$$r_k = x_{k+1} - Ax_k - B_P u_{P,k}^{\mathrm{cmd}} - B_E u_{E,k}^{\mathrm{cmd}}. \tag{2.10}$$

Under the bias model, this residual satisfies

$$r_k = \begin{bmatrix} B_P & B_E \end{bmatrix} \begin{bmatrix} b_P \\ b_E \end{bmatrix} + \eta_k, \tag{2.11}$$

where η_k denotes measurement noise, process noise, and finite-precision effects. A batch least-squares estimate is possible when the combined bias map has sufficient rank. This requirement is physically

meaningful. If the two input channels affect the state in indistinguishable directions, the individual player biases cannot be separated from state-transition alone.

Once the offsets are estimated, affine command compensation subtracts the estimates from the nominal policy outputs:

$$u_{P,k}^{\mathrm{cmd}} = Lx_k - \hat{b}_P, \quad u_{E,k}^{\mathrm{cmd}} = Kx_k - \hat{b}_E. \tag{2.12}$$

The realized dynamics then become

$$x_{k+1} = (A + B_P L + B_E K)x_k + B_P(b_P - \hat{b}_P) + B_E(b_E - \hat{b}_E). \tag{2.13}$$

Exact estimates recover the nominal closed-loop dynamics, while imperfect estimates leave only the residual bias. This relationship provides a direct interpretation of the numerical results. Compensation performance should be consistent with the remaining estimation error rather than described as an unexplained improvement.

Recent studies continues to research identification and offset-free stability under more general conditions. Berberich et al. develop data-driven tracking MPC for nonlinear systems, showing how steady-state tracking and model-free data can be connected [52]. Kuntz and Rawlings examine maximum-likelihood identification with integrating disturbances and offset-free stability under plant-model mismatch [53], [54]. Yang et al. consider joint state and bias estimation for nonlinear systems under non-Gaussian process noise [55]. These studies extend beyond the constant-bias benchmark, however they demonstrate that bias estimation and offset-free control remain active research topics.

In addition, the literature also clarifies what is and is not claimed by the present compensation method. The estimator is not a universal unknown-input observer, and it does not guarantee identification under arbitrary time-varying disturbances. The compensation law does not redesign the saddle policy nor solve a new robust game. Rather, it assumes that the nominal policy has already been obtained and that the persistent command mismatch can be estimated over a calibration interval. Thus, the resulting affine correction is best described as a disturbance-aware execution layer attached to an established nominal algorithm.

This distinction is crucial to the thesis contribution. The policy iteration, value iteration, and Q-learning equations retain their original mathematical roles. The added work defines how the commanded policies

are translated into realized inputs, how persistent offsets are estimated under the information available to each methods, and how compensation quality is measured through nominal, uncompensated, estimated-compensation, and exact-compensation cases.

2.6. Pursuit-Evasion Control and Current Disturbance-Aware Learning

Pursuit-evasion provides the physical interpretation for the two-player zero-sum game. The pursuer's objective is to reduce a relative-state cost or drive the engagement into a capture set, while the evader attempts to increase the same objective or delay capture. Weintraub et al. provide a modern tutorial that links pursuit-evasion differential games to guidance, reachability, and control applications [2]. Bopardikur et al. demonstrate how sensing limitations alter discrete-time pursuit-evasion behavior, which is relevant because the current algorithms are implemented in sampled time and later extensions may not assume complete measurements [56].

The classical pursuit-evasion literature often focuses on geometric barriers, terminal capture conditions, or specific vehicle classes. The present thesis uses pursuit-evasion in a narrower way. The minimizing player is interpreted as the pursuer and the maximizing player as the evader, however the initial MATLAB studies are designed to validate the zero-sum algorithms and the command-offset extension before introducing a full nonlinear engagement. As a result, this approach preserves the application meaning while avoiding a claim that a low-dimensional benchmark reproduces every property of a car, aircraft, spacecraft, or mobile robot.

Recent surveys demonstrate that reinforcement learning has become increasingly common in pursuit-evasion research. Yang et al. review reinforcement learning approaches across pursuit-evasion applications and identify value-based, policy gradient, multi-agent, and hybrid methods [11]. Their review is useful for placing the current thesis within the field; however, the thesis does not attempt to survey or compare all modern reinforcement learning architectures. The focus remains on quadratic Q-learning methods that preserve a direct connection to the zero-sum Riccati solution.

Adaptive dynamic programming (ADP) has also been applied directly to pursuit-evasion problems. Gong et al. develop an ADP solution for agent pursuit-evasion, and Zhang et al. propose fixed-time zero-sum pursuit-evasion control for multi-satellite systems through ADP [57], [58]. Xu et al. develop integral reinforcement learning based optimal pursuit-evasion control for multiple quadrotor vehicles, while Li et al. combine ADP-based pursuit-evasion control with obstacle avoidance [59], [60]. Hence these studies

demonstrate that structured game learning methods remain active even as deep reinforcement learning becomes more visible.

More recent literature has also moved in a direction toward constraints, safety, uncertainty, and physical realism. Kokolakis et al. study safety-aware pursuit-evasion in unknown environments using Gaussian processes and finite-time convergent reinforcement learning [61]. Additionally, Gonultas et al. investigate learning under dynamic and sensor constraints and include physical robotic evaluation [13]. Wang et al. develop a self-play iteration for aircraft pursuit-evasion games, while Chen et al. examine online multi-UAV planning in unknown environments [62], [63]. Wang et al. also further emphasize safety, stability, and robustness in pursuit-evasion differential games [12].

These more recent papers identify the broader frontier, but also help define the present research gap. Many pursuit-evasion learning studies introduce nonlinear dynamics, obstacles, sensing limitations, self-play, or multi-agent coordination. Those are important additions, however they can make it difficult to isolate whether performance differences originate from the game solution method the approximation architecture, the exploration procedure, or the environment. Thus, the current thesis takes a controlled complementary approach. It begins with a shared linear-quadratic game, compares four closely related algorithms, and introduces one structured implementation disturbance whose mathematical effect can be stated exactly.

Additionally, the pursuit-evasion literature supports the longer-term vehicle direction. A later autonomous ground vehicle implementation can interpret b_P and b_E as vectors containing acceleration and steering offsets. The nominal state can be replaced by relative position, heading, and speed errors that are derived from kinematic bicycle dynamics. Rajamani provides the vehicle dynamics formulation for such an extension, while Tiriolo et al. demonstrate the usage of a QCar platform for input-constrained autonomous vehicle control [64], [65]. Quanser documentation then defines the practical command and measurement interfaces for future QCar experiments [66]–[69]. These sources remain relevant, but they support future transfer as opposed to the principal theoretical claims of the present chapter.

As a result, pursuit-evasion serves three roles in this thesis. First, it gives the minimizing and maximizing players a consistent interpretation. Second, it motivates why disturbances affecting either input channel matter strategically rather than only as single-agent tracking errors. Third, it provides a clear pathway from the linear benchmark to later vehicle experiments. The literature does not require the thesis to claim completed hardware validation in order for the pursuit-evasion interpretation to be meaningful.

2.7. Numerical Well-Posedness and Reproducible MATLAB Validation

The literature review would be incomplete if it treated numerical implementation as a separate programming issue. The four algorithms rely on matrix solves, Lyapunov equations, quadratic regressions, block Hessians, and repeated gain updates. A mathematically valid equation may still produce misleading numerical results if the relevant matrices are singular, poorly conditioned, or evaluated outside the admissible policy region.

There are three important checks for the model-based methods. First, the current closed-loop matrix used during policy evaluation must be Schur stable such that the discrete Lyapunov equation has the intended stabilizing solution. Second, the final GARE method must be small relative to the scale of the value matrix. Third, the game curvature conditions must hold. The pursuer block must be positive definite, and the evader Schur complement must be negative definite. A low iteration difference without these checks does not establish a valid saddle solution.

For the model-free methods, the numerical requirements are broader. The quadratic feature matrix needs to have sufficient column rank. Also, the least-squares problem must be adequately conditioned. The estimated H matrix should be symmetrized only to remove roundoff-level asymmetry rather than to hide a structurally inconsistent regression. The recovered policies must satisfy the game curvature conditions, and the resulting closed-loop dynamics must be checked. Horn and Johnson provide the matrix analysis foundation for eigenvalue, definiteness, Schur-complement, and conditioning arguments [70].

Thus, the usage of reciprocal condition estimates and linear solves is part of the numerical verification. Replacing an explicit inverse by MATLAB's backslash operator does not change the underlying equation. Instead, it changes how the equation is evaluated. This is an important distinction when documenting extensions to existing code. A numerically safer solve should not be described as a new algorithm, while a change to the order or meaning of the policy updates would be algorithmic.

The disturbance-estimation problem adds its own rank and conditioning requirements. The combined bias matrix $\begin{bmatrix} B_P & B_E \end{bmatrix}$ must distinguish the two command channels. The calibration residual should be reported such that an accurate state trajectory is not attributed to a bias estimate that was never validated. The exact compensation case provides an additional implementation check. More specifically, when the true biases are subtracted, the nominal and compensated trajectories should agree to numerical precision. If they do not, then the compensation has been inserted inconsistently with the realized input model.

Reproducibility further requires that the MATLAB scripts record more than the final gains. The relevant outputs consist of convergence histories, final residuals, curvature values, spectral radius, true and estimated biases, calibration error, nominal and disturbed trajectories, predicted steady-state offsets, finite-window cost, and exact compensation error. Hence, by saving figures, summary tables, and complete workspaces allows the numerical claims to be traced back to one fixed run.

This approach is consistent with the broader movement toward reproducible benchmark studies. The value of the linear zero-sum problem is not that it is the most complicated pursuit-evasion environment. Its value is that the expected structure is known well enough to expose errors in its implementation. A benchmark that reports only an appealing trajectory cannot distinguish a valid game solution from an accidental closed-loop response. By contrast, matched numerical checks allow each stage of the algorithm and compensation layer to be inspected.

2.8. Synthesis and Research Gap

The literature establishes a mature foundation for nominal zero-sum control. Differential game theory defines the minimax problem, linear-quadratic game theory connects the saddle solution to Riccati equations, and model-based policy iteration and value iteration provide established numerical procedures. Quadratic Q-learning and adaptive dynamic programming then recover corresponding policies from sampled transitions under rank, excitation, and conditioning requirements. Pursuit-evasion gives these methods an adversarial control interpretation, while more recent research extends them toward nonlinear dynamics, safety constraints, sensor limitations, and multi-agent systems.

At the same time, the reviewed literature reveals a separation among these areas. The nominal algorithm studies usually assume that commanded inputs are realized exactly. Offset-free and unknown-input estimation studies address persistent mismatch, however they are usually developed for tracking control, MPC, or state estimation rather than a matched four algorithm zero-sum comparison. Pursuit-evasion learning papers increasingly address safety and disturbances, but they often introduce nonlinear function approximators, self-play, obstacles, or multi-agent coordination before establishing a common model-based and model-free benchmark.

Therefore, the specific gap that is addressed by this thesis is not the absence of policy iteration, value iteration, Q-learning, or disturbance estimation individually. Each of those areas is well established. Instead, the gap lies in applying a single persistent command-channel model, estimation logic, affine

compensation law, and numerical validation framework to four structurally related zero-sum algorithms while maintaining the original nominal update equations.

More specifically, the literature does not commonly provide a matched study that contains all of the following elements

- a model-based GARE policy iteration method;
- a model-based GARE value iteration method;
- a model-free quadratic Q-learning policy iteration method;
- a model-free quadratic Q-learning value iteration method;
- persistent command offset affecting both players;
- bias estimation consistent with the information structure of each method;
- affine compensation applied at the command channel;
- nominal, uncompensated, estimated-compensation, and exact-compensation cases; and
- common stability, curvature, conditioning, convergence, and trajectory diagnostics.

This thesis addresses this gap by starting with the established algorithms that are documented by Rizvi and Lin, and then extending their MATLAB implementation in a traceable manner [5]. The nominal Riccati, policy, and Q-learning relationships remain the algorithmic foundation. This thesis aims to define the disturbance model, determine how the offsets are estimated, modify the executed commands, and then establish matched numerical evidence for the resulting behavior.

This position also clarifies the future research directions. A nonlinear kinematic bicycle model, acceleration and steering and bias vectors, command delay, actuator saturation, output feedback, QLabs simulation, and physical QCar testing can then be added after the four algorithm disturbance-aware foundation is established. These are important extensions since the current framework separates the nominal game solution from the implementation disturbance. Thus, the literature supports a staged research program rather than requiring every possible vehicle and learning complication to be included in a single thesis.

In conclusion, the state of the field supports both the feasibility and the need for the present study. The foundational theory explains why the four algorithms should be related. Current policy iteration, value iteration, and zero-sum learning research demonstrates that the problem remains an active research area. Disturbance-estimation and offset-free control literature provides the necessary tools needed to represent persistent command mismatch. In addition, recent pursuit-evasion studies demonstrate the increasing importance of safety, uncertainty, and implementation constraints. As a result, this thesis connects these

bodies of work through a controlled disturbance-aware zero-sum framework whose equations, assumptions, and MATLAB results remain directly traceable.

3. Mathematical Framework for Disturbance-Aware Zero-Sum Pursuit-Evasion Control

3.1. Role of the Unified Framework

The prior chapter established that the literature supports zero-sum dynamic games, model-based Riccati methods, model-free quadratic Q-learning, persistent input estimation, and affine compensation. The objective of this chapter is to define the common mathematical problem that is used by all four algorithms in this thesis. More specifically, this chapter establishes the state equation, the minimizing and maximizing inputs, the quadratic game objective, the saddle-point policies, the Game Algebraic Riccati Equation (GARE), the quadratic action-value function, the persistent command-offset model, and the numerical conditions that are required before a computed solution can be interpreted as a valid zero-sum controller.

This is a necessary common formulation, since the model-based and model-free methods are not intended to solve different control problems. Model-based policy and value iteration, as well as model-free Q-learning policy and value iteration all refer to the same discrete-time two-player game. Their principal distinction is the information that is used during the computation. The model-based methods evaluate matrix equations by using the known system matrices, while the model-free methods estimate the corresponding quadratic relationships from sampled state and input data. Therefore, a meaningful comparison requires the plant, cost, player interpretation, command offsets, compensation cases, and validation metrics to remain fixed across the four methods [4], [5].

In addition, the chapter also clarifies what the current pursuit-evasion interpretation does or does not claim. The present MATLAB benchmark is a reduced-order linear zero-sum system inherited from the source algorithm files [5]. It is interpreted as a local pursuit-evasion engagement in which the minimizing player acts as the pursuer and the maximizing player acts as the evader. However, the three-state benchmark is not presented as a complete nonlinear vehicle model that contains planar position, heading angle, wheelbase, acceleration, and steering. Those physical states remain part of the future vehicle extension. In this thesis, pursuit-evasion provides the game interpretation, while the controlled linear system provides the mathematical foundational benchmark on which the disturbance-aware algorithm extensions are developed.

This separation is crucial for the structure of this thesis. Chapter 4 develops the two model-based methods from the equations introduced here. Then, Chapter 5 develops the two model-free methods by using the same state, cost, and player definitions. Next, Chapter 6 then compares nominal operation,

uncompensated command bias, estimated compensation, and exact compensation under a common MATLAB protocol. As a result, the current chapter acts as the point where the theoretical, algorithmic, and experimental parts of the thesis are joined.

3.2. Discrete-Time Zero-Sum Pursuit-Evasion Dynamics

The nominal system is a discrete-time linear plant with one minimizing input channel and one maximizing input channel, and can be expressed by the following equation

$$x_{k+1} = Ax_k + B_P u_{P,k} + B_E u_{E,k}, \tag{3.1}$$

where $x_k \in \mathbb{R}^n$ denotes the game state at time step k, $u_{P,k} \in \mathbb{R}^{m_P}$ denotes the pursuer input, and $u_{E,k} \in \mathbb{R}^{m_E}$ denotes the evader input. The matrices $A \in \mathbb{R}^{n\times n}$, $B_P \in \mathbb{R}^{n\times m_P}$, and $B_E \in \mathbb{R}^{n\times m_E}$ define the nominal state and player-input channels. From the MATLAB sources, B_P is denoted by B, while B_E is denoted by E. The notation used in this chapter aims to make the pursuer and evader roles explicit.

Notice that the state equation in Equation (3.1) is shared by both players. As a result, this means that the pursuer and evader do not operate separate plants. Rather, their inputs act on the same state, and the resulting state transition reflects the combined effect of both policies. The pursuer attempts to regulate the state in the direction favored by the performance objective, while the evader attempts to oppose that regulation. This coupled structure is what distinguishes the problem from two independent optimal control problems [1], [3].

A state-space formulation is useful for three reasons. First, it makes the effect of each player explicit through B_P and B_E. Second, it supports a quadratic value function and a GARE-based saddle solution. Third, it allows the same transition data to be reorganized into a quadratic Q-learning regression. Therefore, the common state-space model provides the analytical baseline for the model-based methods and the data-generating environment for the model-free methods.

The present benchmark is time-invariant, fully-observed, and deterministic during the nominal control calculation. These assumptions are intended to be restrictive as time variation, partial state measurement, stochastic process noise, actuator saturation, and nonlinear dynamic are important extensions. However, including these extensions at this stage would make it difficult to determine whether an observed difference is caused by the original algorithm, the command-offset extension, or an unrelated modeling change. As

a result, the current formulation starts with the smallest system that is able to preserve the zero-sum saddle structure and the distinction between known-model and data-based computation.

TABLE I: Common Symbols used in the Disturbance-Aware Zero-Sum Formulation

Symbol	Meaning
x_k	Shared game state at time step k.
$u_{P,k}$	Pursuer or minimizing-player input.
$u_{E,k}$	Evader or maximizing-player input.
A	Nominal discrete-time state matrix.
B_P, B_E	Pursuer and evader input matrices.
Q	Positive semidefinite state-penalty matrix.
R_P, R_E	Positive definite pursuer and evader input-weight matrices.
P	Quadratic value matrix.
L, K	Pursuer and evader state-feedback gain matrices.
b_P, b_E	Persistent realized-input offsets in the two command channels.
$\widehat{b}_P, \widehat{b}_E$	Estimated offsets used by the affine compensation layer.

3.3. Pursuer, Evader, and State-Space Definitions

The minimizing and maximizing players are interpreted as a pursuer and an evader. Under this interpretation, the pursuer attempts to reduce a selected relative-state error, while the evader attempts to preserve or enlarge that error. The interpretation follows from the standard zero-sum pursuit-evasion structure in which both players affect the same engagement state but act with opposing objectives [1], [2].

For the current reduced-order system, an auxiliary engagement output can be expressed by

$$y_k = Cx_k. \tag{3.2}$$

In addition, the verified MATLAB benchmark uses

$$C = \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}. \tag{3.3}$$

Hence, $y_k = x_{1,k} - x_{3,k}$ can be interpreted as a reduced relative engagement variable. The output is useful for linking the state-space benchmark to pursuit-evasion language; however, it does not define the state-penalty matrix in this implementation. The numerical studies use $Q = I_3$, so all three states are penalized directly in the quadratic game objective.

This interpretation is intentionally more limited than a geometric vehicle capture condition. In a complete planar pursuit-evasion model, capture may be defined through the Euclidean separation between two position vectors. In the current linear benchmark, the output given in (3.2) is a local engagement proxy that supports interpretation and reporting. As a result, this chapter does not claim that $y_k = 0$ is identical to physical contact between two vehicles. Instead, it is a reduced-order quantity that allows the four algorithms and the command-offset extension to be studied and examined without changing the original benchmark matrices.

The pursuer and evader policies are both calculated from the same game state. Thus, the system uses a feedback information pattern rather than open-loop input sequences. At each time step, the current state is supplied to both policies, each player generates a command, and the two realized inputs jointly determine the next state. This feedback structure is central to the later policy iteration and Q-learning derivations since both methods seek stationary gain matrices rather than one-time input trajectories.

3.4. Quadratic Game Objective and Saddle-Point Policies

The infinite-horizon performance index is expressed by the equation

$$J(x_0, u_P, u_E) = \sum_{k=0}^{\infty} \left(x_k^\top Q x_k + u_{P,k}^\top R_P u_{P,k} - u_{E,k}^\top R_E u_{E,k} \right), \tag{3.4}$$

where $Q \succeq 0, R_P \succ 0$, and $R_E \succ 0$. The pursuer aims to minimize J, while the evader seeks to maximize the same quantity. As a result, the saddle objective can be written conceptually as

$$J^*(x_0) = \min_{u_P} \max_{u_E} J(x_0, u_P, u_E). \tag{3.5}$$

Notice that the sign structure in Equation (3.4) is essential. The pursuer is penalized for state deviation and its own control effort. The evader appears with a negative input penalty because it acts as the maximizing player. Although no explicit actuator bounds are imposed in the nominal LQ game, the maximizing direction must satisfy the required concavity so that the saddle problem remains finite and well-posed. Rather, R_E and the saddle-point curvature conditions limit the maximizing direction and determine whether the stage game is well posed. In other words, the negative sign creates the adversarial objective, while the matrix conditions later prevent the maximization from becoming unbounded.

For the verified MATLAB benchmark, the weights are given by

$$Q = I_3, \quad R_P = 1, \quad R_E = \gamma^2, \quad \gamma = 1. \tag{3.6}$$

The parameter γ is consistent with the H_∞ interpretation of the source zero-sum game, but the present thesis uses pursuer-evader language such that the same minimizing and maximizing channels that can be interpreted within a pursuit-evasion setting [5], [39].

The objective in (3.4) is evaluated using the inputs that actually enter the plant. This distinction becomes important after command bias is introduced. A controller may compute a nominal command, but the cost accumulated by the physical or simulated plant relies on the realized input in the state equation. Consequently, the later comparisons use the same cost definition for the nominal, biased, and compensated trajectories while changing only the relationship between commanded and realized inputs.

Quadratic Value Function and Feedback Saddle Policies: For a stabilizing linear-quadratic zero-sum game, the value function is assumed to have the quadratic form

$$V(x_k) = x_k^\top P x_k, \tag{3.7}$$

where $P = P^\top \succeq 0$ denotes the game value matrix. Then, the feedback policies are represented by

$$u_{P,k} = L x_k, \quad u_{E,k} = K x_k, \tag{3.8}$$

where $L \in \mathbb{R}^{m_P \times n}$ and $K \in \mathbb{R}^{m_E \times n}$. The sign convention in (3.8) is retained from the MATLAB source code. Therefore, any negative feedback direction is contained in the numerical values of L and K rather than written as an additional minus sign outside the gains.

Substituting Equation (3.8) into Equation (3.1) gives the nominal closed-loop system expressed by

$$x_{k+1} = A_{\text{cl}} x_k, \quad A_{\text{cl}} = A + B_P L + B_E K. \tag{3.9}$$

A stabilizing saddle solution requires A_{cl} to be Schur stable, meaning that every eigenvalue lies strictly inside the unit circle. This condition is then expressed as

$$\rho(A_{\text{cl}}) < 1, \tag{3.10}$$

where $\rho(\cdot)$ denotes the spectral radius.

The saddle policies must also satisfy the minimax ordering. For every admissible state, the pursuer policy should minimize the value against the evader policy, while the evader policy should maximize the value against the pursuer policy. Conceptually, this is expressed as follows

$$J(x_0; L, K') \leq J(x_0; L, K) \leq J(x_0; L', K) \tag{3.11}$$

for admissible alternative gains L' and K'. The exact inequality depends on the policy class and initial state, however the expression captures the central interpretation. More specifically, neither player can improve its objective through a unilateral admissible change when the other player holds the saddle policy fixed [1], [3].

The value matrix P is not only a cost summary. It also connects the model-based and model-free sections of this thesis. In the model-based methods, P is evaluated or updated directly from the system matrices. In the model-free methods, the action-value matrix H is estimated from data, after which the policy gains and an equivalent state-value relationship are recovered. Thus, P and H represent two views of the same nominal game.

3.5. Equivalent Nominal Value Representations

The four algorithms considered in this thesis aim to seek the same nominal saddle solution through two equivalent quadratic representations. The model-based algorithms operate directly on the state-value matrix P, while the model-free algorithms estimate a joint state-action matrix H. This section defines those common mathematical objects without reproducing the complete iteration procedures that will be developed in Chapter 4 and Chapter 5.

Generalized Algebraic Riccati Game Target: The Game Algebraic Riccati Equation (GARE) provides the steady-state model-based solution. To state the equation compactly, define the block game matrix as follows

$$\mathcal{G}(P) = \begin{bmatrix} R_P + B_P^\top P B_P & B_P^\top P B_E \\ B_E^\top P B_P & B_E^\top P B_E - R_E \end{bmatrix} \tag{3.12}$$

Then, the stacked gain matrix is

$$\begin{bmatrix} L \\ K \end{bmatrix} = -\mathcal{G}(P)^{-1} \begin{bmatrix} B_P^\top PA \\ B_E^\top PA \end{bmatrix}. \tag{3.13}$$

In the numerical implementation, 3.13 is evaluated through a linear solve rather than by explicitly forming the inverse. Next, the value matrix satisfies the following equation

$$P = Q + A^\top PA - \begin{bmatrix} A^\top PB_P & A^\top PB_E \end{bmatrix} \mathcal{G}(P)^{-1} \begin{bmatrix} B_P^\top PA \\ B_E^\top PA \end{bmatrix}. \tag{3.14}$$

Equation (3.14) is the common nominal target for the two model-based algorithms and the corresponding model-free Q-learning procedures [3], [14].

The block matrix in Equation (3.12) must be numerically invertible. However, invertibility alone is not sufficient. Now, define

$$H_{PP} = R_P + B_P^\top PB_P, \tag{3.15}$$

$$H_{PE} = B_P^\top PB_E, \quad H_{EP} = B_E^\top PB_P, \tag{3.16}$$

$$H_{EE} = B_E^\top PB_E - R_E. \tag{3.17}$$

The minimizing-player block must be positive definite,

$$H_{PP} \succ 0, \tag{3.18}$$

and the maximizing-player Schur complement must be negative definite,

$$S_E = H_{EE} - H_{EP}H_{PP}^{-1}H_{PE} \prec 0. \tag{3.19}$$

The first condition ensures local convexity in the minimizing-player input. Then the second ensures local concavity in the maximizing-player input after the minimizing direction is accounted for. These conditions distinguish a valid saddle point from a stationary point that has the wrong curvature.

An equivalent closed-loop policy evaluation identity is

$$P = Q + L^\top R_P L - K^\top R_E K + A_{\mathrm{cl}}^\top PA_{\mathrm{cl}}. \tag{3.20}$$

This form is especially useful for policy iteration, since the gains remain fixed during the policy evaluation step. Additionally, it is also useful for verification, since a converged P, L, and K should satisfy both the GARE form as well as the closed-loop Lyapunov form within numerical tolerance. The detailed usage of (3.20) in policy iteration, and the repeated Riccati mapping used in value iteration, are presented in Chapter 4.

Quadratic Action-Value Representation: The model-free methods use a quadratic action-value function. Define the joint state-action vector

$$z_k = \begin{bmatrix} x_k \\ u_{P,k} \\ u_{E,k} \end{bmatrix} \in \mathbb{R}^{n_z}, \quad n_z = n + m_P + m_E. \tag{3.21}$$

The one-step action-value function is

$$\mathcal{Q}(z_k) = z_k^\top H z_k, \tag{3.22}$$

where $H = H^\top$ is partitioned as

$$H = \begin{bmatrix} H_{xx} & H_{xP} & H_{xE} \\ H_{Px} & H_{PP} & H_{PE} \\ H_{Ex} & H_{EP} & H_{EE} \end{bmatrix}. \tag{3.23}$$

For a known value matrix P, the exact nominal action-value matrix is given by

$$H = \begin{bmatrix} Q + A^\top P A & A^\top P B_P & A^\top P B_E \\ B_P^\top P A & R_P + B_P^\top P B_P & B_P^\top P B_E \\ B_E^\top P A & B_E^\top P B_P & B_E^\top P B_E - R_E \end{bmatrix}. \tag{3.24}$$

The model-free algorithms do not construct (3.24) by substituting the system matrices into the formula. Instead, they estimate the unique elements of H from sampled Bellman relationships [4], [5].

The saddle policies are recovered directly from the input-related blocks of H, given by

$$\begin{bmatrix} L \\ K \end{bmatrix} = - \begin{bmatrix} H_{PP} & H_{PE} \\ H_{EP} & H_{EE} \end{bmatrix}^{-1} \begin{bmatrix} H_{Px} \\ H_{Ex} \end{bmatrix}. \tag{3.25}$$

Equation (3.25) has the same block saddle structure as the model-based gain equation. This is the key reason that quadratic Q-learning is a direct model-free counterpart to the GARE solution rather than an unrelated reinforcement learning controller.

Since H is symmetric, as it contains

$$n_H = \frac{n_z(n_z+1)}{2} \tag{3.26}$$

unique coefficients. Thus, a symmetric quadratic feature vector $\phi(z_k)$ is constructed from squared terms and doubled cross terms. The sampled Bellman equations take the regression form

$$\Phi\theta \approx r, \tag{3.27}$$

where θ contains the unique coefficients of H, Φ is the feature matrix, and r denotes the target vector formed from stage costs and successor-state terms. At minimum,

$$\operatorname{rank}(\Phi) = n_H \tag{3.28}$$

should hold for exact least-squares identification of the quadratic parameters. Full numerical rank is necessary, but the condition number also matters. A feature matrix can be technically full rank while remaining, so ill-conditioned that the estimated coefficients are dominated by numerical error. Consequently, the model-free chapters will aim to report both rank and conditioning rather than treating the least-squares solution as valid merely since MATLAB returns a vector.

Model-free policy iteration and model-free value iteration differ in how the Bellman target is formed and how the policy is updated. Policy iteration evaluates the Q-function associated with a current gain pair, and then improves the pair through the saddle-policy recovery equation. Value iteration updates the value or action-value approximation directly from the sampled optimality relation. The detailed update equations will be presented in Chapter 5 so that the current chapter can remain focused on the shared mathematical objects of both methods. The key point is that neither method uses A, B_P, or B_E directly in its Q-learning regression.

Table II summarizes the information structures as well as computational roles of the four methods without duplicating the algorithmic derivations.

TABLE II: Common Roles and Information Structures of the Four Thesis Algorithms

Method	Information Used in the Nominal Update	Principal Computational Structure
Model-Based Policy Iteration	Known A, B_P, and B_E	Lyapunov policy evaluation followed by saddle-gain improvement.
Model-Based Value Iteration	Known A, B_P, and B_E	Repeated GARE or Bellman value mapping followed by gain recovery.
Model-Free Q-Learning Policy Iteration	Sampled states, realized inputs, costs, and successor states	Quadratic Q-function regression for fixed policies followed by policy improvement.
Model-Free Q-Learning Value Iteration	Sampled states, realized inputs, costs, and successor states	Sample-based value or Q-function updates followed by gain recovery.

3.6. Persistent Command-Channel Offset Model

The nominal algorithms assume that the command generated by each policy is the input that enters the plant. The disturbance-aware extension removes this idealized equivalence by separating the commanded input from the realized input. This a crucial distinction since a policy can be mathematically correct for the nominal game while the executed plant still experiences a persistent offset. Moreover, the offset does not require the GARE or the quadratic Q-function to be replaced. Instead, it introduces an affine execution-side term that must be identified and compensated after the nominal saddle policies have been obtained.

For the pursuer, the realized input can be expressed as

$$u_{P,k}^{\mathrm{act}} = u_{P,k}^{\mathrm{cmd}} + b_P, \tag{3.29}$$

and for the evader,

$$u_{E,k}^{\mathrm{act}} = u_{E,k}^{\mathrm{cmd}} + b_E, \tag{3.30}$$

where $b_P \in \mathbb{R}^{m_P}$ and $b_E \in \mathbb{R}^{m_E}$ are constant or slowly varying command-channel offsets over the calibration and simulation windows. The offsets are matched disturbances since each term enters through the same input matrix as the corresponding player command. Therefore, the realized plant is given by

$$x_{k+1} = Ax_k + B_P u_{P,k}^{\mathrm{act}} + B_E u_{E,k}^{\mathrm{act}}. \tag{3.31}$$

If the nominal feedback commands are applied without compensation,

$$u_{P,k}^{\mathrm{cmd}} = Lx_k, \quad u_{E,k}^{\mathrm{cmd}} = Kx_k, \tag{3.32}$$

then the closed-loop dynamics becomes

$$x_{k+1} = A_{\mathrm{cl}} x_k + B_P b_P + B_E b_E. \tag{3.33}$$

The constant affine forcing term is

$$d_b = B_P b_P + B_E b_E. \tag{3.34}$$

And when $I - A_{\mathrm{cl}}$ is nonsingular, the corresponding biased equilibrium is

$$x_\infty^{\mathrm{bias}} = (I - A_{\mathrm{cl}})^{-1} d_b. \tag{3.35}$$

Equation (3.35) distinguishes stability from regulation. A Schur-stable feedback matrix can still drive the realized plant toward a nonzero equilibrium when the command offsets remain uncompensated. As a result, the disturbance-aware problem is not created by a failure of the nominal Riccati or Q-learning solution. It is created by the difference between the policy command and the input that is actually applied. This observation supplies the mathematical reasoning for adding an estimation and compensation layer while retaining the nominal gain pair.

The same offset model can represent several execution-side effects. A repeatable acceleration error, steering-center offset, motor-command conversion error, actuator calibration mismatch, or persistent input-reconstruction error can be approximated by b_P and b_E over a finite experiment. This thesis does not claim that every fault is constant. Rather, the constant-offset assumption provides a transparent first disturbance model whose influence can be derived exactly and whose compensation can be verified without changing the four source algorithms [15], [16].

3.7. Offset Estimation under Model-Based and Model-Free Information

The model-based and model-free methods require bias estimates for the same purpose, however they do not possess the same information. The model-based algorithms know A, B_P, and B_E, whereas the model-free Bellman regressions are intentionally constructed without those matrices. Consequently, the estimation procedures are matched in objective but are not forced into the same numerical form.

Model-Based Transition Residual Estimation: For the model-based case, define the one-step transition residual as follows

$$r_k = x_{k+1} - Ax_k - B_P u_{P,k}^{\mathrm{cmd}} - B_E u_{E,k}^{\mathrm{cmd}}. \tag{3.36}$$

Substituting the realized-input model gives

$$r_k = \underbrace{\begin{bmatrix} B_P & B_E \end{bmatrix}}_{G_b} \underbrace{\begin{bmatrix} b_P \\ b_E \end{bmatrix}}_{b} + v_k, \tag{3.37}$$

where v_k collects transition-measurement noise and unmodeled effects. For a constant bias over N_{cal} samples, the sample-mean residual satisfies

$$\bar{r} = G_b b + \bar{v}. \tag{3.38}$$

The least-squares estimate is expressed by

$$\widehat{b} = G_b^{\dagger} \bar{r}, \tag{3.39}$$

where $(\cdot)^{\dagger}$ denotes the Moore-Penrose pseudoinverse. In the MATLAB implementation, the estimate is obtained through a linear least-squares solve rather than by explicitly forming an inverse. This formulation uses the known plant matrices only in the model-based calibration stage and leaves the original GARE policy or value update unchanged.

The two individual offsets are identifiable from the transition residual only when

$$\mathrm{rank}(G_b) = m_P + m_E. \tag{3.40}$$

If the two input directions are linearly dependent, the net affine disturbance d_b may still be identifiable even when the separate player offsets are not. Thus, this rank condition is a physical identifiability requirement rather than only a numerical convenience.

For independent isotropic transition noise with covariance $\sigma_v^2 I$, the covariance of the sample-mean least-squares estimate is

$$\mathrm{Cov}(\widehat{b}) = \frac{\sigma_v^2}{N_{\text{cal}}}(G_b^\top G_b)^{-1}, \tag{3.41}$$

provided that G_b has full column rank. This expression supplies the theoretical individual-estimate standard deviations that are used in the Monte Carlo validation in Chapter 6.

Model-Free Command Mismatch Estimation: For the model-free case, the source implementation assumes that commanded and realized inputs can be measured during a calibration period. The offsets are estimated directly as follows

$$\widehat{b}_P = \frac{1}{N_{\text{cal}}} \sum_{k=1}^{N_{\text{cal}}} \left(u_{P,k}^{\text{meas}} - u_{P,k}^{\text{cmd}} \right), \tag{3.42}$$

$$\widehat{b}_E = \frac{1}{N_{\text{cal}}} \sum_{k=1}^{N_{\text{cal}}} \left(u_{E,k}^{\text{meas}} - u_{E,k}^{\text{cmd}} \right). \tag{3.43}$$

Notice that these equations do not use A, B_P, and B_E. As a result, the Q-learning update remains model-free with respect to the plant dynamics. The command-mismatch calculation is an execution-side measurement step rather than a hidden model-based reconstruction inside the Bellman regression.

Separate measured inputs are assumed in (3.42) and (3.43). If those measurements are unavailable, additional structure such as an observer, an augmented state, a plant model, or another identifiability assumption would be required [17]. Clearly stating this limitation is crucial since the compensation results depend not only on the controller, but also on what is measured during calibration.

3.8. Affine Compensation and Residual Closed-Loop Behavior

After the offsets are estimated, then the policy-generated inputs are modified before they enter the biased command channels. The compensated commands are as follows

$$u_{P,k}^{\text{cmd}} = Lx_k - \widehat{b}_P, \tag{3.44}$$

$$u_{E,k}^{\text{cmd}} = Kx_k - \widehat{b}_E. \tag{3.45}$$

Thus, it follows that the realized inputs are

$$u_{P,k}^{\text{act}} = Lx_k - \widehat{b}_P + b_P, \tag{3.46}$$

$$u_{E,k}^{\text{act}} = Kx_k - \widehat{b}_E + b_E. \tag{3.47}$$

Next, define the residual bias errors

$$\widetilde{b}_P = b_P - \widehat{b}_P, \quad \widetilde{b}_E = b_E - \widehat{b}_E. \tag{3.48}$$

Hence, substitution into the plant gives

$$x_{k+1} = A_{\text{cl}}x_k + B_P\widetilde{b}_P + B_E\widetilde{b}_E. \tag{3.49}$$

Equation (3.49) contains the complete disturbance-aware interpretation. When the estimates are exact, it follows that

$$\widehat{b}_P = b_P, \quad \widehat{b}_E = b_E, \tag{3.50}$$

the residuals offsets vanish and

$$x_{k+1} = A_{\text{cl}}x_k. \tag{3.51}$$

As a result, exact compensation recovers the nominal closed-loop dynamics without recomputing P, H, L, or K. The exact case is not presented as an implementable solution when the offsets are unknown. It is a verification. So, if exact cancellation does not reproduce the nominal trajectory to numerical precision, the command and realized-input bookkeeping is inconsistent.

When the estimates contain error, the compensated plant remains subject to a smaller affine term.

Provided that $I - A_{\text{cl}}$ is nonsingular, its residual equilibrium is

$$x_{\infty}^{\text{comp}} = (I - A_{\text{cl}})^{-1} \left(B_P \widetilde{b}_P + B_E \widetilde{b}_E \right). \tag{3.52}$$

This expression separates the three quantities that determine the final regulation error. The first is the estimation error itself, and the second is the direction through which each residual enters the state. The third is the closed-loop amplification that is contained in $(I - A_{\text{cl}})^{-1}$. Thus, a small scalar bias-estimation error does not automatically guarantee an equally small state error. A nearly marginal closed-loop is able to amplify a constant residual more strongly than a well-damped closed-loop.

The deviation from the nominal trajectory is define as follows

$$d_k = x_k^{\text{case}} - x_k^{\text{nom}}. \tag{3.53}$$

For a matched initial condition, $d_0 = 0$, and the deviation recurrence is

$$d_{k+1} = A_{\text{cl}} d_k + B_P \widetilde{b}_P + B_E \widetilde{b}_E. \tag{3.54}$$

Its finite-step solution is expressed as

$$d_k = \sum_{j=0}^{k-1} A_{\text{cl}}^j \left(B_P \widetilde{b}_P + B_E \widetilde{b}_E \right). \tag{3.55}$$

Moreover, the MATLAB implementations compare this analytical recurrence with the simulated trajectory difference as the agreement to floating-point precision verifies that the observed deviation is caused by the intended affine forcing rather than by inconsistent simulation logic.

The affine compensation layer preserves the nominal feedback gains. This design decision is key to the claim of this thesis. Each source algorithm computes L and K according to its original policy iteration or value iteration structure. Calibration estimates b_P and b_E, and execution subtracts those estimates before the commands enter the biased plant. Therefore, the extension is traceable to the original algorithms while remaining mathematically sufficient to remove a matched constant offset.

TABLE III: Common Execution Cases Used Across the Four Algorithms

Case	Definition and Purpose
Nominal operation	The command offsets are zero and the feedback commands enter the plant directly. This case supplies the reference trajectory and finite-window cost.
Uncompensated offsets	The true offsets are added to the policy commands, but no estimates are subtracted. This case isolates the effect of execution-side mismatch when the nominal gains are unchanged.
Estimated compensation	The offsets are estimated according to the information structure of the algorithm and are subtracted from the commanded inputs. This case measures practical recovery in the presence of calibration error.
Exact compensation	The true offsets are used in place of the estimates. This case verifies the algebra and implementation rather than representing a realizable controller with unknown biases.

3.9. Common Validation Cases

Each algorithm is evaluated through the same four execution cases. As a result, the cases are defined here so that in Chapter 4-6 do not change the disturbance model nor the meaning of compensation from one method to another.

The game matrices, cost weights, initial condition, true offsets, simulation horizon, and reporting definitions remain common across the four methods. Thus, the comparison separates three questions. First, does the nominal algorithm converge to a valid saddle solution? Second, how strongly does that fixed gain pair respond to persistent command offsets? Third, how much of the nominal behavior is recovered after estimated or exact compensation?

The common protocol does not predetermine that one algorithm must outperform the others. Under matched assumptions and exact arithmetic, all four methods seek the same nominal saddle policies. Their practical differences are expected to appear in convergence behavior, model information, data requirements, conditioning, and sensitivity to calibration error. Therefore, the evaluation emphasizes common diagnostics rather than presenting model-based and model-free control as unrelated competitors.

3.10. Common Numerical Metrics and Validity Conditions

A zero-sum iteration can stop changing without producing a valid saddle solution. Similarly, a compensation routine can appear to reduce an error while still being inconsistent with the commanded and realized inputs. As a result, the MATLAB study uses two connected layers of verification. The first

verifies the nominal saddle solution, while the second applies that verified gain pair to the common disturbance-aware benchmark.

Verification of the Nominal Saddle Solution: The first check is closed-loop stability. After the gains are recovered, the nominal matrix in (3.9) must satisfy (3.10). For policy iteration, this condition is also checked during policy evaluation since the infinite-horizon Lyapunov equation requires an admissible closed-loop.

The second check is the local saddle curvature. The symmetric part of the minimizing-player block must be positive definite, while the symmetric part of the maximizing-player Schur complement must be negative definite:

$$\lambda_{\min}\left(\frac{H_{PP}+H_{PP}^{\top}}{2}\right) > 0, \tag{3.56}$$

$$\lambda_{\max}\left(\frac{S_E+S_E^{\top}}{2}\right) < 0. \tag{3.57}$$

These tests distinguish a valid minimax saddle point from a stationary point having the wrong curvature. Symmetrization prevents small floating-point asymmetries from being treated as meaningful game structure.

The third check concerns numerical invertibility. The reciprocal condition number of the block game matrix should satisfy

$$\mathrm{rcond}(\mathcal{G}) > \varepsilon_{\text{cond}}. \tag{3.58}$$

The MATLAB implementations do not explicitly compute $\mathcal{G}^{-1}$. Instead, the block gain equations are evaluated through linear solves by using the backslash operator. This approach is more efficient and more numerically reliable than multiplying by an explicitly formed inverse [70].

The fourth check then evaluates the defining Riccati equation. Let $\mathcal{T}(P)$ denote the right-hand side of (3.14). The residual is

$$R_{\text{GARE}} = P - \mathcal{T}(P), \tag{3.59}$$

and the normalized residual is

$$\varepsilon_{\text{GARE}} = \frac{\|R_{\text{GARE}}\|_F}{1 + \|P\|_F}. \tag{3.60}$$

This is a stronger check than only monitoring the change between successive iterates. An update sequence can stagnate before reaching an accurate fixed point, whereas the residual directly tests the equation that the final value matrix is supposed to satisfy.

The policy evaluation identity in (3.20) provides a second residual,

$$R_{\text{Lyap}} = P - \left(Q + L^\top R_P L - K^\top R_E K + A_{\text{cl}}^\top P A_{\text{cl}}\right), \tag{3.61}$$

which is also normalized by $1 + \|P\|_F$; moreover, agreement between the Game Algebraic Riccati Equation (GARE) and Lyapunov forms is crucial for policy iteration.

The model-free methods require additional checks. The quadratic feature matrix needs to have full numerical rank, its condition number must remain acceptable, and the estimated H matrix must be symmetric to numerical tolerance. The sampled Bellman residual and the changes in L and K are also reported. When a model-based reference is used only for verification, relative gain error can be computed as

$$\varepsilon_L = \frac{\|L_Q - L_{\text{ref}}\|_F}{1 + \|L_{\text{ref}}\|_F}, \quad \varepsilon_K = \frac{\|K_Q - K_{\text{ref}}\|_F}{1 + \|K_{\text{ref}}\|_F}. \tag{3.62}$$

These quantities are implementation diagnostics and are not supplied to the learner during Q-function regression.

Verification of Estimation and Compensation: The disturbance-aware layer is verified by reporting the true offsets, estimated offsets, signed and absolute estimation errors, theoretical and empirical estimator spread, and the residual bias terms. The exact-compensation trajectory error is defined as

$$\varepsilon_{\text{exact}} = \max_{0 \le k \le N_{\text{sim}}} \|x_k^{\text{exact}} - x_k^{\text{nom}}\|_2. \tag{3.63}$$

This quantity should be near floating-point precision when the exact and nominal cases use the same gain pair, initial condition, simulation horizon, and arithmetic.

The principal trajectory metric is the cost-weighted deviation is

$$\|d_k\|_Q = \sqrt{d_k^\top Q d_k}. \tag{3.64}$$

Since $Q = I_3$ in the present benchmark, this metric is numerically equivalent to the Euclidean state-deviation norm. By writing the metric in the Q-weighted form, it keeps it aligned with the state penalty in the game objective and allows the definition to remain valid if a different positive semidefinite Q is used in later research.

The finite-window accumulated game cost is reported as

$$J_N = \sum_{k=0}^{N_{\text{sim}}-1} \left[x_k^\top Q x_k + (u_{P,k}^{\text{act}})^\top R_P u_{P,k}^{\text{act}} - (u_{E,k}^{\text{act}})^\top R_E u_{E,k}^{\text{act}} \right]. \tag{3.65}$$

This quantity is a reproducible summary of the simulated interval and is not substituted for the infinite-horizon theoretical objective. Since the evader contribution enters with a negative sign, then J_N is interpreted together with the state histories, realized inputs, convergence diagnostics, and residual bias measures rather than as a stand-alone measure of pursuit quality. Table IV provides a summary of the numerical checks that are used across the thesis.

3.11. Common Numerical Benchmark

The common protocol needs to be established before the numerical matrices are introduced. This is an intentional decision, as the objective of the benchmark is first to define what remains across the algorithms and only then to state the particular numerical realization inherited from the source files.

The state matrix is given by

$$A = \begin{bmatrix} 0.906488 & 0.0816012 & -0.000500000 \\ 0.0741349 & 0.901210 & -0.000708383 \\ 0 & 0 & 0.132655 \end{bmatrix}, \tag{3.66}$$

and the pursuer and evader input matrices are

$$B_P = \begin{bmatrix} -0.00150808 \\ -0.00960000 \\ 0.867345 \end{bmatrix}, \quad B_E = \begin{bmatrix} 0.00951892 \\ 0.000383730 \\ 0 \end{bmatrix}. \tag{3.67}$$

TABLE IV: Numerical Verification Metric Summary across the Four Algorithms

Check	**Interpretation**
Iteration change	Determines whether the value matrix, Q-function, and gain sequences have stopped changing.
GARE residual	Tests whether the converged model-based value matrix satisfies the defining Riccati equation.
Lyapunov residual	Tests whether the converged policy-iteration result satisfies the fixed-policy evaluation identity.
Closed-loop spectral radius	Verifies Schur stability of the nominal pursuer–evader gain pair.
Pursuer curvature	Verifies positive curvature in the minimizing-player input.
Evader Schur complement	Verifies negative curvature in the maximizing-player input after eliminating the pursuer block.
Reciprocal condition number	Detects near-singularity of the block game matrix before a gain solve is accepted.
Feature rank and conditioning	Determines whether the model-free quadratic parameters are identifiable and numerically reliable.
Bellman residual	Measures consistency of the estimated Q-function with the sampled Bellman equation.
Bias-estimation error	Measures calibration accuracy in each command channel.
Estimator spread	Compares empirical individual-estimate standard deviation with the theoretical covariance prediction.
Cost-weighted deviation	Measures departure from the nominal trajectory in the state metric used by the game.
Command-channel error	Measures the difference between the realized input and the policy input.
Analytical recurrence mismatch	Verifies agreement between the simulated state deviation and the affine deviation recurrence.
Exact-compensation error	Verifies that exact bias cancellation reproduces the nominal closed-loop trajectory.

The values in A define two relatively persistent and mutually coupled states together with a faster auxiliary state. The first two diagonal entries are near 0.9, so those components decay slowly in open-loop and exchange information through the off-diagonal terms 0.0816012 and 0.0741349. The third state has a much smaller diagonal value, 0.132655, and thus decays more rapidly when it is not being driven. The small negative entries in the third column connect that faster internal state back into the first two components. As a result, this organization produces a reduced-order dynamic chain rather than three independent scalar modes.

The player matrices create different control directions. The pursuer input has its dominant effect on the third state through the entry 0.867345, while its direct influence on the first two states is comparatively small. Consequently, the pursuer affects the full state through the internal dynamics of the plant. The evader input acts stronger on the first state through 0.00951892, has a weaker effect on the second state, and has no direct third-state component. Thus, the two players do not possess identical authority even

though both use scalar inputs. The unequal input directions are part of what gives the benchmark a nontrivial zero-sum structure.

The auxiliary engagement output and the quadratic weights are as follows

$$C = \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}, \quad Q = I_3, \quad R_P = 1, \quad R_E = 1. \tag{3.68}$$

The output $y_k = x_{1,k} - x_{3,k}$ provides a reduced pursuit-evasion interpretation, while $Q = I_3$ penalizes all three state components in the actual benchmark cost. This is another important distinction since replacing Q with $C^\top C$ would define a different zero-sum game and would change the computed value matrix, feedback gains, convergence histories, and numerical results.

The numerical matrices are retained from the source code such that the thesis extension remains traceable to the original algorithms. Moreover, they are not claimed to be a direct discretization of planar vehicle position, heading, acceleration, and steering. Rather, they define the reduced-order local game on which the disturbance-aware mathematics is verified. The pursuit-evasion interpretation is given by the maximizing and maximizing roles and by the auxiliary relative engagement output.

The common initial condition is

$$x_0 = \begin{bmatrix} 10 & 5 & -2 \end{bmatrix}^\top \tag{3.69}$$

This choice excites all three states and prevents the numerical comparison from being reduced to a single-mode response. The persistent scalar offsets are

$$b_P = 0.040, \quad b_E = -0.010 \tag{3.70}$$

Their opposite signs allow the two command channels to be able to produce competing affine effect. The values are large enough to create a visible difference from the nominal response while remaining small relative to the dynamic range of the simulated commands.

The common closed-loop horizon is given by $N_{\text{sim}} = 500$ time steps and the calibration length is $N_{\text{cal}} = 250$. For the model-based estimator validation, the transition-noise standard deviation is 10^{-4} and the Monte Carlo study uses 1000 independent calibration trials. Model-free regression sample counts and exploration parameters are algorithm-specific implementation settings and are stated in Chapter 5, while

TABLE V: Common Numerical Benchmark and Experiment Settings

Setting	**Value or Interpretation**
State dimension	$n = 3$
Player input dimensions	$m_P = m_E = 1$
Auxiliary engagement output	$C = \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}$
State penalty	$Q = I_3$
Input weights	$R_P = 1$ and $R_E = 1$
True command offsets	$b_P = 0.040$ and $b_E = -0.010$
Calibration length	$N_{\text{cal}} = 250$
Closed-loop horizon	$N_{\text{sim}} = 500$ time steps
Initial condition	$x_0 = \begin{bmatrix} 10 & 5 & -2 \end{bmatrix}^\top$
Model-based calibration noise	Standard deviation 10^{-4}
Monte Carlo estimator trials	1000
Nominal interpretation	Reduced-order pursuit–evasion regulation benchmark; not a complete nonlinear vehicle model

the plant, cost, offsets, closed-loop horizon, and execution cases remain common.

3.12. Assumptions and Limits of the Framework

There are several delimitations that follow directly from the equations in this chapter. First, the present plant is linear and time-invariant; moreover, nonlinear pursuit-evasion dynamics are outside the current derivation. Second, the full state is available to both policies. As a result, output feedback estimation appears in the broader source literature, however it is not added to the current four algorithm comparison. Third, each player has one scalar input. Vector-valued acceleration and steering commands remain part of a future vehicle extension. Fourth, the command offsets are constant over the calibration and simulation windows so varying biases may be approximated over short intervals. However, abrupt faults or continuously drifting offsets would require either online estimation, change detection, or adaptive compensation. Fifth,m the compensation law assumes that matched disturbances that enter through the player input channels. An unmatched disturbance cannot generally be canceled only by subtracting a scalar command offset. Sixth, the model-free calibration assumes that the realized inputs can be measured as without those measurements, an additional observer or augmented-state structure would be required. ' Finally, the current pursuit-evasion interpretation is local and reduced order. This thesis does not claim a completed nonlinear vehicle simulation, QLabs validation, nor physical QCar experiments. Those applications directions remain important since persistent accelerations and steering offsets have clear physical meaning. On the contrary, establishing the disturbance-aware algorithmic foundation on

the original linear benchmark is necessary before the framework is transferred to a higher-dimensional vehicle model.

These limits make the contribution more testable rather than less significant. The source algorithms are preserved, the disturbance model is explicit, the estimator assumptions are stated and the numerical checks are reproducible. Consequently, later results can be traced to a specific equation, matrix property, or implementation decision rather than attributed to a broad and unverified notion of robustness.

3.13. Summary

This chapter developed the common mathematical framework that will be used by the four disturbance-aware zero-sum algorithms. The nominal plant is a discrete-time linear system with one minimizing pursuer input and one maximizing evader input. A quadratic performance objective produces a feedback saddle solution as described by the value matrix P, the action-value matrix H, the pursuer gain L, the evader gain K, and the Game Algebraic Riccati Equation (GARE). The model-based methods evaluate these relationships using the known plant matrices, while the model-free methods estimate the corresponding quadratic action-value structure from sampled transitions.

Moreover, the chapter then separated commanded and realized inputs through persistent offsets in both player channels. Those offsets introduce a constant affine term and may create a nonzero equilibrium even when the nominal feedback matrix remains Schur stable. Model-based transition residuals and model-free command-versus-measurement calibration provide two information-consistent estimation procedures. Subtracting the estimates from the policy commands leaves only the residual bias errors, while exact compensation recovers the nominal closed-loop dynamics.

The final portion of the chapter connected the mathematical formulation to the common MATLAB evaluation. More specifically, it defined the stability, curvature, conditioning, Riccati, Lyapunov, Bellman, rank, estimation, and compensation checks before introducing the numerical benchmark. The placement of the matrix values clarifies their purpose, as they are the fixed reduced-order realization on which the common game, command offsets, calibration assumptions, and four execution cases are tested.

The next chapter will develop the model-based GARE policy iteration and value iteration algorithms under this framework while preserving the nominal Riccati equations and adding transition residual calibration and affine compensation. Then, chapter 5 develops the model-free quadratic Q-learning counterparts under the same plant, cost, player interpretation, and disturbance-aware protocol.

4. Disturbance-Aware Model-Based GARE Policy and Value Iteration

4.1. Introduction

The previous chapter established a common mathematical framework that will be used by all four algorithms within this thesis. Recall that it defined the discrete-time two-player plant, the minimizing pursuer and maximizing evader, the quadratic game objective, the GARE target, the persistent command-offset model, the affine compensation law, as well as the numerical conditions that are required before a computed solution can be interpreted as a valid saddle controller. The objective of this chapter is more narrow. More specifically, this chapter aims to develop the two model-based procedures that compute the nominal saddle solution from the known plant matrices and then shows how the disturbance-aware execution layer is attached to each procedure without changing the Riccati equations that define the nominal game.

The two procedures are model-based GARE policy iteration and model-based GARE value iteration. Both utilize the known matrices A, B_P, and B_E, and both seek the same stabilizing state-value matrix P together with the pursuer and evader feedback gains L and K. Their differences lie in the path that is used to approach that solution. Policy iteration evaluates the complete infinite-horizon value of a fixed admissible gain pair before improving the policies, while value iteration applies the Riccati optimality mapping directly to a current value approximation and then recovers the associated gains. Consequently, the methods solve the same nominal game but apply differing requirements on initialization, intermediate iterates, and convergence interpretation [3], [5].

Preserving this distinction is crucial to the thesis. More specifically, if the two methods were reduced to one generic Riccati solver, then a later comparison could not determine whether a difference in convergence behavior arose from fixed-policy evaluation, repeated optimality updates, or an implementation choice. As a result, the algorithms are developed separately and remain recognizable as the model-based procedures that were organized by Rizvi and Lin [5]. At the same time, the chapter does not present the inherited Riccati equations as being novel. Instead, this thesis's contribution begins where the nominal algorithms end; more specifically, persistent command offsets are estimated from known-model transition residuals, the estimates are subtracted from the player commands, and the resulting residual bias dynamics are verified under the same four execution cases established in the prior chapter.

It is also essential to define the chapter boundary. Chapter 4 contains the mathematical formulations, the original algorithmic cores, the disturbance-aware extensions, and the conditions under which those ex-

tensions are meaningful. However, it does not report numerical convergence values, calibration outcomes, figures, or comparative performance. In Chapter 6, these items will be discussed where the model-based and model-free methods are evaluated under a single MATLAB protocol. This separation allows this chapter to discuss why the algorithms and extensions are constructed in a particular manner before the numerical evidence is introduced.

4.2. Modified Policy Iteration

The model-based information structure assumes that the nominal plant matrices and quadratic weights are available to the controller design. The plant is defined by the expression

$$x_{k+1} = Ax_k + B_P u_{P,k} + B_E u_{E,k}, \tag{4.1}$$

where $x_k \in \mathbb{R}^n, u_{P,k} \in \mathbb{R}^{m_P}$, and $u_{E,k} \in \mathbb{R}^{m_E}$. The pursuer minimizes and the evader maximizes

$$J(x_0, u_P, u_E) = \sum_{k=0}^{\infty} \left(x_k^\top Q x_k + u_{P,k}^\top R_P u_{P,k} - u_{E,k}^\top R_E u_{E,k} \right), \tag{4.2}$$

with $Q \succeq 0, R_P \succ 0$, and $R_E \succ 0$. The stationary feedback policies are given by

$$u_{P,k} = Lx_k, \quad u_{E,k} = Kx_k, \tag{4.3}$$

and the corresponding nominal closed-matrix is

$$A_{\text{cl}} = A + B_P L + B_E K. \tag{4.4}$$

Recall that in Chapter 3, the signs of the feedback directions are contained in the numerical values of L and K. Thus, this convention allows for the derivation to be consistent with the source code algorithms as well as avoids introducing an addition sign convention when the same gains are later used in the command-compensation layer.

The known-model access serves two distinct roles. First, the matrices enter directly into policy evaluation, Riccati value updates, and saddle-gain recovery. Second, the same matrices make it possible to estimate persistent command offsets from observed transition residuals. These uses should not be combined conceptually. The Riccati algorithm answers which nominal policies solve the zero-sum game.

The residual estimator answers why the realized plant transition differs from the transition that is predicted by those nominal commands. Thus, the estimator is attached to the execution layer rather than inserted into the GARE.

For a symmetric matrix P, define the quadratic input and state-coupling blocks as follows

$$H_{PP}(P) = R_P + B_P^\top P B_P, \tag{4.5}$$

$$H_{PE}(P) = B_P^\top P B_E, \tag{4.6}$$

$$H_{EP}(P) = B_E^\top P B_P, \tag{4.7}$$

$$H_{EE}(P) = B_E^\top P B_E - R_E, \tag{4.8}$$

$$H_{Px}(P) = B_P^\top P A, \tag{4.9}$$

$$H_{Ex}(P) = B_E^\top P A. \tag{4.10}$$

Then, it follows that the joint game matrix and state-coupling matrix are defined by

$$G(P) = \begin{bmatrix} H_{PP}(P) & H_{PE}(P) \\ H_{EP}(P) & H_{EE}(P) \end{bmatrix}, \quad F(P) = \begin{bmatrix} H_{Px}(P) \\ H_{Ex}(P) \end{bmatrix}. \tag{4.11}$$

The stationary gain pair associated with P satisfies the expression

$$G(P) \begin{bmatrix} L \\ K \end{bmatrix} = -F(P). \tag{4.12}$$

This equation displays the coupled nature of the game, as the off-diagonal terms H_{PE} and H_{EP} demonstrate that the pursuer and evader policies are not obtained from independent LQR problems. Each input direction changes the stationary condition faced by the other player, and the gain pair as a result must be recovered from a joint minimax system.

The symbolic solution can be written with $G(P)^{-1}$, however the mathematical object of interest is the solution of the linear system, not the inverse itself. A numerically finite inverse does not guarantee that the game matrix is well-conditioned, or that the resulting stationary point has the correct curvature. For that reason, the later validity conditions distinguish three questions. More specifically, whether the linear equations can be solved, whether the recovered closed-loop is stable, and whether the local input

Hessian has the required minimizing and maximizing signs [70].

4.3. Nominal GARE Policy Iteration

Admissible Initialization and Fixed-Policy Evaluation: Policy iteration commences with an admissible gain pair (L^0, K^0). At iteration i, the current closed-loop matrix is

$$A^i = A + B_P L^i + B_E K^i. \tag{4.13}$$

In order for the infinite-horizon policy evaluation to be meaningful, A_i must be Schur stable; that is,

$$\rho(A^i) < 1. \tag{4.14}$$

This is a required condition before the value that is associated with the current policy pair can be evaluated. If an unstable pair were inserted into the infinite-horizon Lyapunov equation, then the returned matrix would not represent a finite quadratic value for the closed-loop game.

For an admissible pair, define the fixed-policy stage matrix given by

$$S^i = Q + (L^i)^\top R_P L^i - (K^i)^\top R_E K^i. \tag{4.15}$$

Then, the policy evaluation step solves

$$(A^i)^\top P^i A^i - P^i + S^i = 0, \tag{4.16}$$

or equivalently,

$$P^i = S^i + (A^i)^\top P^i A^i. \tag{4.17}$$

The equation can be interpreted by repeated substitution. When A^i is Schur stable,

$$P^i = \sum_{j=0}^{\infty} ((A^i)^\top)^j S^i (A^i)^j. \tag{4.18}$$

Thus, $x_0^\top P^i x_0$ is the accumulated quadratic game value that is generated by holding the current pursuer and evader policies as fixed. This representation clarifies why policy evaluation and policy improvement

are different operations. The Lyapunov equation does not question whether (L^i, K^i) is optimal; rather, it measures the infinite-horizon consequence of continuing to use that pair.

The existence of the Lyapunov solution alone is not enough to establish a valid zero-sum policy. Since the stage matrix contains the negative evader penalty, then it follows that S^i need not be positive definite even when Q, R_P, and R_E have their required signs. Thus, the interpretation of P^i remains tied to the zero-sum saddle structure rather than to a single-player positive-cost argument as the later curvature tests are what determine whether the stationary input pair has the correct convex-concave ordering.

Coupled Saddle-Policy Improvement: After P^i has been evaluated, then the current state-action quadratic form can be expressed as

$$\begin{aligned} Q^i(x, u_P, u_E) = x^\top (Q + A^\top P^i A)x + 2x^\top A^\top P^i B_P u_P + 2x^\top A^\top P^i B_E u_E \\ + u_P^\top H_{PP}(P^i)u_P + 2u_P^\top H_{PE}(P^i)u_E + u_E^\top H_{EE}(P^i)u_E. \end{aligned} \tag{4.19}$$

Then the stationary conditions with respect to the minimizing and maximizing inputs are

$$\frac{\partial Q^i}{\partial u_P} = 0, \quad \frac{\partial Q^i}{\partial u_E} = 0. \tag{4.20}$$

After substituting $u_P = L^{i+1}x$ and $u_E = K^{i+1}x$, it follows that these conditions produce

$$G(P^i) \begin{bmatrix} L^{i+1} \\ K^{i+1} \end{bmatrix} = -F(P^i). \tag{4.21}$$

Therefore, the gain pair is improved simultaneously. The pursuer update relies on the evader coupling blocks, and the evader update depends on the pursuer coupling blocks. This is the direct algebraic reason that the iteration cannot be interpreted as two independent regulator updates acting on the same plant.

Moreover, the source code algorithm expresses similar improvement through Schur complements. With all blocks evaluated at P^i,

$$L^{i+1} = -\left(H_{PP} - H_{PE}H_{EE}^{-1}H_{EP}\right)^{-1}\left(H_{Px} - H_{PE}H_{EE}^{-1}H_{Ex}\right), \tag{4.22}$$

$$K^{i+1} = -\left(H_{EE} - H_{EP}H_{PP}^{-1}H_{PE}\right)^{-1}\left(H_{Ex} - H_{EP}H_{PP}^{-1}H_{Px}\right). \tag{4.23}$$

These expressions make the elimination order more visible. The first equation accounts for the max-

Algorithm 1 Nominal Model-Based GARE Policy Iteration

1: Specify A, B_P, B_E, Q, R_P, and R_E.
2: Select an admissible initial pair (L^0, K^0) such that

$$\rho(A + B_P L^0 + B_E K^0) < 1,$$

and set $i = 0$.
3: Form $A^i = A + B_P L^i + B_E K^i$ and verify that A^i remains Schur stable.
4: Solve

$$(A^i)^\top P^i A^i - P^i + Q + (L^i)^\top R_P L^i - (K^i)^\top R_E K^i = 0.$$

5: Construct the game blocks from P^i and compute (L^{i+1}, K^{i+1}) from the coupled saddle-policy update.
6: Increment i and repeat the evaluation–improvement cycle until

$$\|P^i - P^{i-1}\|_F < \varepsilon_P.$$

imizing direction while forming the minimizing update, and the second accounts for the minimizing direction while forming the maximizing update. When the relevant blocks are nonsingular, it follows that the Schur complement formulas and the joint block solve produces the same gain pair.

The policy improvement state is only meaningful when the local game is well-posed. In particular, the pursuer block needs to satisfy

$$H_{PP}(P^i) \succ 0, \tag{4.24}$$

and the evader Schur complement must satisfy

$$S_E(P^i) = H_{EE}(P^i) - H_{EP}(P^i) H_{PP}(P^i)^{-1} H_{PE}(P^i) < 0. \tag{4.25}$$

The first inequality gives positive curvature in the minimizing input, while the second gives negative curvature in the maximizing direction after the pursuer input has been eliminated. A stationary solution that violates either sign condition is not the saddle point required by the game objective.

The Nominal Model-Based GARE Policy Iteration Algorithm 1, adapted from Rizvi and Lin's Algorithm 3.1 uses the pursuer-evader notation of this thesis and preserves the nominal structure [5]. The current polices are evaluated, then the two policies are improved through the game blocks, and the cycle is repeated. Moreover, additional histories of L and K may be recorded later for validation, but those histories do not redefine the source algorithm. Their purpose is to confirm that a small value-matrix

change is accompanied by stable policy quantities.

Convergence Interpretation and Nominal Verification: A decreasing iteration change does not by itself prove that the final matrices solve the game. Rather, the converged result must satisfy the fixed-point, closed-loop, and curvature conditions that were introduced in the previous chapter. Define the Riccati mapping as

$$\mathcal{T}(P) = Q + A^\top PA - F(P)^\top G(P)^{-1} F(P). \tag{4.26}$$

The GARE residual is given by

$$R_{\text{GARE}} = P - \mathcal{T}(P), \tag{4.27}$$

and the normalized residual is

$$\varepsilon_{\text{GARE}} = \frac{\|R_{\text{GARE}}\|_F}{1 + \|P\|_F}. \tag{4.28}$$

In addition, policy iteration also provides an independent closed-loop identity that is expressed as

$$R_{\text{Lyap}} = P - \left(Q + L^\top R_P L - K^\top R_E K + A_{\text{cl}}^\top P A_{\text{cl}}\right). \tag{4.29}$$

Agreement between the GARE and Lyapunov forms is crucial, since the equations arise from differing viewpoints. The GARE residual tests optimality, while the Lyapunov residual tests whether the final value matrix evaluates the final gain pair. A small value change together with large equation residuals would indicate stagnation or an inconsistent implementation rather than convergence to the intended saddle solution.

The final spectral radius and curvature conditions complete the nominal verification. The spectral radius is used to establish that the gain pair generates a Schur-stable nominal closed-loop, while the curvature signs establish that the stationary input pair has the minimizing-maximizing ordering of the game. The reciprocal conditions number of $G(P)$ identifies whether those conclusions rest on a well-resolved block solve or on a nearly singular numerical system. These tests do not add a new controller. Instead, they establish that the output of the inherited controller computation is mathematically interpretable.

4.4. Disturbance-Aware Extension of GARE Policy Iteration

Reasoning for Application of the Extension after Nominal Convergence: The nominal policy iteration procedure assumes that the command produced by each feedback law is the input that enters the plant. The disturbance-aware model separates those quantities as follows:

$$u_{P,k}^{\text{act}} = u_{P,k}^{\text{cmd}} + b_P, \quad u_{E,k}^{\text{act}} = u_{E,k}^{\text{cmd}} + b_E. \tag{4.30}$$

When the uncompensated commands are $u_{P,k}^{\text{cmd}} = Lx_k$ and $u_{E,k}^{\text{cmd}} = Kx_k$, it follows that the realized dynamics become

$$x_{k+1} = A_{\text{cl}}x_k + B_P b_P + B_E b_E. \tag{4.31}$$

The constant forcing term changes the equilibrium of the executed plant, but it does not change the linear part A_{cl}. Therefore, a nominally Schur-stable controller can remain stable while producing a persistent state displacement.

The offsets are not inserted into the Lyapunov policy evaluation equation nor into the saddle-gain improvement as doing so would define an affine optimal control problem with a different value function structure. This could be a potential research direction, but it is not explored here. This thesis preserves the nominal zero-sum game and treats the offsets as execution-side mismatches that are estimated and canceled after the nominal gain pair has been obtained.

Moreover, this ordering also creates a clear diagnostic hierarchy. A failed GARE residual, curvature test, or spectral radius check indicates a problem within the algorithm. A valid nominal controller followed by a biased trajectory indicates a command-channel mismatch. The failure of exact compensation to reproduce the nominal trajectory indicates inconsistent command and realized-input bookkeeping. By keeping the layers separate, it follows that the disturbance-aware study can attribute an observed error to the controller, the estimator, or the execution model rather than combining those causes into one modified Riccati equation.

Known-Model Transition Residual Estimation: During calibration, define the one-step transition residual expression given by

$$r_k = x_{k+1} - Ax_k - B_P u_{P,k}^{\text{cmd}} - B_E u_{E,k}^{\text{cmd}} \tag{4.32}$$

Substitution of the realized-input equations gives

$$r_k = G_b b + v_k, \quad G_b = \begin{bmatrix} B_P & B_E \end{bmatrix}, \quad b = \begin{bmatrix} b_P \\ b_E \end{bmatrix}, \tag{4.33}$$

where v_k collects transition-measurement noise and effects not represented by the constant-offset model. The residual has a simple interpretation; that is, the residual is the portion of the observed transition that cannot be explained by the nominal plant and the recorded commands.

For N_{cal} samples with constant offsets, the sample mean satisfies the expression

$$\bar{r} = G_b b + \bar{v}, \quad \bar{r} = \frac{1}{N_{\mathrm{cal}}} \sum_{k=1}^{N_{\mathrm{cal}}} r_k. \tag{4.34}$$

Then, the least-squares estimate is expressed by

$$\widehat{b} = \arg\min_{\beta} \|G_b \beta - \bar{r}\|_2^2 = G_b^{\dagger} \bar{r}. \tag{4.35}$$

A stacked formulation that uses all residuals produces the same estimate when the bias map is constant and the samples are equally weighted. The sampled-mean form makes the role of a persistent offset especially clear; moreover, repeated measurements reduce the random component of the residual while the constant component remains.

The individual offsets are only identifiable if they satisfy the expression

$$\mathrm{rank}(G_b) = m_P + m_E. \tag{4.36}$$

This is both a physical and numerical condition. More specifically, if B_P and B_E act as indistinguishable directions, then the transition data may reveal the net forcing $B_P b_P + B_E b_E$ without revealing how that forcing should be assigned to the command channels. In that situation, exact cancellation of the net disturbance may still be possible under additional design choices, however the separate pursuer and evader offsets are not uniquely determined by the residual equation alone.

Conditioning is able to determine how strongly noise is amplified by the identification map. When G_b is full column rank, but nearly rank deficient, it follows that small transition errors can produce comparatively large changes in $\hat{b}$. For independent isotropic residual noise with a covariance $\sigma_v^2 I$, the sample-mean estimator covariance is given by

$$\mathrm{Cov}(\widehat{b}) = \frac{\sigma_\nu^2}{N_{\mathrm{cal}}} \left(G_b^\top G_b \right)^{-1}. \tag{4.37}$$

The expression separates three influences on estimator speed; more specifically, the transition-noise level, the calibration length and the geometry of the two input directions. If N_{cal} is increased, then the it reduces the covariance by averaging, while poor directional separation in G_b increases the covariance through the inverse Gram matrix. Later, in Chapter 6, this prediction is evaluated numerically rather than assumed from only one calibration run.

Affine Compensation and Residual Bias Dynamics: Now, partition $\hat{b} = \begin{bmatrix} \hat{b}_P & \hat{b}_E \end{bmatrix}^\top$, then it follows that the compensated commands are given by

$$u_{P,k}^{\mathrm{cmd}} = Lx_k - \hat{b}_P, \quad u_{E,k}^{\mathrm{cmd}} = Kx_k - \hat{b}_E. \tag{4.38}$$

Then, the realized inputs are given as

$$u_{P,k}^{\mathrm{act}} = Lx_k + \tilde{b}_P, \quad u_{E,k}^{\mathrm{act}} = Kx_k + \widetilde{b}_E, \tag{4.39}$$

where

$$\tilde{b}_P = b_P - \hat{b}_P, \quad \tilde{b}_E = b_E - \hat{b}_E. \tag{4.40}$$

Hence,

$$x_{k+1} = A_{\mathrm{cl}} x_k + B_P \tilde{b}_P + B_E \tilde{b}_E. \tag{4.41}$$

The compensation law does not remove the effect of estimation error as it replaces the original offsets with the residual offsets. This distinction is crucial since it links the estimator directly to the remaining state displacement.

Let x_k^{nom} denote the nominal trajectory and let x_k^{comp} denote the estimation-compensation trajectory from the same initial condition. The deviation given by

$$d_k = x_k^{\mathrm{comp}} - x_k^{\mathrm{nom}}, \tag{4.42}$$

which satisfies

$$d_{k+1} = A_{\text{cl}} d_k + B_P \tilde{b}_P + B_E \tilde{b}_E, \quad d_0 = 0. \tag{4.43}$$

Hence,

$$d_k = \sum_{j=0}^{k-1} A_{\text{cl}}^j \left(B_P \tilde{b}_P + B_E \tilde{b}_E \right). \tag{4.44}$$

When A_{cl} is Schur stable, then

$$d_\infty = (I - A_{\text{cl}})^{-1} \left(B_P \tilde{b}_P + B_E \tilde{b}_E \right). \tag{4.45}$$

(4.45) shows why the same scalar estimation error can have different state consequences under different closed-loop dynamics. The residual bias is first mapped through the input direction and is then amplified or attenuated by $(I - A_{\text{cl}})^{-1}$.

It then follows immediately that a useful norm bound can be given by

$$\|d_\infty\|_2 \leq \|(I - A_{\text{cl}})^{-1}\|_2 \left(\|B_P\|_2 \|\tilde{b}_P\|_2 + \|B_E\|_2 \|\tilde{b}_E\|_2 \right). \tag{4.46}$$

The bound is not used as a tight performance prediction; rather, its role is interpretive as well as to separate estimate accuracy, input-direction magnitude, and closed-loop sensitivity, thereby preventing compensation quality from being judged solely by bias error.

Exact compensation sets $\hat{b}_P = b_P$ and $\hat{b}_E = b_E$. In that case, $\tilde{b}_P = \tilde{b}_E = 0$ and the realized dynamics can be reduced to

$$x_{k+1} = A_{\text{cl}} x_k. \tag{4.47}$$

Thus, the exact case is a verification condition, as it does not assume that unknown offsets can be known in practice. In addition, the exact case questions whether the command transformation and realized-input equations are internally consistent. If exact cancellation fails to reproduce the nominal trajectory, then the discrepancy cannot be attributed to estimator noise.

The Disturbance-Aware Extension of GARE Policy Iteration (Algorithm 2) maintains its nominal core

Algorithm 2 Disturbance-Aware Extension of GARE Policy Iteration

Phase I: Compute and verify the nominal policy-iteration solution.
Execute Algorithm 4.1 until the value matrix and gain pair have converged.
Verify the GARE residual, Lyapunov residual, closed-loop spectral radius, saddle curvature, and game-matrix conditioning.
Phase II: Estimate the persistent command offsets.
Record x_k, x_{k+1}, $u_{P,k}^{\text{cmd}}$, and $u_{E,k}^{\text{cmd}}$ during calibration.
Form $r_k = x_{k+1} - Ax_k - B_P u_{P,k}^{\text{cmd}} - B_E u_{E,k}^{\text{cmd}}$.
Compute $\widehat{b} = G_b^{\dagger}\bar{r}$ through a least-squares solve.
Verify the rank and conditioning of G_b and determine the estimator covariance under the stated noise model.
Phase III: Apply the four matched execution cases.
Evaluate nominal operation with zero offsets.
Evaluate uncompensated operation with the true offsets added to the realized channels.
Evaluate estimated compensation using $Lx_k - \widehat{b}_P$ and $Kx_k - \widehat{b}_E$.
Evaluate exact compensation using $Lx_k - b_P$ and $Kx_k - b_E$.
Phase IV: Verify the disturbance-aware layer.
Report the offset-estimation errors and the residual affine forcing $B_P\widetilde{b}_P + B_E\widetilde{b}_E$.
Compare the observed state deviation with the analytical affine recurrence.
Verify that exact compensation reproduces the nominal trajectory to numerical precision.

from 1, as the calibration, execution, and verification phases are the thesis extension. Moreover, the extension only begins after the nominal policy iteration solution has been verified. By removing Phases II-IV recovers the inherited model-based algorithm. While retaining them produces the disturbance-aware procedure without changing the Lyapunov evaluation nor the saddle-policy improvement. This is a crucial distinction that is the key structural claim of this chapter.

4.5. Nominal GARE Value Iteration

Riccati Optimality Mapping from the One-Step Game: Value iteration approaches the same GARE solution without solving a complete fixed-policy Lyapunov equation at every step. Suppose that the current value approximation is given by the expression

$$V^i(x) = x^\top P^i x. \tag{4.48}$$

The one-step dynamic programming expression is

$$\begin{aligned} V^{i+1}(x) = \min_{u_P}\max_{u_E}\Big[&x^\top Qx + u_P^\top R_P u_P - u_E^\top R_E u_E \\ &+ (Ax + B_P u_P + B_E u_E)^\top P^i (Ax + B_P u_P + B_E u_E)\Big]. \end{aligned} \tag{4.49}$$

Then, the stationary input pair that is associated with P^i satisfies

$$G(P^i)\begin{bmatrix} L^i \\ K^i \end{bmatrix} = -F(P^i). \tag{4.50}$$

Substitution of the stationary inputs into the one-step expression produces the Riccati mapping

$$\mathcal{T}(P^i) = Q + A^\top P^i A - F(P^i)^\top G(P^i)^{-1} F(P^i), \tag{4.51}$$

and the value update is

$$P^{i+1} = \mathcal{T}(P^i). \tag{4.52}$$

The above derivation explains why value iteration is not merely policy iteration written with different indices. Policy iteration evaluates the infinite-horizon value of a fixed gain pair, while value iteration performs one optimality backup with the current value approximation and advances directly to the next matrix.

An intermediate value iteration matrix does not necessarily equal the complete infinite-horizon value of the gain pair displayed at that same iteration. Moreover, its meaning is instead tied to repeated application of the optimality operator. This difference affects how intermediate stability is interpreted. Policy iteration requires an admissible current pair before each Lyapunov evaluation. On the other hand, value iteration requires the game blocks to remain well-posed during each Riccati step and requires the final recovered gain pair to satisfy the stabilizing saddle conditions.

Gain Recovery and Iterate Definition: After P^{i+1} has been formulated, then the gain pair that is associated with the updated value matrix is recovered from

$$G(P^{i+1})\begin{bmatrix} L^{i+1} \\ K^{i+1} \end{bmatrix} = -F(P^{i+1}). \tag{4.53}$$

The recovery equation has the same block structure that is used by policy improvement; moreover, what changes is the origin of the value matrix that is inserted into the blocks. In policy iteration, P^i is obtained by evaluating the current policies. In value iteration, P^{i+1} is obtained by applying the optimality map to the previous value approximation.

Algorithm 3 Nominal Model-Based GARE Value Iteration

1: Specify A, B_P, B_E, Q, R_P, and R_E.
2: Select a symmetric initial value approximation P^0 for which the required block solves are well posed, and set $i = 0$.
3: Construct $G(P^i)$ and $F(P^i)$.
4: Apply

$$P^{i+1} = Q + A^\top P^i A - F(P^i)^\top G(P^i)^{-1} F(P^i).$$

5: Recover the current gain pair from

$$G(P^{i+1}) \begin{bmatrix} L^{i+1} \\ K^{i+1} \end{bmatrix} = -F(P^{i+1}).$$

6: Check block conditioning, saddle curvature, and the spectral radius of $A + B_P L^{i+1} + B_E K^{i+1}$.
7: Stop when

$$\|P^{i+1} - P^i\|_F < \varepsilon_P;$$

otherwise increment i and repeat the Riccati mapping.

The gain sequence is still important even though value iteration is organized around the value matrix. Monitoring the recovered gains will show whether the policy quantities approach stable values as the Riccati sequence converges. In addition, it also permits the same final checks used by policy iteration; that is, the closed-loop spectral radius, the pursuer curvature, the evader Schur complement, and the block-matrix conditioning. These checks do not turn value iteration into policy iteration, as they determine whether the stationary policies implied by the current value approximation are numerically and game-theoretically meaningful.

The final convergence test again requires more than a small iteration change. More specifically,the converged matrix must satisfy the expression,

$$P = \mathcal{T}(P), \tag{4.54}$$

and the recovered gains must generate a Schur stable closed-loop with the correct saddle curvature. When policy iteration and value iteration both converge to the same stabilizing GARE solution, then their final matrices and gains should agree within numerical tolerance. In addition, their intermediate histories may differ since one sequence is built from fixed-policy evaluations and the other from repeated optimality backups.

Initialization and Well-Posedness: Policy iteration and value iteration impose different initialization demands. Policy iteration begins with a gain pair whose closed-loop is already admissible, while value iteration begins with a symmetric matrix P^0 for which the game matrix and Riccati update are defined. A convenient value initialization does not guarantee that every intermediate block matrix will remain well-conditioned or that every recovered gain pair will be stabilizing. As a result, the iteration must remain within a region where the required minimax solves are meaningful.

This observation is one reason not to make a universal convergence claim from one numerical example. More specifically, this thesis verifies convergence for the selected benchmark and reports the conditions that support that conclusion. It does not claim that every symmetric initialization converges for every zero-sum plant. This an active area within the literature as it continues to study initialization, monotonicity, admissibility, and single-loop variants precisely because these properties depend on the game and on the chosen iteration structure [7].

The final fixed-point residual is crucial for value iteration since the algorithm 3 applies the Riccati map directly, the residual

$$R_{\mathrm{GARE}} = P - \mathcal{T}(P) \tag{4.55}$$

checks the defining equation of the limit. A small $\|P^{i+1} - P^i\|_F$ indicates that the sequence has slowed, while a small GARE residual indicates that the resulting matrix satisfies the intended optimality relation. Therefore, the two measurements answer different questions and will be elaborated on further in Chapter 6.

4.6. Disturbance-Aware Extension of GARE Value Iteration

The command offsets are properties of the execution channels, not properties of the numerical route that is used to calculate the nominal gain pair. As a result, the value iteration extension uses the same residual estimator and affine command law developed for policy iteration as only the nominal computation phase changes. After value iteration converges, then the final matrix P and recovered gains (L, K) are verified through the GARE residual, spectral radius, saddle curvature, and game matrix conditioning. Then, calibration forms

$$r_k = x_{k+1} - Ax_k - B_P u_{P,k}^{\mathrm{cmd}} - B_E u_{E,k}^{\mathrm{cmd}}, \tag{4.56}$$

solves for $\hat{b}$, and applies

$$u_{P,k}^{\text{cmd}} = Lx_k - \hat{b}_P, \quad u_{E,k}^{\text{cmd}} = Kx_k - \hat{b}_E. \tag{4.57}$$

The compensated closed-loop is again given by

$$x_{k+1} = A_{\text{cl}}x_k + B_P\tilde{b}_P + B_E\tilde{b}_E. \tag{4.58}$$

Hence, the theoretical relationship between estimation error and residual state displacement is identical for the two model-based methods once their final gains are fixed.

Using the same estimator and compensation law is essential for a fair comparison. If policy iteration and value iteration were paired with different calibration procedures, a later difference in recovery could not be attributed cleanly to the nominal iteration structure. Holding the disturbances-aware layer fixed ensures that the model-based comparison asks a single question. That is, how do two established Riccati procedures reach and verify the nominal saddle solution before being subjected to the same execution mismatch?

The nominal core of Algorithm 4 is Algorithm 3, and the remaining phases match the policy iteration extension. The difference between Algorithms 2 and 4 is concentrated in Phase I. More specifically, while policy iteration reaches the nominal solution through repeated evaluate-improve cycles, value iteration reaches it through repeated Riccati optimality updates. After convergence, both methods supply a nominal gain pair to the same calibration, compensation, and verification framework.

Algorithm 4 Disturbance-Aware Extension of GARE Value Iteration

The nominal core is Algorithm 4.3. The remaining phases match the policy-iteration extension.
Phase I: Compute and verify the nominal value-iteration solution.
Execute Algorithm 4.3 until the Riccati sequence and recovered gains have converged.
Verify the GARE residual, closed-loop spectral radius, saddle curvature, and game-matrix conditioning.
Phase II: Estimate the persistent command offsets.
Collect calibration transitions and the corresponding commanded inputs.
Form the known-model residuals and compute $\hat{b} = G_b^{\dagger}\bar{r}$.
Verify the rank and conditioning of G_b and determine the estimator covariance under the stated noise model.
Phase III: Apply the matched execution cases.
Evaluate nominal and uncompensated operation with the converged value-iteration gains.
Evaluate estimated compensation using $Lx_k - \hat{b}_P$ and $Kx_k - \hat{b}_E$.
Evaluate exact compensation using the true offsets.
Phase IV: Verify the disturbance-aware layer.
Report the offset-estimation errors, residual affine forcing, and predicted residual equilibrium.
Compare the simulated deviation with the analytical recurrence.
Verify that exact compensation reproduces the nominal trajectory to numerical precision.

4.7. Similar Theoretical Properties of the Model-Based Extensions

Invariance of the Nominal Riccati Solution: The disturbance-aware layer does not change the nominal value matrix nor feedback gains. The gains are calculated from the zero-offset game, and compensation is applied as an affine translation of the commands. Thus, the matrices P, L, and K remain the solution of the nominal GARE. This invariance is intentional as it makes the original algorithms recoverable and allows the effect of command mismatch to be studied without redefining the objective or introducing an affine value function.

The word *extension* should thus be interpreted carefully. More specifically, the thesis extends what the established algorithms do in execution and validation. It does not claim to derive a new Riccati equation. The nominal controller remains linear in the state, while the executed commands becomes affine because the offset estimate is subtracted. This is sufficient for matched constant command bias since the disturbance enters through the same channels as the players inputs.

Stability versus Offset-Free Regulation: Since the persistent offsets enter additively, the linear part of the closed-loop remains A_{cl}. If A_{cl} is said to be Schur stable, then the biased and compensated systems remain bounded under constant offsets. However, their equilibrium points are generally nonzero:

$$x_{\infty}^{\text{bias}} = (I - A_{\text{cl}})^{-1}(B_P b_P + B_E b_E), \tag{4.59}$$

$$x_{\infty}^{\text{comp}} = (I - A_{\text{cl}})^{-1}(B_P \tilde{b}_P + B_E \tilde{b}_E). \tag{4.60}$$

The above equations distinguish stability from regulation. A controller can preserve bounded behavior and still fail to reproduce the nominal trajectory since the executed command contains a constant mismatch. Compensation improves regulation by reducing the affine forcing, not by moving the eigenvalues of the nominal closed-loop matrix.

As a result, this distinction prevents an overstatement of this contribution. The method is not a robust redesign that changes the worst-case saddle solution for a new disturbance set. Instead, it is an offset-aware execution layer for matched persistent command errors, as time-varying faults, unmatched disturbances, saturation, and state-estimation error would require additional analysis and are not resolved merely by subtracting a constant estimate.

Consistency of Exact and Estimated Compensation: Exact compensation provides an algebraic consistency check as with $\hat{b} = b$, the realized inputs equal the original policy inputs and the state recurrence is identical top the nominal recurrence. From a matched initial condition,

$$x_k^{\text{exact}} = x_k^{\text{nom}} \tag{4.61}$$

for every k in exact arithmetic as any numerical difference should therefore be at the level that is expected from floating point operations.

Estimated compensation provides a graded rather than binary result as its effectiveness is governed by $\tilde{b}$, not by the magnitude of the original offsets alone. In addition, the residual state error is also shaped by B_P, B_E, and A_{cl}. Therefore, the thesis reports both bias-estimation error and state-deviation recovery. A small estimator error is useful evidence, however it is not a complete description of closed-loop recovery.

4.8. Comparison Between Original and Disturbance-Aware Procedures

Table VI separates the original model-based algorithms for the extended thesis algorithms [5]. The distinction is necessary since the phrase *disturbance-aware GARE algorithm* could otherwise suggest that the Riccati equations themselves have been replaced and that is not the formulation here.

The table establishes the appropriate novelty claim. Algorithms 2 and 4 are not replacements for the original Riccati procedures as their nominal cores remain identifiable and can be recovered by removing the calibration and compensation phases. The contribution here is the mathematically explicit bridge from those cores to persistent command mismatch, together with verification conditions that show when the bridge behaves as intended.

Moreover, this model-based chapter also provides the reference structure for the next chapter. The model-free methods will not use A, B_P, or B_E in their Bellman regressions, however they will seek the same nominal saddle policies and will use the same four execution cases. Therefore, the differences between the chapters is an information structure difference rather than a change in the underlying game.

TABLE VI: Relationship Between the Original Model-Based Algorithms and the Disturbance-Aware Algorithms

Component	Original Model-Based Procedure	Disturbance-Aware Thesis Procedure
Nominal update	Uses GARE policy iteration or GARE value iteration to compute the saddle solution.	Preserves the same Lyapunov, Riccati, and coupled gain equations.
Input assumption	Treats the computed policy input as the input applied to the plant.	Separates policy, commanded, and realized inputs through persistent offsets.
Offset estimation	Contains no persistent command-offset calibration.	Uses known-model transition residuals to estimate the pursuer and evader offsets.
Command law	Applies Lx_k and Kx_k directly.	Applies $Lx_k - \hat{b}_P$ and $Kx_k - \hat{b}_E$ to the biased command channels.
Execution cases	Evaluates nominal operation.	Evaluates nominal, uncompensated, estimated-compensation, and exact-compensation cases under matched conditions.
Verification	Focuses on nominal convergence and the Riccati solution.	Retains nominal checks and adds identifiability, estimator covariance, residual-affine, analytical-recurrence, and exact-trajectory checks.
Claim	Produces a model-based zero-sum saddle solution for the stated LQ game.	Extends the verified nominal solution with a matched execution-side calibration and compensation layer.

4.9. Summary and Transition

This chapter developed the two model-based algorithmic procedures that are used in this thesis. The GARE policy iteration algorithm begins with an admissible gain pair, evaluates its infinite-horizon

value through a discrete Lyapunov equation, and improves the pursuer and evader policies through a coupled saddle solve. The GARE value iteration algorithm begins with a symmetric value approximation, advances that matrix through the Riccati optimality map, and then recovers the associated gains. Both algorithms seek the same stabilizing GARE solution, however their intermediate matrices and convergence interpretations remain distinctive.

Then, the chapter attached the common persistent command-offset model to each procedure. Since the plant matrices are known, transition residuals provide a direct calibration equation for the two command offsets. The rank and conditioning of the combined input map determine whether those offsets can be separated reliably, while the estimator covariance predicts how calibration noise propagates into the estimates. Affine compensation subtracts the estimates from the nominal policy commands and leaves only the residual estimation errors in the realized closed-loop.

Hence, the resulting deviation equations connect calibration accuracy to state recovery. Exact compensation reproduces the nominal dynamics and thus serves as an implementation check. Estimated compensation produces a residual affine forcing whose effect relies on the bias errors, the player input directions, and the closed-loop sensitivity. These relationships preserve the nominal Riccati equations while making the execution mismatch explicit and testable.

There are no numerical results that are presented in this chapter. This is relegated to Chapter 6, as it implements the equations that are developed here under the common benchmark, reports the convergence and validity diagnostics, evaluates the offset estimator, and compares nominal, uncompensated, estimated compensation, and exact compensation behavior. Before that numerical comparison, the next chapter will develop the corresponding model-free quadratic Q-learning policy iteration and value iteration algorithms under the same game and disturbance-aware framework.

5. Disturbance-Aware Model-Free Quadratic Q-Learning Policy and Value Iteration

5.1. Introduction

Recall that Chapter 3 established the common mathematical framework that is used by all four algorithms within this thesis, and the previous chapter developed the two model-based algorithms that use the known plant matrices to compute the nominal saddle solution. The objective of this chapter is to develop the corresponding model-free algorithms. These methods address the same discrete-time zero-sum pursuit-evasion game, utilize the same quadratic objective, and seek the same pursuer and evader feedback policies. Moreover, their defining difference is the information that is supplied to the update. Rather than inserting A, B_P, and B_E into a Riccati or Lyapunov equation, the model-free algorithms reconstruct the required quadratic relationships from sampled transitions.

The two algorithms are quadratic Q-learning policy iteration and quadratic Q-learning value iteration. Both algorithms are organized around a joint state-action value function whose matrix contains the information that is needed to recover the minimizing and maximizing policies. Policy iteration evaluates the Q-function that is associated with a fixed gain pair and then improves that pair, while value iteration advances the Q-function through a sampled optimality backup and recovers the gains from the updated matrix. As a result, the distinction between the methods is not whether one uses data and the other does not as both are data-based. The distinction between these methods is not whether one uses data or not; rather, it lies in how the Bellman relation is arranged during the iteration and in the meaning that is assigned to the intermediate Q-function estimates [4]–[6].

Hence, preserving that distinction is crucial for the thesis comparison. If both learning algorithms were reduced to one generic least-squares routine, a later difference in convergence could not be attributed to fixed-policy evaluation, bootstrapped value updates, or the conditioning of the associated regression. Therefore, the nominal policy iteration and value iteration algorithms remain separately identifiable. Neither algorithm is claimed as a novel reinforcement learning contribution; instead, the nominal Q-learning algorithms are adapted from Rizvi and Lin [5]. The thesis contribution begins with their application under imperfect command realization; more specifically, persistent command offsets are estimated from measured command mismatch, transitions are labeled with the realized inputs that produced them, and the learned policies are deployed through the affine compensation law that was introduced in Chapter 3.

The phrase *model-free* requires a precise boundary. In this chapter, it means that the Bellman regressions used to estimate the quadratic action-value matrix do not receive the analytical transition matrices. The

simulated environment may contain those matrices, and the next chapter may use a model-based result as an external verification reference, but neither source is applied to the learner. Direct measurement of state, commanded input, realized input, stage cost, and successor state are allowed since they are part of the sampled interaction record. This distinction prevents command-channel calibration from being mistaken for a hidden Riccati computation.

Moreover, the chapter boundary is equally important. This chapter contains the mathematical parameterization, the nominal Q-learning algorithms, the disturbance-consistent data requirements, the affine extension, and the conditions under which the learned policies can be interpreted as valid zero-sum saddle policies. This chapter does not report the numerical batch size, exploration frequencies, tolerances, convergence histories, learned gains, figures, nor comparative performance. Those items are relegated to the next chapter, where all four methods are implemented and evaluated under one MATLAB protocol. Therefore, this chapter elaborates what must be computed and why, and the next chapter will provide the numerical evidence.

5.2. Model-Free Information Structure and Common Action-Value Objects

Measured Transition Tuples and Realized Stage Cost: The model-free learner operates on transition tuples of the form

$$\mathcal{D} = \left\{ \left(x_k, u_{P,k}^{\text{act}}, u_{E,k}^{\text{act}}, \ell_k, x_{k+1} \right) \right\}_{k=1}^{N}, \tag{5.1}$$

where $x_k \in \mathbb{R}^n$ denotes the measured game state, while $u_{P,k}^{\text{act}} \in \mathbb{R}^{m_P}$ and $u_{E,k}^{\text{act}} \in \mathbb{R}^{m_E}$ are the inputs that actually entered the plant, and x_{k+1} is the measured successor state. The corresponding one-step game cost is

$$\ell_k = x_k^\top Q x_k + (u_{P,k}^{\text{act}})^\top R_P u_{P,k}^{\text{act}} - (u_{E,k}^{\text{act}})^\top R_E u_{E,k}^{\text{act}}. \tag{5.2}$$

The pursuer minimizes the accumulated cost, while the evader maximizes it. The negative evader penalty preserves the minimax structure of the game and requires the same convex-concave curvature conditions that were used in the previous chapter.

The use of realized inputs in (5.1) and (5.2) is not a stylistic choice. A sampled Bellman equation associates a state-action pair with the transition and cost produced by that pair. If a command $u_{P,k}^{\text{cmd}}$ is stored

while the plant actually receives $u_{P,k}^{\text{cmd}}+b_P$, then the recorded action does not match the observed successor state. The regression may still return a finite coefficient vector, but it is fitting internally inconsistent data. The learned quadratic form would then mix policy approximation error with an unmodeled labeling error at the command interface.

This observation gives the disturbance-aware extension a data-level role that does not arise in the previous chapter. For the model-based algorithms, the known matrices permit the offset to be reconstructed from a state-transition residual after nominal controller design. For Q-learning, the action recorded in the dataset part of the Bellman feature vector. As a result, command realization must be handled before the data can be interpreted as samples of a state-action value function. Calibration and accurate action labeling does not alter the Bellman equation, rather, they ensure that the equation is applied to the transition that actually occurred.

Quadratic Joint State-Action Representation: Now, define the joint state-action vector that is given by

$$z_k = \begin{bmatrix} x_k \\ u_{P,k} \\ u_{E,k} \end{bmatrix} \in \mathbb{R}^{n_z}, \quad n_z = n + m_P + m_E. \tag{5.3}$$

For a linear-quadratic zero-sum game, the action-value function associated with a stationary policy pair is expressed as

$$Q(z_k) = z_k^\top H z_k, \quad H = H^\top. \tag{5.4}$$

Then, partition H according to the state, pursuer input, and evader input:

$$H = \begin{bmatrix} H_{xx} & H_{xP} & H_{xE} \\ H_{Px} & H_{PP} & H_{PE} \\ H_{Ex} & H_{EP} & H_{EE} \end{bmatrix}. \tag{5.5}$$

Symmetry implies that $H_{Px} = H_{xP}^\top, H_{Ex} = H_{xE}^\top$, and $H_{EP} = H_{PE}^\top$ in exact arithmetic. Moreover, these relationships are enforced when the coefficient vector is reconstructed into a matrix, since a small asymmetric numerical component has no independent meaning in quadratic form.

The representation in (5.4) is structure matched to the game rather than selected as a generic nonlinear approximator. If a value matrix P were known, then the exact nominal action-value matrix would be

$$H(P) = \begin{bmatrix} Q + A^\top PA & A^\top PB_P & A^\top PB_E \\ B_P^\top PA & R_P + B_P^\top PB_P & B_P^\top PB_E \\ B_E^\top PA & B_E^\top PB_P & B_E^\top PB_E - R_E \end{bmatrix}. \tag{5.6}$$

The model-free algorithms do not reconstruct (5.6) by substituting A, B_P, B_E, and P. Rather, they estimate the independent entries of H from sampled Bellman relationships. Equation (5.6) is important as it establishes the target structure and explains why the learned input blocks have the same saddle interpretation as the GARE blocks in the previous chapter [4], [14].

Since H is said to be symmetric, then it follows that it contains

$$n_H = \frac{n_z(n_z + 1)}{2} \tag{5.7}$$

independent coefficients. Then, define a symmetric quadratic feature map $\phi : \mathbb{R}^{n_z} \to \mathbb{R}^{n_H}$ that contains each squared component of z and twice each distinct cross product. With a consistent ordering of the independent coefficients,

$$z^\top Hz = \phi(z)^\top \theta, \tag{5.8}$$

where $\theta \in \mathbb{R}^{n_H}$ denotes the vector of independent entries of H. The Q-function is quadratic in the state and inputs, but linear in the unknown parameters. This distinction is what allows for exact least-squares recovery in the ideal linear-quadratic setting when the data provides sufficient excitation.

The doubled-crossed terms in $\phi(z)$ are bookkeeping rather than additional model assumption. For example, if $H_{ij} = H_{ji}$ and $i \neq j$, then it follows that the quadratic form contains $2H_{ij}z_iz_j$. Including $2z_iz_j$ in the feature vector allows for the corresponding parameter to be stored once. A different symmetric-vectorization convention would be equally valid if the feature construction and matrix reconstruction remained mutually consistent.

Saddle-Policy Recovery from the Learned Matrix: For a fixed-state, it follows that the input-dependent part of (5.4) is

$$\begin{aligned} Q(x, u_P, u_E) = x^\top H_{xx}x + 2u_P^\top H_{Px}x + 2u_E^\top H_{Ex}x \\ + u_P^\top H_{PP}u_P + 2u_P^\top H_{PE}u_E + u_E^\top H_{EE}u_E. \end{aligned} \tag{5.9}$$

Then, the stationary conditions with respect to the minimizing and maximizing input are

$$\frac{\partial Q}{\partial u_P} = 0, \quad \frac{\partial Q}{\partial u_E} = 0. \tag{5.10}$$

Under stationary feedback $u_P = Lx$ and $u_E = Kx$, these conditions give

$$\underbrace{\begin{bmatrix} H_{PP} & H_{PE} \\ H_{EP} & H_{EE} \end{bmatrix}}_{\mathcal{H}_u} \begin{bmatrix} L \\ K \end{bmatrix} = - \underbrace{\begin{bmatrix} H_{Px} \\ H_{Ex} \end{bmatrix}}_{\mathcal{H}_x}. \tag{5.11}$$

Hence, once H has been estimated, the pursuer and evader gains are then recovered from a coupled block solve. The learner does not require a separate actor parameterization since the quadratic action-value matrix supplies an analytical saddle-policy recovery equation.

Moreover, a finite solution of (5.11) is not sufficient. The minimizing block must satisfy

$$H_{PP} \succ 0, \tag{5.12}$$

and the maximizing-player Schur complement must satisfy

$$S_E = H_{EE} - H_{EP} H_{PP}^{-1} H_{PE} \prec 0. \tag{5.13}$$

The first condition provides positive curvature in the pursuer input, while the second provides curvature in the evader direction after the minimizing input has been eliminated. These signs distinguish a local minimax saddle from a stationary point having the wrong game-theoretic ordering.

Additionally, the conditioning of $\mathcal{H}_u$ also matters as a nearly singular input block can produce large gain changes from small coefficients errors even when the least-squares estimate of H appears accurate in Frobenius norm. Thus, the learning problem contains two separate conditioning questions. More specifically, whether the feature regression reliably identifies θ as well as whether the recovered input block converts that estimate into reliable stable saddle gains. The next chapter reports both since good Bellman fit does not automatically imply robust policy recovery.

5.3. Nominal Q-Learning Policy Iteration

Fixed-Policy Bellman Equation: Q-learning policy iteration begins with a gain pair (L^i, K^i) and evaluates the quadratic action-value function that is generated when those policies are used at the successor state. For each measured transition, define the successor policy actions

$$u_{P,k+1}^{(i)} = L^i x_{k+1}, \quad u_{E,k+1}^{(i)} = K^i x_{k+1}, \tag{5.14}$$

as well as the corresponding successor state-action given by

$$z_{k+1}^{(i)} = \begin{bmatrix} x_{k+1} \\ L^i x_{k+1} \\ K^i x_{k+1} \end{bmatrix}. \tag{5.15}$$

For a fixed policy pair, the undiscounted Bellman identity is

$$Q^i(z_k) = \ell_k + Q^i\left(z_{k+1}^{(i)}\right). \tag{5.16}$$

Hence, substitution of (5.8) into (5.16) yields

$$\left[\phi(z_k) - \phi\left(z_{k+1}^{(i)}\right)\right]^\top \theta^i = \ell_k. \tag{5.17}$$

The equality is exact in the ideal deterministic linear-quadratic setting when the parameterization is exact and the current policy pair generates a finite infinite-horizon value. In finite data with numerical noise, the equations are solved in the least-squares manner.

This interpretation parallels model-based policy evaluation. Recall that in the previous chapter, the Lyapunov equation questions what value is generated by holding (K^i, L^i) fixed. Here, the feature-difference equation questions what quadratic state-action value is consistent with sampled transitions when the same fixed pair is used at the successor state. The analytical plant matrices have been replaced by measured one-step consequences, however the evaluate-improve logic remains recognizable.

Policy evaluation is only meaningful when the current pair produces a finite value over the sampled operating region. In the nominal linear setting, a Schur stable closed-loop provides the standard infinite-horizon interpretation. The learner does not need the matrices to enforce this property internally, but the

final policy can be checked by the external benchmark provided in Chapter 6. More generally, a dataset that wanders outside the region where the quadratic approximation is meaningful can invalidate thew interpretation even if the regression is full rank. Data coverage and closed-loop admissibility are thus related but distinct requirements.

Least-Squares Policy Evaluation and Feature Identifiability: First, stack the N fixed-policy Bellman equations into

$$\Psi^i \theta^i \approx \ell, \tag{5.18}$$

where the kth row of Ψ^i is given by

$$\left[\phi(z_k) - \phi\left(z_{k+1}^{(i)}\right)\right]^\top, \tag{5.19}$$

and $\ell = \left[\ell_1, \ldots, \ell_N\right]^\top$. Then, the policy evaluation estimate is

$$\widehat{\theta}^i = \arg\min_{\theta} \|\Psi^i \theta - \ell\|_2^2. \tag{5.20}$$

The matrix $\hat{H}^i$ is reconstructed from $\hat{\theta}^i$ and symmetrized. The next gain pair is the obtained from

$$\widehat{\mathcal{H}}_u^i \begin{bmatrix} L^{i+1} \\ K^{i+1} \end{bmatrix} = -\widehat{\mathcal{H}}_x^i. \tag{5.21}$$

The regression in (5.20) plays the role of policy evaluation, while the block solve in (5.21) plays the role of policy improvement.

Unique recovery of all quadratic coefficients requires

$$\operatorname{rank}(\Psi^i) = n_H. \tag{5.22}$$

This condition is stronger than requiring $N \geq n_H$. A large sample count can still produce a rank-deficient matrix if the states and player inputs do not excite independent quadratic directions. For example, applying proportional exploration to both players can make cross terms indistinguishable, while collecting data near a single state can leave most state-related features unresolved. Persistent excitation in this setting means that the joint state-action samples span the quadratic feature space that is relevant to H.

Algorithm 5 Nominal Model-Free Q-Learning Policy Iteration

1: Collect an off-policy transition batch $\{x_k, u_{P,k}, u_{E,k}, \ell_k, x_{k+1}\}_{k=1}^{N}$ with sufficient joint state–action excitation.
2: Construct z_k and the symmetric quadratic feature vector $\phi(z_k)$ for every sample.
3: Select an initial quadratic matrix H^0 or parameter vector θ^0, recover the corresponding gain pair (L^0, K^0), and set $i = 0$.
4: For the current gains, form $z_{k+1}^{(i)} = [x_{k+1}^\top, (L^i x_{k+1})^\top, (K^i x_{k+1})^\top]^\top$.
5: Build Ψ^i from the feature differences $[\phi(z_k) - \phi(z_{k+1}^{(i)})]^\top$.
6: Solve $\Psi^i \theta^i \approx \ell$ in the least-squares sense and reconstruct a symmetric H^i.
7: Recover (L^{i+1}, K^{i+1}) from the input-related blocks of H^i.
8: Verify feature rank, regression conditioning, Bellman residual, saddle curvature, and the changes in H, L, and K.
9: Increment i and repeat the fixed-policy evaluation–improvement cycle until the selected convergence conditions are satisfied.

Full numerical rank is necessary, however it is not sufficient. If the singular values of Ψ^i are highly unequal, measurement noise and finite precision can be strongly amplified in $\hat{\theta}^i$. Thus, the condition number describes reliability of the identified quadratic coefficients, while rank describes their formal identifiability. These quantities should not be replaced by regularization alone. A ridge term may reduce variance in an ill-conditioned problem, however it also changes the estimated Bellman solution and does not create information that the data never contained.

The fixed dataset may be reused across policy iterations since the measured transition on the left side does not change. However, what changes is the successor-policy feature given by $\phi(z_{k+1}^{(i)})$. Reuse is one of the practical advantages of off-policy least-squares policy iteration. More specifically, the behavior inputs that generated the batch need not equal the policy currently being evaluated, provided that the data sufficiently cover the relevant feature directions and the Bellman equations are formed with the measured actions [38], [40]. The benefit does not remove the need for careful coverage as reusing a poor batch simply repeats the same information deficiency at every iteration.

Algorithm 5 is adapted from Rizvi and Lin (Algorithm 3.3), and uses it in the context of the pursuer-evader notation of this thesis [5].

Convergence Interpretation and Nominal Verification: A small change in H or in the recovered gains does not by itself prove that the learned Q-function satisfies the Bellman equations. Hence, define the normalized policy evaluation residual as

$$\varepsilon^i_{B,\mathrm{PI}} = \frac{\left\| \Psi^i \widehat{\theta}^i - \ell \right\|_2}{1 + \|\ell\|_2}. \tag{5.23}$$

This quantity tests consistency with the sampled fixed-policy Bellman relation as it should be interpreted together with the numerical rank and condition number of Ψ^i. A small residual from a rank-deficient regression may reflect nonuniqueness rather than accurate identification, while a full-rank but ill-conditioned regression may fit the batch closely and still yield unstable coefficients under small perturbations.

The reconstructed matrix should also satisfy a symmetry check like

$$\varepsilon^i_{\mathrm{sym}} = \frac{\left\| \hat{H}^i - (\hat{H}^i)^\top \right\|_F}{1 + \left\| \hat{H}^i \right\|_F}. \tag{5.24}$$

When H is formulated directly from independent symmetric coefficients, this quantity is near zero by construction. Reporting it remains useful because it confirms that feature ordering, coefficient placement, and block extraction are mutually consistent.

The final input blocks must satisfy both (5.12) and (5.13), and the joint block $\mathcal{H}_u$ must be sufficiently well-conditioned for the gain solve to be meaningful. The final gain pair is then checked against the closed-loop stability requirement of the common benchmark. This external stability check does not make the learner model-based. Rather, it is a validation step that is performed after learning, just as a model-free estimate can be compared with a known analytical solution without using that solution during the regression.

When a model-based reference $(L_{\mathrm{ref}}, K_{\mathrm{ref}})$ is available, then relative gain errors may be reported as

$$\varepsilon_L = \frac{\|L - L_{\mathrm{ref}}\|_F}{1 + \|L_{\mathrm{ref}}\|_F}, \quad \varepsilon_K = \frac{\|K - K_{\mathrm{ref}}\|_F}{1 + \|K_{\mathrm{ref}}\|_F}. \tag{5.25}$$

These errors test nominal recovery of the same zero-sum solution as they do not enter the learning update and do not establish that Q-learning is better than the Riccati method. Agreement is the expected outcome when both methods are able to solve the same linear-quadratic game and the sampled regression recovers the exact quadratic structure.

5.4. Disturbance-Aware Extension of Q-Learning Policy Iteration

Reasoning for Calibration and Action Labeling Preceding Learning: The nominal policy iteration algorithm assumes that the action stored in each transition is the action that generated the successor state. Persistent command offsets separate the requested and realized inputs:

$$u_{P,k}^{\text{act}} = u_{P,k}^{\text{cmd}} + b_P, \quad u_{E,k}^{\text{act}} = u_{E,k}^{\text{cmd}} + b_E. \tag{5.26}$$

If the dataset stores the commanded values while the plant evolves under the realized values, then both the feature vector and the stage cost are mislabeled. The error is systematic since the same offset appears in every sample, and it cannot be removed by adding more samples of the same incorrectly labeled interaction.

Thus, the model-free extension begins with command-channel calibration. During a calibration interval, the commanded and realized inputs are measured separately as

$$u_{P,k}^{\text{meas}} = u_{P,k}^{\text{cmd}} + b_P + \eta_{P,k}, \quad u_{E,k}^{\text{meas}} = u_{E,k}^{\text{cmd}} + b_E + \eta_{E,k}, \tag{5.27}$$

where $\eta_{P,k}$ and $\eta_{E,k}$ denote input-measurement noise. The sample mean estimates are given by the equations

$$\hat{b}_P = \frac{1}{N_{\text{cal}}} \sum_{k=1}^{N_{\text{cal}}} \left(u_{P,k}^{\text{meas}} - u_{P,k}^{\text{cmd}} \right), \tag{5.28}$$

$$\hat{b}_E = \frac{1}{N_{\text{cal}}} \sum_{k=1}^{N_{\text{cal}}} \left(u_{E,k}^{\text{meas}} - u_{E,k}^{\text{cmd}} \right). \tag{5.29}$$

No plant-transition matrix appears in these equations. The estimator uses the measured difference at each command channel and thus remains consistent with the model-free information boundary of the Bellman update.

If the measurement noises are zero mean, independent across samples, and have covariances Σ_P and Σ_E, then it follows that

$$\text{Cov}(\hat{b}_P) = \frac{\Sigma_P}{N_{\text{cal}}}, \quad \text{Cov}(\hat{b}_E) = \frac{\Sigma_E}{N_{\text{cal}}}. \tag{5.30}$$

Unlike the model-based residual estimator, the direct command-mismatch estimator does not require the two plant input directions to be linearly independent. Rather, it requires separate measurements of the two realized inputs and this is a fundamental trade-off. The previous chapter used model knowledge hence model-based and transition geometry, while this chapter uses command-interface measurement. If separate realized inputs are unavailable, then the simple estimator in (5.28) and (5.29) cannot distinguish the two offsets without additional observer or model structure.

Disturbance-Consistent Exploration Data: Let $v_{P,k}$ and $v_{E,k}$ denote desired realized exploration signals. To reduce the known portion of the channel mismatch during data collection, send

$$u_{P,k}^{\text{cmd}} = v_{P,k} - \hat{b}_P, \quad u_{E,k}^{\text{cmd}} = v_{E,k} - \hat{b}_E. \tag{5.31}$$

The realized inputs become

$$u_{P,k}^{\text{act}} = v_{P,k} + \tilde{b}_P, \quad u_{E,k}^{\text{act}} = v_{E,k} + \tilde{b}_E. \tag{5.32}$$

where

$$\tilde{b}_P = b_P - \hat{b}_P, \quad \tilde{b}_E = b_E - \hat{b}_E. \tag{5.33}$$

The measured realized inputs in (5.32), not the desired signals or the sent commands, are stored in z_k and are used in the stage cost. As a result, this choice makes the Bellman sample consistent even when calibration is imperfect.

Compensation during exploration and correct action labeling serve different purposes. Subtracting $\hat{b}_P$ and $\hat{b}_E$ keeps the realized exploration closer to the intended excitation design. Then, recording $u_{P,k}^{\text{act}}$ and $u_{E,k}^{\text{act}}$ ensures that the sample remains mathematically valid. The second requirement is essential as a dataset can remain Bellman consistent without perfect compensation if the realized actions are measured honestly. Conversely, perfect intended exploration does not help if the stored action differs from the input that generated the transition.

The realized feature matrix still must satisfy both the rank and conditioning requirements. A residual constant bias may shift the distribution of the samples and can either improve or degrade excitation depending on the geometry of the dataset. It should not be treated as a substitution for deliberate

variation in both player channels. The exploration signals must independently excite the pursuer and evader directions, and the resulting states must cover the region in which the learned quadratic policy will be interpreted.

Measurement noise in the realized inputs introduces an additional limitation since the measured actions appear in the regressors, ordinary least-squares becomes an errors-in-variables problem when that noise is appreciable. This thesis assumes that sufficiently accurate input measurement for the structured least-squares formulation. Instrumental variable, total least-squares, or joint state-input estimation methods are possible extensions; however, adding these extensions would change the learning problem beyond the controlled command-offset study that is developed here.

Affine Deployment and Residual Closed-Loop Behavior: After the nominal Q-learning policy iteration converges, then the learned feedback gains are deployed by

$$u_{P,k}^{\text{cmd}} = Lx_k - \hat{b}_P, \quad u_{E,k}^{\text{cmd}} = Kx_k - \hat{b}_E. \tag{5.34}$$

The realized inputs are given by

$$u_{P,k}^{\text{act}} = Lx_k + \tilde{b}_P, \quad u_{E,k}^{\text{act}} = Kx_k + \tilde{b}_E. \tag{5.35}$$

For theoretical interpretation, the resulting plant dynamics are given by

$$x_{k+1} = A_{\text{cl}}x_k + B_P\tilde{b}_P + B_E\tilde{b}_E, \quad A_{\text{cl}} = A + B_PL + B_EK, \tag{5.36}$$

and equation (5.36) is not used by the learner. It is the common physical interpretation of the policy after learning and demonstrates that the affine compensation law replaces the original offsets with their estimation errors.

Let x_k^{nom} and x_k^{comp} denote the nominal and estimated-compensation trajectories from the same initial condition as their deviation satisfies the expression

$$d_{k+1} = A_{\text{cl}}d_k + B_P\tilde{b}_P + B_E\tilde{b}_E, \quad d_0 = 0, \tag{5.37}$$

with the finite-step solution of

$$d_k = \sum_{j=0}^{k-1} A_{\text{cl}}^j \left(B_P \tilde{b}_P + B_E \tilde{b}_E \right). \tag{5.38}$$

When A_{cl} is Schur stable, then

$$d_\infty = (I - A_{\text{cl}})^{-1} \left(B_P \tilde{b}_P + B_E \tilde{b}_E \right). \tag{5.39}$$

Recall that the same residual bias relationship appeared in the previous chapter since it is a property of the executed plant, not of the method that is used to obtain L and K.

Exact compensation sets $\hat{b}_P = b_P$ and $\hat{b}_E = b_E$. Then $\tilde{b}_P = \tilde{b}_E = 0$ and (5.36) reduces to the nominal learned-policy dynamics. Thus, exact compensation verifies command bookkeeping independently of Q-function accuracy. A learned policy could be suboptimal and still pass the exact compensation test. Conversely, a highly accurate nominal policy could fail the test if command and realized inputs were handled inconsistently.

The Disturbance-Aware Extension of the Q-Learning Policy Iteration algorithm is provided in Algorithm 6. More specifically, Algorithm 5 defines the nominal Q-learning policy iteration, while calibration, disturbance-consistent data construction, affine execution, and verification constitute the extension of this thesis. The disturbance-aware extension succeeds nominal policy iteration since the Bellman regression requires a valid transition dataset. Removing calibration and disturbance-aware execution phases recovers the nominal Q-learning policy iteration algorithm. These phases do not provide the analytical plant matrices to the regression nor modify the saddle policy recovery equation. Rather, they ensure that the dataset and deployment law remain consistent with persistent command mismatch.

Algorithm 6 Disturbance-Aware Extension of Q-Learning Policy Iteration

Phase I: Calibrate the command channels.
Apply known calibration commands and measure the realized pursuer and evader inputs.
Estimate $\widehat{b}_P$ and $\widehat{b}_E$ from the sample means of command-versus-measurement differences.
Record the calibration spread and state the assumption that the two realized inputs are measured separately.
Phase II: Construct a disturbance-consistent transition batch.
Choose desired realized exploration sequences $v_{P,k}$ and $v_{E,k}$ with sufficient joint excitation.
Send $v_{P,k} - \widehat{b}_P$ and $v_{E,k} - \widehat{b}_E$ to the biased command channels.
Measure and store x_k, $u_{P,k}^{\text{act}}$, $u_{E,k}^{\text{act}}$, x_{k+1}, and the realized stage cost.
Construct the quadratic features from the measured realized inputs and verify feature rank and conditioning.
Phase III: Execute the nominal Q-learning policy iteration.
Apply Algorithm 5.1 to the fixed disturbance-consistent batch until the Q-function and gain sequences satisfy the selected convergence and validity checks.
Phase IV: Apply and verify the matched execution cases.
Evaluate nominal, uncompensated, estimated-compensation, and exact-compensation operation with the learned gains.
Report the Bellman diagnostics, feature diagnostics, offset errors, residual affine forcing, analytical recurrence mismatch, and exact-compensation trajectory error.

5.5. Nominal Q-Learning Value Iteration

Bootstrapped Bellman Target: Q-learning value iteration uses the same measured state-action features, but advances the action-value approximation through a sampled optimality backup. Let θ^i and H^i denote the current Q-function estimate, and let (L^i, K^i) be the saddle gains that are recovered from H^i. For each transition, form the successor policy vector

$$z_{k+1}^{(i)} = \begin{bmatrix} x_{k+1} \\ L^i x_{k+1} \\ K^i x_{k+1} \end{bmatrix}. \tag{5.40}$$

The current bootstrapped target is

$$y_k^i = \ell_k + \phi\left(z_{k+1}^{(i)}\right)^\top \theta^i. \tag{5.41}$$

Let Φ denote the fixed measured feature matrix whose kth row is $\phi(z_k)^\top$. Then, the next parameter estimate satisfies

$$\Phi\theta^{i+1} \approx y^i, \tag{5.42}$$

and is obtained from

$$\widehat{\theta}^{i+1} = \arg\min_{\theta} \|\Phi\theta - y^i\|_2^2. \tag{5.43}$$

After reconstructing a symmetric H^{i+1}, the next gain pair is recovered from its input-related blocks.

The distinction from policy iteration is exact and limited. Policy iteration places the successor-policy feature on the left-side and solves for the Q-function of a fixed policy pair given by

$$\left[\phi(z_k) - \phi\left(z_{k+1}^{(i)}\right)\right]^\top \theta^i = \ell_k. \tag{5.44}$$

Value iteration places the current successor Q-value in the target and advances the approximation expressed by

$$\phi(z_k)^\top \theta^{i+1} = \ell_k + \phi\left(z_{k+1}^{(i)}\right)^\top \theta^i. \tag{5.45}$$

The first equation evaluates one policy after improvement, then the second performs one sampled optimality backup using the current value approximation. This is the model-free counterpart of the distinction between the Lyapunov evaluate-improve cycle and the Riccati mapping in the previous chapter.

Definition of the Iterates and Fixed Data Matrix: The value iteration matrix H^i should not automatically be interpreted as the fully evaluated infinite-horizon Q-function of (L^i, K^i), as its meaning is tied to repeated application of the bootstrapped Bellman operator. The gains that are recovered from H^i define the successor saddle actions that are used in the next target, however the matrix itself is an intermediate value approximation rather than the solution of a separate fixed-policy equation at every iteration.

The feature matrix Φ remains fixed since it is constructed from the measured current state-action samples as only the target y^i changes. As a result, this structure can reduce repeated matrix construction relative to policy iteration, but it does not remove the necessity for sufficient excitation. Unique least-squares recovery requires

$$\operatorname{rank}(\Phi) = n_H, \tag{5.46}$$

Algorithm 7 Nominal Model-Free Q-Learning Value Iteration

1: Collect an off-policy transition batch with sufficient joint state–action excitation.
2: Construct the fixed feature matrix Φ from the measured vectors z_k and form the stage-cost vector ℓ.
3: Select an initial symmetric H^0 or parameter vector θ^0, recover the corresponding gain pair (L^0, K^0), and set $i = 0$.
4: Form $z_{k+1}^{(i)} = [x_{k+1}^\top, (L^i x_{k+1})^\top, (K^i x_{k+1})^\top]^\top$ for every transition.
5: Construct the bootstrapped targets $y_k^i = \ell_k + \phi(z_{k+1}^{(i)})^\top \theta^i$.
6: Solve $\Phi\theta^{i+1} \approx y^i$ in the least-squares sense and reconstruct a symmetric H^{i+1}.
7: Recover (L^{i+1}, K^{i+1}) from the input-related blocks of H^{i+1}.
8: Verify feature rank, regression conditioning, Bellman residual, saddle curvature, and the changes in H, L, and K.
9: Increment i and repeat the sampled value backup until the selected convergence conditions are satisfied.

and reliable recovery requires acceptable conditioning. A fixed ill-conditioned Φ contaminates every value iteration update since the target relies on the current parameters; coefficient noise may also be propagated through the bootstrapping sequence.

The policy recovery block needs to remain well-posed at every iteration where gains are extracted. If H_{PP}^i loses positive definiteness, if the evader Schur complement loses negative definiteness, or if $\mathcal{H}_u^i$ becomes nearly singular, then the next saddle action is not reliably defined. These failures are not equivalent to a large Bellman residual since one concerns the fit of the value equation, while the other concerns the minimax structure implied by the fitted matrix.

Convergence, Initialization, and Well-Posedness: The normalized value iteration regression residual is given by

$$\varepsilon_{B,\mathrm{VI}}^i = \frac{\left\| \Phi\widehat{\theta}^{i+1} - y^i \right\|_2}{1 + \|y^i\|_2}. \tag{5.47}$$

This residual tests the current sampled backup, not the complete fixed-point equation by itself. A converged value-iteration sequence should exhibit small changes in H, L, and K, small sampled Bellman residuals, and valid final saddle blocks. When an analytical benchmark is available , gain and closed-loop comparisons provide additional evidence that the learned fixed point corresponds to the intended GARE solution.

Initialization influences the sequence since the first target relies on θ^0 and on the gains that are recovered from H^0. A symmetric matrix is not automatically a valid game Q-function, as its input block may have the wrong curvature, be singular, or produce an unstable gain pair. Thus, this thesis distinguishes between

Algorithm 8 Disturbance-Aware Extension of Q-Learning Value Iteration

Phase I: Calibrate the command channels.
Measure commanded and realized pursuer and evader inputs over the calibration interval.
Compute $\widehat{b}_P$ and $\widehat{b}_E$ from the direct command-mismatch sample means and characterize their measurement uncertainty.
Phase II: Construct the fixed disturbance-consistent dataset.
Convert the desired realized exploration signals into commands by subtracting the offset estimates.
Record the measured realized actions, states, successor states, and realized stage costs.
Construct the fixed feature matrix Φ and verify full numerical rank and acceptable conditioning.
Phase III: Execute the nominal Q-learning value iteration.
Apply Algorithm 5.3 until the Q-function and gain sequences satisfy the selected convergence and saddle-validity checks.
Phase IV: Apply and verify the matched execution cases.
Evaluate nominal, uncompensated, estimated-compensation, and exact-compensation operation with the learned value-iteration gains.
Report feature and Bellman diagnostics, offset errors, residual affine forcing, analytical recurrence mismatch, and exact-compensation trajectory error.

a mathematically writable initialization and a useful initialization whose saddle recovery is well-posed. The numerical study verifies the selected initialization for the benchmark but does not claim that every symmetric H^0 converges for every zero-sum plant.

Bootstrapping is able to magnify approximation error when the fitted operator is poorly conditioned or when the current policies drive successor states outside the data-supported region. This is a key reason why the chapter avoids a broad reinforcement learning superiority claim. In the exact linear-quadratic setting with sufficiently rich data, the quadratic representation is able to recover the nominal solution. Outside that setting, the stability and convergence rely on coverage, noise, approximation error, and the geometry of the minimax blocks.

Policy iteration and value iteration may converge to the same nominal saddle gains while displaying different transient histories. Policy iteration repeatedly solves a fixed-policy Bellman equation, so each regression has a policy evaluation interpretation. Value iteration repeatedly updates a bootstrapped target such that intermediate matrices represent successive approximations under the sample optimality operator. The next chapter will preserve these histories as opposed to only reporting the final gains.

5.6. Disturbance-Aware Extension of Q-Learning Value Iteration

The persistent command offsets are properties of the measurement and execution channels, not properties of the Bellman arrangement are used to estimate H. Thus, Q-learning value iteration uses the

same calibration equations, disturbance-consistent action labeling, as well as the affine deployment law developed for policy iteration and only the nominal learning phase changes.

Calibration first estimates the pursuer and evader offsets from direct command-versus-realization differences. The exploration batch then converts the desired realized excitation into commands and records the measured realized inputs in the feature vectors and stage costs. The nominal Q-learning value iteration algorithm applies (5.41)–(5.43) to the fixed batch. After convergence, deployment follows (5.34), and (5.36) describes the residual closed-loop dynamics.

Using the same calibration and data construction method for both model-free algorithms is necessary for a fair comparison. If policy iteration and value iteration used different action measurements, exploration definitions, or offset estimators, then differences in the learned gains could not be attributed solely to the Bellman update. By holding the disturbance-aware layer fixed, the next chapter will be able to evaluate how the two established Q-learning algorithms recover and verify the nominal saddle solution from the same physically consistent transition data.

Algorithm 7 defines the nominal Q-learning value iteration algorithm, while Algorithm 8 uses the calibration, data-construction, affine-execution, and verification phases that are similar to the ones that were used for policy iteration. The key difference between Algorithm 6 and Algorithm 8 lies in the nominal learning stage. Policy iteration evaluates a fixed policy through a feature-difference equation and then improves the saddle gains, while value iteration places the current successor Q-value in the target and advances the approximation directly. After convergence, both algorithms provide a learned gain pair to the same command calibration, affine compensation, and verification framework.

5.7. Similar Theoretical Properties and Limitations of the Model-Free Extensions

Invariance of the Nominal Bellman Core: The disturbance-aware layer does not insert the command offsets into the nominal Bellman equations. More specifically, the quadratic policy iteration regression remains (5.18), and the value iteration remains (5.42). Calibration changes how the actions are measured and how commands are transformed at the execution interface. However, it does not add the plant matrices to the learner or redefine the nominal Q-function target.

This invariance is crucial to the novelty claim as this thesis extends the conditions under which the original learning algorithms are supplied with data and deployed on biased command channels. It does not claim to derive a novel class of zero-sum Q-learning. Removing the calibration, compensated data

collection commands, and affine deployment recovers the nominal algorithms. Thus, retaining those operations makes the data and execution layers explicit without hiding the source algorithms within a larger black-box routine.

Nominal Equivalence Target and the Meaning of Model-Free Recovery: For the common linear-quadratic game, the model-based and model-free methods seek compatible nominal policies. The GARE value matrix P induces the exact action-value matrix provided in (5.6), and the saddle gains are recovered from that matrix equal the gains obtained from the GARE blocks. Therefore, if the quadratic feature regression is exact, the data cover all independent coefficients, and the learning iteration reaches the appropriate fixed point, the learned gains should agree with the model-based gains up to numerical error.

This expected agreement is a consistency result rather than a ranking; moreover, model-free recovery is valuable since the learner does not need the analytical transition matrices. It is not expected to outperform an exact Riccati computation on the same known nominal model simply because the word *learning* appears in the algorithm name. Its additional burdens are data collection, excitation, feature rank, regression conditioning, and action measurement. The comparison in the next chapter documents those differences as opposed to forcing a predetermined conclusion that one class is better overall.

In addition, a small gain error is also not the only criterion for success. The learned matrix must satisfy the sampled Bellman equations, the input blocks must define a valid saddle, and the recovered policies must generate a stable nominal closed-loop. Conversely, a small Bellman residual alone does not prove correct policy recovery if the feature matrix is ill-conditioned or if the gain block amplifies coefficient error. The diagnostic set is intentionally redundant since each quantity addresses a different failure mode.

Data Rank, Conditioning, and Coverage: The feature rank condition is the model-free counterpart of algebraic well-posedness in the GARE methods. The learner attempts to identify n_H independent coefficients, so the sampled equations need to contain at least that many independent quadratic directions. The requirement applies to the realized data, not to the intended exploration commands. Saturation, command offsets, or correlated player signals can reduce the effective rank even when the nominal input design appears rich.

Conditioning describes the sensitivity of the identified coefficients to perturbation. Let $\sigma_{\max}$ and $\sigma_{\min}$ denote the largest and smallest relevant singular values of a full-rank regression matrix. Then,

$$\kappa_2(\Phi) = \frac{\sigma_{\max}(\Phi)}{\sigma_{\min}(\Phi)} \tag{5.48}$$

provides a standard measure of sensitivity. A large condition number means that some quadratic directions are only weakly distinguished by the data. Hence, increasing the number of nearly redundant samples may reduce random residual variance without repairing this geometric flaw.

In addition, coverage also has a physical interpretation. The learned quadratic form is intended to represent the game over the region visited during training and evaluation. A full-rank batch constructed from extreme states may be algebraically identifiable, but irrelevant to the local operating region. A well-conditioned batch near one point may still fail to support a policy that drives the state through a broader domain. This thesis uses a controlled linear benchmark in which the exact quadratic structure is global, but the same diagnostic logic is retained since later nonlinear extensions would make the coverage explicitly local.

Stability, Offset-Free Regulation, and Exact Compensation: Once the learned gains are fixed, then the command-offset dynamics are the same as those developed in Chapters 3 and 4. If A_{cl} is said to be Schur stable, then a constant uncompensated offset produces a bounded but generally nonzero equilibrium,

$$x_{\infty}^{\mathrm{bias}} = (I - A_{\mathrm{cl}})^{-1}(B_P b_P + B_E b_E), \tag{5.49}$$

and estimated compensation gives

$$x_{\infty}^{\mathrm{comp}} = (I - A_{\mathrm{cl}})^{-1}(B_P \widetilde{b}_P + B_E \widetilde{b}_E). \tag{5.50}$$

Compensation changes the affine forcing, but does not move the eigenvalues of the nominal learned closed-loop matrix. Stability and offset-free regulation thus remain distinct claims.

Exact compensation provides a precise implementation test. If the true offsets are subtracted and the nominal and exact cases use the same learned gains and initial condition, then

$$x_k^{\mathrm{exact}} = x_k^{\mathrm{nom}} \tag{5.51}$$

for every k in exact arithmetic. Any remaining discrepancy should be at floating-point level; moreover, passing this test does not establish that the learned policy is the optimal saddle policy. Instead, it establishes that the command transformation and realized-input simulation are internally consistent.

Estimated compensation provides a graded outcome that is governed by $\tilde{b}_P$ and $\tilde{b}_E$. The effect of those residuals relies on the player input directions and on the closed-loop sensitivity. Thus, the next chapter

will report both offset-estimation error and state-deviation recovery. A direct estimator can have a small scalar error while the resulting state displacement remains visible if $(I - A_{\mathrm{cl}})^{-1}$ strongly amplifies the residual direction.

Limitations of the Present Model-Free Claim: This chapter assumes a linear time-invariant plant, full-state measurement, a quadratic Q-function that exactly matches the nominal game, and separately measured realized inputs. It does not claim general convergence for arbitrary nonlinear pursuit-evasion systems, arbitrary function approximators, time-varying faults, or unmeasured action channels. In addition, it also does not claim that a finite least-squares residual establishes robustness outside the data-supported region.

The command offsets are constant over the calibration and evaluation windows. Slowly varying offsets may be approximated locally, however online drift would require recursive estimation or adaptive compensation. Abrupt faults would require detection and possibly a different safety response. Similarly, unmatched disturbances that do not enter either through B_P or B_E cannot generally be canceled by subtracting a command bias.

The present algorithms are structured quadratic Q-learning methods as opposed to deep reinforcement learning architectures. This restriction is deliberate, as it keeps the learned object directly comparable with the GARE solution and allows feature rank, Bellman residual, saddle curvature, and gain recovery to be examined explicitly. Neural approximation, self-play, nonlinear policy classes, and safety-constrained exploration are potential future research directions.

5.8. Relationship between the Nominal and Disturbance-Aware Algorithms

Table VII separates the published model-free algorithms from the thesis addition. The distinction is necessary since the phrase *disturbance-aware Q-learning algorithms* could otherwise suggest that the Bellman equations themselves have been replaced and this is not the case.

The table establishes the novelty boundaries of this chapter. Algorithm 6 and Algorithm 8 do not replace the original Q-learning algorithms as their nominal Bellman equations and gain recovery updates remain explicit and are recovered by removing the calibration, disturbance-consistent data construction, and affine execution phases. The contribution is the connection between sampled zero-sum policy recovery and persistent command mismatch, together with verification conditions that separate learning error, data error, estimator error, and command-bookkeeping error.

The relationship with the prior chapter is now more direct. The model-based methods use transition matrices both to compute the nominal solution and to estimate command offsets from state residuals. The model-free methods estimate the nominal Q-function from measured transitions and estimate the offsets from direct command mismatch. Both classes use the sam e game, the same affine execution model, and the same four evaluation cases; moreover, their difference is thus an information-structure difference rather than a change in the physical problem.

5.9. Summary and Transition

The objective of this chapter developed the two model-based algorithms that are used within this thesis. Quadratic Q-learning policy iteration constructs a fixed-policy Bellman regression from measured state-action transitions, estimates the associated symmetric action-value matrix, and improves the pursuer and evader policies through a coupled saddle solve. Quadratic Q-learning value iteration uses the same measured feature representation but places the current successor Q-value in a bootstrapped target and advances the action-value approximation direct. Both methods seek the same nominal saddle policies, however their intermediate matrices and convergence interpretations remain distinct.

Then, the chapter attached the common persistent command-offset framework to each algorithm. Since the Bellman sample must be labeled by the input that generated the transition, the model-free extension begins with direct command-channel calibration and disturbance-consistent data construction. The offset estimates are obtained without supplying the plant matrices to the learner. Desired realized exploration signals are converted into commands by subtracting the estimates, while the measured realized inputs are stored in the quadratic features and stage cost.

After learning, affine compensation subtracts the estimated offsets from the learned feedback commands and leaves only the residual bias errors in the realized closed-loop. The resulting deviation equations are the same as those obtained for the model-based methods once the gains are fixed. Exact compensation reproduces the nominal learned-policy dynamics and thus serves as an implementation check. Estimated compensation produces a residual affine forcing whose effect relies on the estimation errors, the player input directions, and the closed-loop sensitivity.

In addition, there are no numerical results as the next chapter implements the equations developed in Chapters 4 and 5 under the same benchmark, reports the Riccati and Bellman convergence diagnostics, verifies feature rank and conditioning, evaluates the two offset estimators, and compares nominal, un-

TABLE VII: Relationship Between the Nominal Model-Free Algorithm and the Disturbance-Aware Algorithm

Component	**Nominal Model-Free Algorithm**	**Disturbance-Aware Thesis Algorithm**
Nominal update	Estimates a quadratic Q-function from sampled Bellman equations and recovers the saddle gains from its input blocks.	Preserves the same policy-iteration or value-iteration Bellman core and the same coupled gain-recovery equation.
Plant information	Does not supply A, B_P, or B_E to the learning regression.	Retains the same model-free regressor boundary; calibration uses direct command-interface measurement rather than a transition model.
Action assumption	Treats the action stored in a transition as the action that generated the successor state and stage cost.	Separates policy, commanded, and realized inputs and stores the measured realized action in every Bellman sample.
Offset estimation	Contains no persistent command-offset calibration.	Estimates each player offset from direct command-versus-realization differences over a calibration interval.
Exploration data	Applies exploratory actions and records the resulting transition.	Converts desired realized exploration into compensated commands and records the realized actions that actually produced the transition.
Policy deployment	Applies Lx_k and Kx_k directly to ideal command channels.	Applies $Lx_k - \widehat{b}_P$ and $Kx_k - \widehat{b}_E$ to the biased command channels.
Execution cases	Evaluates nominal learned-policy behavior.	Evaluates nominal, uncompensated, estimated-compensation, and exact-compensation cases under matched conditions.
Verification	Focuses on Q-function convergence, Bellman consistency, and learned saddle policies.	Retains nominal checks and adds calibration spread, action-label consistency, residual-affine, analytical-recurrence, and exact-trajectory checks.
Claim	Recovers a model-free zero-sum saddle solution for the stated quadratic game under the required data conditions.	Extends the verified nominal learner with a measured execution-side calibration and affine compensation layer.

compensated, estimated compensation, and exact compensation behavior across all four algorithms. That unified numerical chapter provides the evidence needed to determine whether the model-free algorithms recover the nominal saddle policies and whether the common affine extension restores the intended command-channel behavior.

6. MATLAB Validation and Comparative Evaluation

6.1. Introduction

Chapters 3 through 5 established the mathematical and algorithmic foundation of this thesis before numerical evidence could be introduced. Chapter 3 defined the common discrete-time zero-sum game, the persistent command-channel offset model, the affine compensation law, and the numerical conditions that are required for a computational solution to be interpreted as a valid saddle controller. Then, Chapter 4 developed the extended model-based Game Algebraic Riccati Equation (GARE) policy iteration and value iteration algorithms, while Chapter 5 developed the corresponding extended model-free quadratic Q-learning policy iteration and value iteration algorithms. The objective of this chapter is to implement these four algorithms in MATLAB under one fixed protocol and to determine whether the disturbance-aware extension behaves as predicted.

This chapter does not aim to introduce another control problem or a fifth algorithm. The plant, quadratic objective, minimizing and maximizing channels, command offsets, initial condition, calibration lengths, and execution horizon remain fixed. The numerical distinction among the four algorithms is limited to the nominal computation that gives the saddle gains and to the information structure that is used by the offset estimator. The extended GARE algorithms use the known plant matrices in their nominal updates and estimate the offsets from known-model transition residuals. The Q-learning algorithms estimate the quadratic action-value matrix from sampled transitions without inserting the analytical plant matrices into the Bellman regression, and their offset estimator uses direct differences between commanded and measured realized inputs.

Making this separation is crucial to interpreting the results; moreover, convergence of an iterative matrix sequence does not solely establish closed-loop stability, saddle-point validity, accurate offset estimation, or successful compensation. Conversely, a compensated trajectory that appears close to nominal does not prove that the underlying nominal game solution is correct. For that reason, the numerical evidence is organized into three layers. The first layer evaluates algorithm convergence and the defining GARE or Bellman relationships. The second layer evaluates the nominal game solution through stability, curvature, rank, conditioning, and reference-agreement checks. The third layer evaluates execution under persistent offsets through estimator statistics, state-deviation recovery, command-channel bookkeeping, analytical affine recursion checks, and finite-window signed cost.

In addition, this chapter also preserves the novelty boundary that was mentioned prior in the thesis. Policy and Value Iteration, GARE Theory, and quadratic Q-learning are established methods within the broader literature. The contribution that is evaluated here is the matched disturbance-aware extended algorithms; more specifically, persistent offsets are estimated under the information structure appropriate to each method, the estimates are subtracted from the nominal commands, and the resulting behavior is compared with uncompensated and exact compensation cases. Therefore, the numerical study questions whether the extended algorithm can be applied to all four established algorithms without altering their nominal optimality equations and whether the resulting compensated execution recovers the nominal behavior to the level that is predicted by the estimator error.

6.2. Common Benchmark and MATLAB Protocol

Discrete-Time Benchmark: All four MATLAB programs utilize the same three-state discrete-time zero-sum plant, given by

$$x_{k+1} = A_d x_k + B_P u_{P,k} + B_E u_{E,k}, \tag{6.1}$$

where

$$A_d = \begin{bmatrix} 0.906488 & 0.0816012 & -0.0005 \\ 0.0741349 & 0.90121 & -0.000708383 \\ 0 & 0 & 0.132655 \end{bmatrix}, \tag{6.2}$$

$$B_P = \begin{bmatrix} -0.00150808 \\ -0.0096 \\ 0.867345 \end{bmatrix}, \quad B_E = \begin{bmatrix} 0.00951892 \\ 0.00038373 \\ 0 \end{bmatrix}. \tag{6.3}$$

Then, the auxiliary output matrix stored in these programs is

$$C = \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}. \tag{6.4}$$

The numerical saddle solution is calculated from the full state, so the output matrix does not alter the feedback calculation that is reported in this chapter. More specifically, it remains part of the benchmark

record since it identifies the reduced engagement output that is associated with original source code from Rizvi and Lin [5].

The infinite-horizon zero-sum stage cost is expressed by

$$\ell_k = x_k^\top Q x_k + u_{P,k}^\top R_P u_{P,k} - u_{E,k}^\top R_E u_{E,k}, \tag{6.5}$$

where

$$Q = I_3, \quad R_P = 1, \quad R_E = \gamma^2, \quad \gamma = 1. \tag{6.6}$$

The pursuer is the minimizing player acting through B_P, and the evader is the maximizing player acting through B_E. Then, the MATLAB sign convention is

$$u_{P,k}^{\text{policy}} = L x_k, \quad u_{E,k}^{\text{policy}} = K x_k, \tag{6.7}$$

so any negative feedback direction is contained in the numerical values of L and K.

The common initial state is given by

$$x_0 = \begin{bmatrix} 10 & 5 & -2 \end{bmatrix}^\top. \tag{6.8}$$

Every execution case uses a horizon of $N_{\text{sim}} = 500$ transitions. Moreover, this horizon is long enough for the stable homogeneous response to decay and for the affine offset response to approach its small steady displacement. The same horizon is used when calculating the signed finite-window cost such that differences among the cases are not caused by unequal truncation.

Table VIII separates the common experimental conditions from the method specific iterations. This is crucial since the later comparison aims to isolate computational and information-structure differences. Changing the plant, offsets, noise, or initial conditions between methods would make the final trajectory comparison ambiguous.

Persistent Command Offsets and Four Execution Cases: The disturbance-aware extended algorithms separate the policy output, commanded input, and realized input. For the pursuer channel,

$$u_{P,k}^{\text{act}} = u_{P,k}^{\text{cmd}} + b_P, \tag{6.9}$$

TABLE VIII: Common MATLAB Benchmark and Validation Settings

Quantity	**Value**	**Numerical Role**
State dimension	$n = 3$	Shared reduced-order zero-sum engagement state.
Pursuer and evader input dimensions	$m_P = m_E = 1$	Scalar minimizing and maximizing channels.
State weight	$Q = I_3$	Makes the reported Q-weighted deviation equal to the Euclidean state-deviation norm.
Input weights	$R_P = R_E = 1$	Shared zero-sum input weighting.
True pursuer offset	$b_P = 0.040$	Persistent addition to the realized pursuer input.
True evader offset	$b_E = -0.010$	Persistent addition to the realized evader input.
Calibration length	$N_{\text{cal}} = 250$	Number of samples used by either offset estimator.
Calibration noise standard deviation	10^{-4}	Transition noise for the model-based estimator or realized-input measurement noise for the model-free estimator.
Monte Carlo trials	$N_{\text{MC}} = 1000$	Estimator-distribution validation.
Execution horizon	$N_{\text{sim}} = 500$	Matched trajectory and cost comparison.
Random seed	7	Reproducible calibration and Monte Carlo.
Nominal convergence tests	GARE PI: 10^{-7}, Other three: 10^{-10}	P-change rule for GARE PI; principal matrix and both gains otherwise.

and for the evader channel,

$$u_{E,k}^{\text{act}} = u_{E,k}^{\text{cmd}} + b_E. \tag{6.10}$$

The four matched execution cases differ only in how the command is formed.

In the nominal cases, the offsets are absent and the policy, command, and realized inputs coincide. In the uncompensated case, the policy input is issued directly as the command, after which the true offset is added by the command channel. In the estimated compensation cases, the estimated offset is subtracted before the command enters the channel,

$$u_{P,k}^{\text{cmd}} = Lx_k - \widehat{b}_P, \quad u_{E,k}^{\text{cmd}} = Kx_k - \widehat{b}_E. \tag{6.11}$$

The realized inputs are then

$$u_{P,k}^{\text{act}} = Lx_k + (b_P - \widehat{b}_P), \quad u_{E,k}^{\text{act}} = Kx_k + (b_E - \widehat{b}_E). \tag{6.12}$$

Finally, exact compensation substitutes the true offsets for the estimates. In exact arithmetic, this removes the affine forcing completely and reproduces the nominal homogeneous closed-loop. Thus, exact compensation is a verification case rather than a practical estimator.

The stage cost in every case is evaluated with the realized inputs that actually generate the transition. As a result, this prevents a biased input from being omitted from the performance calculation merely because the controller requested a different command. It also keeps the model-free dataset internally consistent; more specifically, the action stored in a Bellman sample is the realized action that is associated with the measured successor state and stage cost.

Calibration Protocols: The model-based methods estimate the two offsets from the known-model transition residual given by the expression

$$r_k = x_{k+1} - A_d x_k - B_P u_{P,k}^{\text{cmd}} - B_E u_{E,k}^{\text{cmd}}. \tag{6.13}$$

Under the persistent-offset model,

$$r_k = \begin{bmatrix} B_P & B_E \end{bmatrix} \begin{bmatrix} b_P \\ b_E \end{bmatrix} + w_k. \tag{6.14}$$

The estimate is formed from the mean residual by a least-squares solve. The matrix $\begin{bmatrix} B_P & B_E \end{bmatrix}$ has a rank of two in this benchmark, so the two offsets are separately identifiable. Its condition number is approximately 91.05, which anticipates a larger uncertainty in one recovered direction even though the calibration noise is isotropic in the state equation.

The model-free methods do not use A_d, B_P, or B_E in the Bellman regression or in the command-offset. Rather, the realized input is measured directly during calibration:

$$u_{P,k}^{\text{meas}} = u_{P,k}^{\text{cmd}} + b_P + \eta_{P,k}, \quad u_{E,k}^{\text{meas}} = u_{E,k}^{\text{cmd}} + b_E + \eta_{E,k}. \tag{6.15}$$

The sample-mean command mismatches are

$$\widehat{b}_P = \frac{1}{N_{\text{cal}}} \sum_{k=1}^{N_{\text{cal}}} \left(u_{P,k}^{\text{meas}} - u_{P,k}^{\text{cmd}} \right), \tag{6.16}$$

$$\widehat{b}_E = \frac{1}{N_{\text{cal}}} \sum_{k=1}^{N_{\text{cal}}} \left(u_{E,k}^{\text{meas}} - u_{E,k}^{\text{cmd}} \right). \tag{6.17}$$

This direct estimator has a different information requirement from the model-based residual estimator. It does not require that the plant input directions, but it does require separate measurements of the two realized command channels. As a result, differences in estimator accuracy later in the chapter must be interpreted as differences in available information and inverse geometry, not as an inherent superiority of the GARE or Q-learning policy computation.

Saved Outputs and Reproducibility: Each MATLAB program produces a method-specific results folder. The stored files include the complete numerical workspace, a plain-text report, summary and execution-case tables, convergence histories, Monte Carlo estimator samples, command histories, state trajectories, and publication figures in MATLAB FIG as well as PNG formats. The figures are generated from the same arrays written to the tabular files. This one-to-one relationship between reported numbers and plotted curves reduces the possibility that a manually copied result differs from the underlying run.

The MATLAB programs also perform internal checks before the result package is accepted. The model-based programs test the GARE and Lyapunov residuals, the game-matrix reciprocal condition number, the pursuer and evader curvature signs, and the closed-loop spectral radius. The model-free programs add feature rank, feature conditioning, Bellman residual, and model-based reference agreement checks. All four programs compare the simulated disturbed trajectory with the analytical affine-deviation recurrence and verify that exact compensation reproduces the nominal trajectory to floating-point precision.

6.3. Evaluation Metrics and Interpretation

Iteration and Equation Residuals: Convergence is evaluated from successive changes in the principal matrix and both feedback gains. GARE policy iteration terminates when the Frobenius norm of the P change is below 10^{-7}, with gain histories retained as diagnostics. GARE value iteration and both Q-learning methods require the principal matrix change and both gain changes to fall below 10^{-10}, hence preventing small matrix change from masking continued policy movement.

For the model-based methods, the final P is substituted into the defining GARE and the fixed-policy Lyapunov identity. The Frobenius norms of the residual matrices measure whether the computed value matrix satisfies both the optimality equation and the closed-loop evaluation equation. For the model-free methods, the least-squares regression residual measures how accurately the current sampled equation is solved, while the final fixed-policy or fixed-point Bellman residual measures whether the learned H is consistent with the converged policy and successor relationship. These residuals answer related, but distinct questions, and are thus reported separately.

Stability, Saddle Curvature, and Conditioning: The nominal closed-loop matrix is defined by

$$A_{\text{cl}} = A_d + B_P L + B_E K. \tag{6.18}$$

Schur stability requires that

$$\rho(A_{\text{cl}}) < 1. \tag{6.19}$$

A spectral radius below unity shows that the nominal homogeneous response decays; however, it does not mean that a persistent affine offset produces zero steady error. Under imperfect compensation residual offsets enter as a constant forcing term.

The game matrix that is used to recover the two policies must also remain numerically nonsingular, and the local saddle directions must have the correct curvature. The pursuer block needs to be positive, while the Schur complement evader block must be negative. These signs distinguish a minimizing pursuer direction and a maximizing evader direction from a stationary point with the wrong local geometry.

For the model-free methods, the symmetric 5×5 Q-function matrix contains 15 unique coefficients, such that the sampled quadratic feature matrix must have rank 15. Moreover, its condition number is stated since full rank alone does not guarantee numerical robustness. The analytical plant matrices are used outside the Bellman learner to generate the simulated transitions, track spectral radius as a simulation-side validation gate, and form an offline GARE reference after learning. They are not provided to the least-squares regression nor the coupled learned-gain equation.

Estimator Accuracy and Monte Carlo Spread: The primary calibration estimate is evaluated through its signed error, absolute error, and percentage error. The Monte Carlo study then estimates the distribution of the same estimator under repeated noise realizations. The individual estimate standard deviation describes

the variability of the one calibration experiment, whereas the standard error of the Monte Carlo mean describes uncertainty in the estimated distribution mean across the finite number of trials. These quantities should not be interchanged. ' The estimator distribution is relevant to the closed-loop result since the residual realized-input offsets are exactly

$$\widetilde{b}_P = b_P - \widehat{b}_P, \quad \widetilde{b}_E = b_E - \widehat{b}_E. \tag{6.20}$$

A smaller estimator error should produce a smaller affine forcing, however the state effect also relies on B_P, B_E, and the resolvent of the closed-loop matrix. Thus, estimator percentage error and trajectory deviation reduction are also reported.

State-Deviation and Cost Metrics: For each disturbed case, the state deviation from nominal is given by

$$d_k = x_k^{\text{case}} - x_k^{\text{nom}}. \tag{6.21}$$

The plotted metric is

$$\|d_k\|_Q = \sqrt{d_k^\top Q d_k}. \tag{6.22}$$

Since $Q = I_3$ in this benchmark, this is also the Euclidean state-deviation norm. The maximum reduction is achieved by estimated compensation is

$$\mathcal{R}_{\max} = 100 \left(1 - \frac{\max_k \|d_k^{\text{est}}\|_Q}{\max_k \|d_k^{\text{uncomp}}\|_Q} \right) \%. \tag{6.23}$$

The analytical deviation recurrence is expressed as

$$d_{k+1} = A_{\text{cl}} d_k + B_P \widetilde{b}_P + B_E \widetilde{b}_E. \tag{6.24}$$

The maximum difference between this analytical sequence and the deviation that is obtained by subtracting two independently simulated trajectories is reported as an implementation consistency check.

Finally, the finite-window sign cost is

$$J_{0:N_{\text{sim}}-1} = \sum_{k=0}^{N_{\text{sim}}-1} \left(x_k^\top Q x_k + (u_{P,k}^{\text{act}})^\top R_P u_{P,k}^{\text{act}} - (u_{E,k}^{\text{act}})^\top R_E u_{E,k}^{\text{act}} \right). \tag{6.25}$$

The cost is signed since the evader term enters with the maximizing sign. A small-signed cost difference from nominal is interpreted together with the trajectory deviation metric rather than treated as a standalone measure.

6.4. Model-Based GARE Policy Iteration Results

Nominal Convergence and Saddle Validity: The GARE policy iteration code converged in four outer iterations. Each outer step first evaluated the complete infinite-horizon value of the current admissible pair through a discrete Lyapunov equation and then updated the pursuer and evader gains through the coupled saddle solve. Figure 6.1 demonstrates that the value matrix and gain changes decreased by several orders of magnitude over these four steps. The rapid outer convergence is consistent with policy iteration, however it should not be interpreted as requiring only four scalar calculations as each step includes a complete fixed policy matrix evaluation.

TABLE IX: Nominal Numerical Validation for Model-Based GARE Policy Iteration

Diagnostic	**Observed Value**
Converged	Yes
Outer Iterations	4
Closed-Loop Spectral Radius	0.9819707273
GARE Residual	6.4048×10^{-15}
Normalized GARE Residual	2.2076×10^{-16}
Lyapunov Residual	4.3512×10^{-15}
Normalized Lyapunov Residual	1.4998×10^{-16}
Game-Matrix Reciprocal Condition Number	0.56579379
Minimum Pursuer Curvature	1.761840343
Maximum Evader Curvature	−0.998509990

The residuals in Table IX are close to floating-point precision. The spectral radius is below unity, the pursuer curvature is positive, and the reduced evader curvature is negative. These checks demonstrate that the four-step sequence did not merely stop changing; rather, it reached the stabilizing saddle solution of the stated game.

Known-Model Offset Estimation: The model-based calibration estimated the pursuer offset as $\widehat{b}_P = 0.0400013670$, and the evader offset as $\widehat{b}_E = -0.0102874376$. The pursuer percentage error was ap-

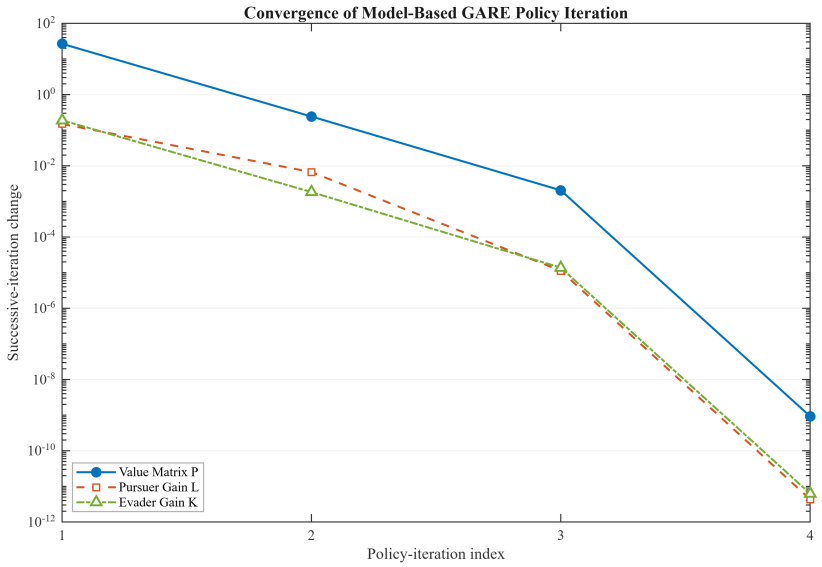

Fig. 6.1: Model-Based GARE Policy Iteration Convergence

proximately 0.00342%, while the evader percentage error was approximately 2.874%. The difference in accuracy is consistent with the geometry of the matrix $\begin{bmatrix} B_P & B_E \end{bmatrix}$. The calibration residual must separate two offsets through plant input directions whose condition number is approximately 91.05. Hence, the evader channel is recovered with a broader sampling distribution even though the same transition noise-level acts on the state equation.

Figure 6.2 shows the Monte Carlo distributions for the known model estimator. The pursuer estimates are tightly concentrated, whereas the evader estimates have a wider spread since the two offsets are inferred through the plant input directions. The primary estimate lies within the expected distribution in both channels, confirming the predicted direction-dependent uncertainty.

Trajectory Recovery and Analytical Verification: Without compensation, the persistent offsets produced a maximum Q-weighted state deviation of 4.12863×10^{-2}. Estimated compensation reduced the maximum deviation to 1.13499×10^{-4}, which corresponds to a reduction of approximately 99.7251%. Exact compensation reproduced the nominal trajectory to approximately 1.39×10^{-17}. The analytical affine recurrence agreed with the independently simulated deviation to approximately 2.69×10^{-15} in the uncompensated case as 4.68×10^{-15} in the estimated compensation case.

Figure 6.3 shows the physical interpretation of these values. The uncompensated deviation approaches a nonzero steady displacement since the stable homogeneous dynamics are driven by a constant affine term. Estimated compensation leaves a much smaller affine term, so the response approaches a correspondingly

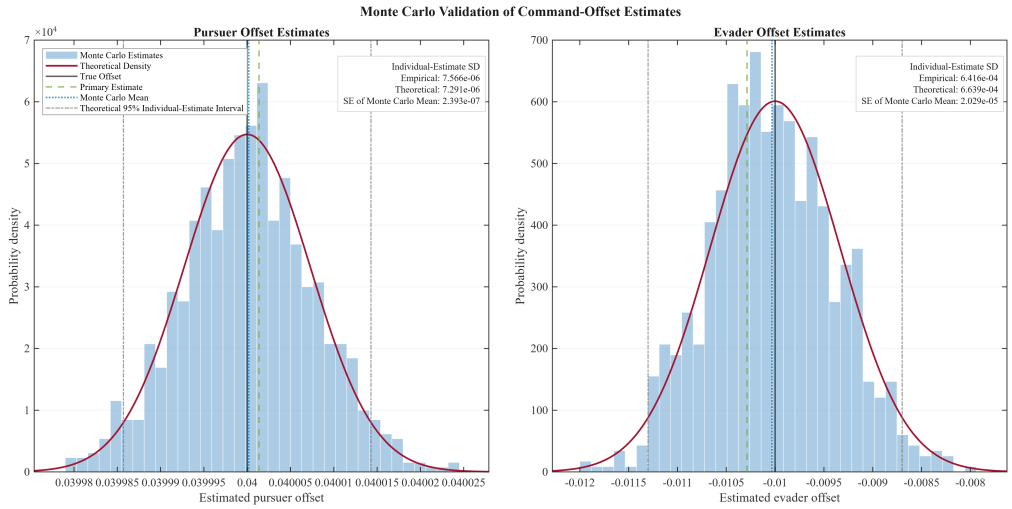

Fig. 6.2: Known-Model Offset Estimator for GARE Policy Iteration

smaller steady displacement. The result does not require a change to L or K; rather, it follows from translating the command by the estimated offsets.

The signed finite-window nominal cost was $J_{\text{nom}} = 3169.85881235989$. Without compensation, the cost increased to $J_{\text{uncomp}} = 3171.20561554776$, which is a difference of approximately 1.3468. Estimated compensation produced $J_{\text{est}} = 3169.85876338411$, which differs from the nominal by approximately -4.90×10^{-5}. Exact compensation reproduced the nominal cost to the reported precision. The small negative difference in the estimated case does not imply that imperfect compensation outperformed the saddle solution. Rather, it is a finite-window signed cost effect caused by the small residual offsets acting on both minimizing and maximizing input terms. The trajectory deviation and exact compensation checks provide the more direct recovery interpretation.

Command-Channel Behavior: Figure 6.4 separates the policy output, the compensated command, and the realized input during late-time portion of the simulation. In the pursuer channel, the command is shifted downward by the estimated positive offset before the true positive offset is added by the command channel. In the evader channel, the command is shifted in the opposite direction since the true offset is negative. In both cases, the realized estimated compensation input nearly overlaps the nominal input, while the uncompensated input retains a persistent displacement.

The small remaining separation is exactly the offset estimation residual. Taken together, Figures 6.1-6.4 establish the complete model-based policy iteration argument. More specifically, the nominal algorithm reaches a valid GARE saddle solution, the residual estimator recovers the offsets with the uncertainty

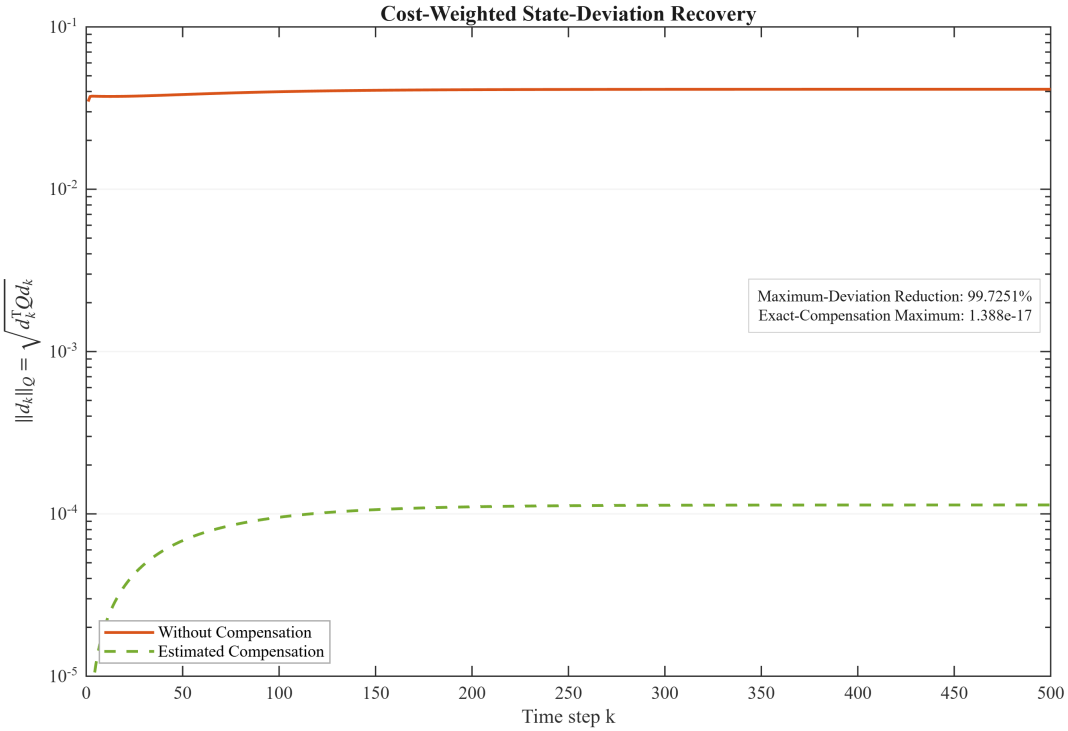

Fig. 6.3: GARE Policy Iteration State Deviation Recovery

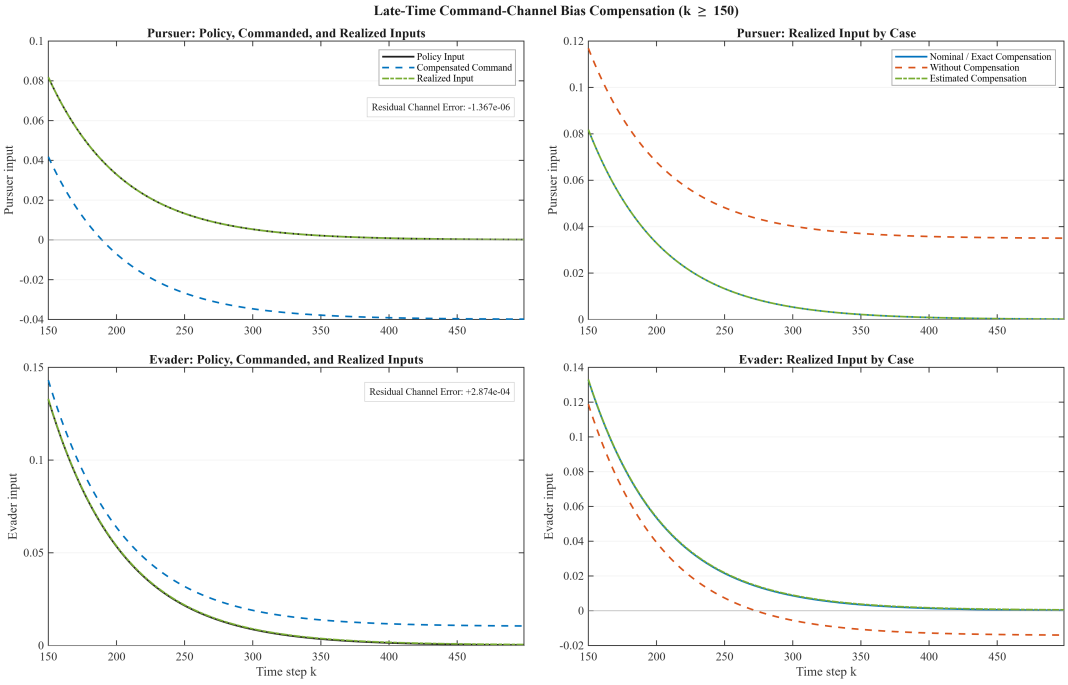

Fig. 6.4: GARE Policy Iteration Command Channel Compensation

predicted by the input geometry, and the affine correction restores the nominal execution behavior to the level permitted by the estimator error.

6.5. Model-Based GARE Value Iteration Results

Nominal Convergence and Saddle Validity: GARE value iteration approached the same nominal saddle solution through repeated applications of the Riccati optimality mapping; moreover, it converged in 633 value iteration steps. The longer sequence is not inconsistent with the four policy iteration updates since policy iteration solves a complete fixed-policy evaluation problem before each improvement, whereas value iteration advances the value approximation through one optimality mapping per step.

TABLE X: Nominal Numerical Validation for Model-Based GARE Value Iteration

Diagnostic	**Observed value**
Converged	Yes
Value-Iteration Steps	633
Closed-Loop Spectral Radius	0.9819707273
GARE Residual	9.4437×10^{-11}
Normalized GARE Residual	3.2551×10^{-12}
Lyapunov Residual	9.4440×10^{-11}
Normalized Lyapunov Residual	3.2552×10^{-12}
Game-Matrix Reciprocal Condition Number	0.56579379
Minimum Pursuer Curvature	1.761840343
Maximum Evader Curvature	−0.998509990

The residual magnitudes are consistent with the 10^{-10} stopping tolerance. The final spectral radius, conditioning, and curvature values agree with the policy iteration result to the displayed precision, confirming that the two model-based algorithms reach the same controller by different numerical paths.

Figure 6.5 demonstrates an approximately geometric decrease in the value matrix and gain changes over the 633 updates. The nearly straight trends on the logarithmic scale indicate regular contraction over most of the iteration. The gain changes remain below the value matrix change near convergence, and all three sequences approach their selected numerical floors without reversal or sustained oscillation.

Known-Model Offset Estimation: The value iteration implementation uses the same known-model residual structure as the policy iteration implementation. Figure 6.6 presents the value iteration Monte Carlo record rather than referring the reader back to a figure from another method. This keeps the result package aligned method by method and makes clear which output belongs to the GARE value iteration program.

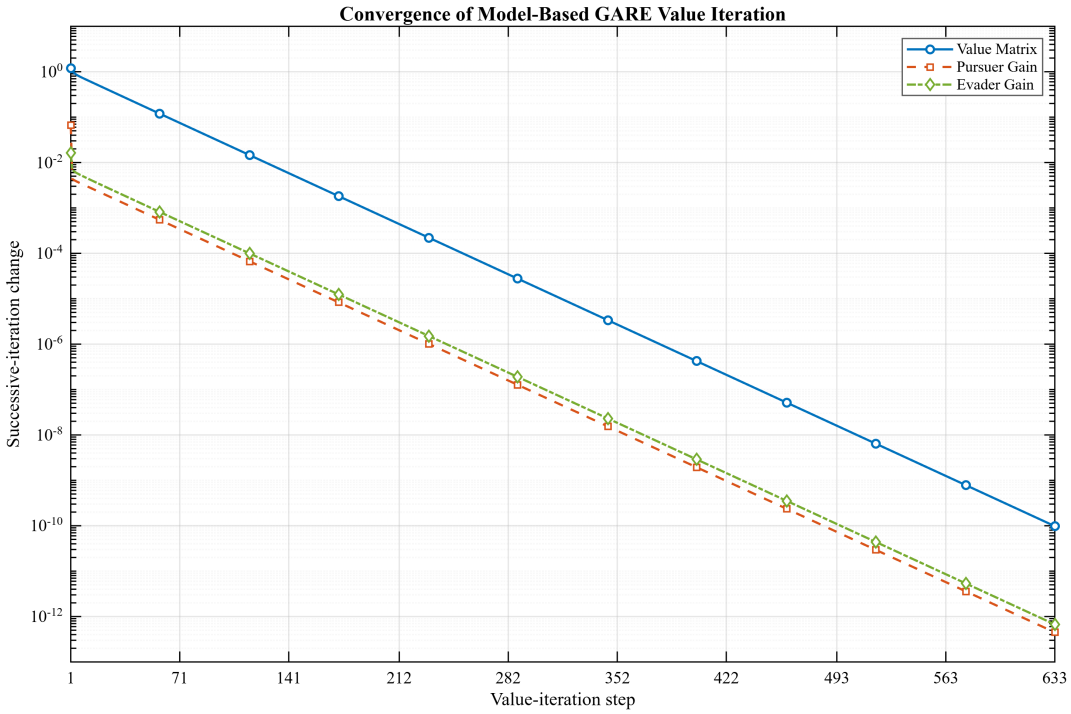

Fig. 6.5: Model-Based GARE Value Iteration Convergence

The pursuer distribution remains narrow, whereas the evader distribution is much wider. The theoretical density again follows the empirical histogram, and the primary estimate remains within the expected sampling range. Hence, the repeated asymmetry is attributable to the common input direction geometry rather than to the policy or value iteration update used to compute the nominal gains.

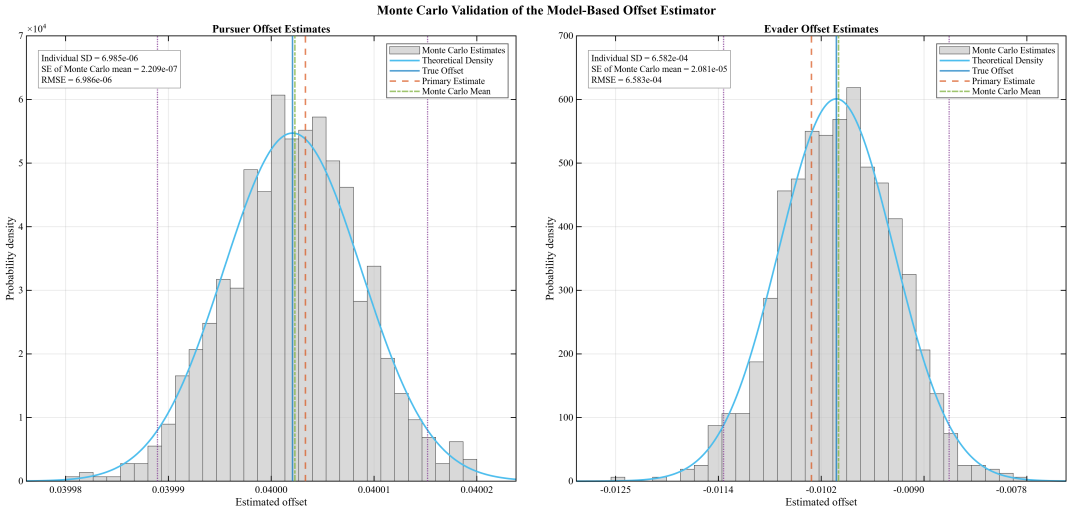

Fig. 6.6: Known-Model Offset Estimator Monte Carlo Validation for GARE Value Iteration

Trajectory Recovery and Cost: The uncompensated maximum deviation was 4.12863×10^{-2}, and estimated compensation reduced it to approximately 1.13499×10^{-4}, again yielding a 99.7251% reduction. Exact compensation differed from the nominal by approximately 2.00×10^{-15}. The uncompensated and estimated analytical mismatches were approximately 3.15×10^{-15} and 5.15×10^{-15}, respectively.

Figure 6.7 demonstrates that the value iteration controller produces the same disturbance recovery pattern as the policy iteration controller. The uncompensated curve settles near 4.13×10^{-2}, while the estimated compensation curve approaches a residual level near 1.13×10^{-4}. This agreement is expected since both algorithms recover the same gain pair and use the same form of known-model offset compensation.

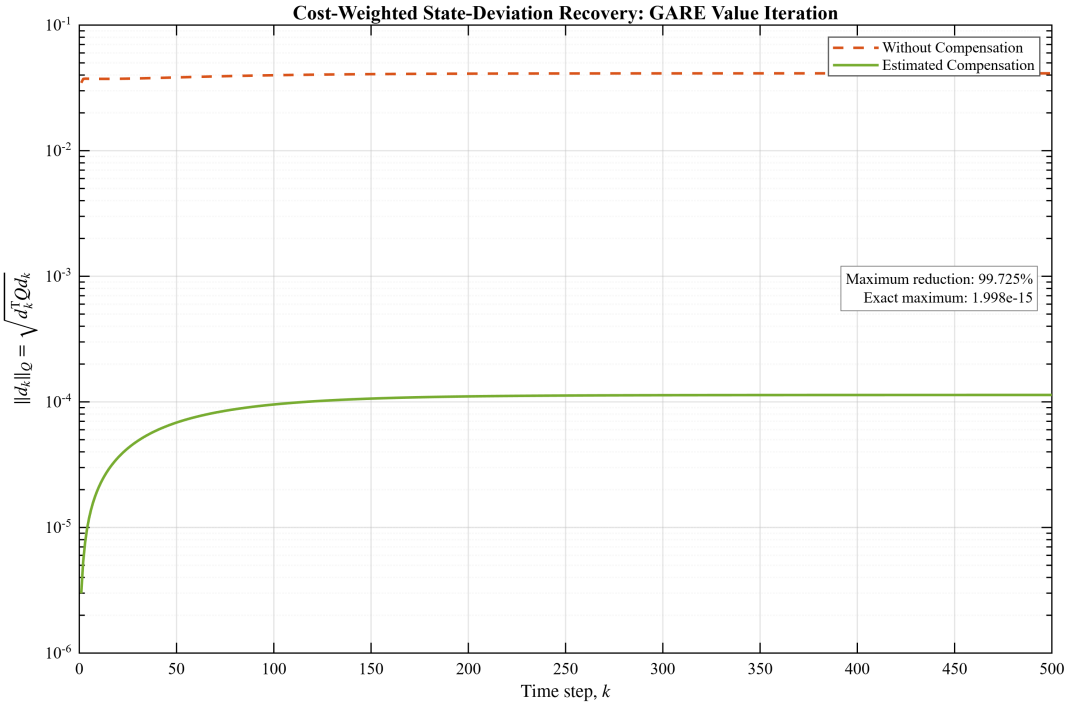

Fig. 6.7: GARE Value Iteration State Deviation Recovery

The signed finite-window costs were $J_{\text{nom}} = 3169.85881235989$, $J_{\text{uncomp}} = 3171.20561554657$, and $J_{\text{exact}} = 3169.85881235989$. Their agreement with the policy iteration costs follows from the shared final saddle solution.

Command Channel Behavior: Figure 6.8 demonstrates the value iteration command histories in the same four panel organization that was used for policy iteration. The left panels identify the amount by which the commands are translated before the offsets enter the channel. The right panels demon-

strate that estimated and exact compensation nearly reproduce the nominal realized inputs, whereas the uncompensated inputs retain their persistent displacements.

The close agreement between Figures 6.5-6.8 and the corresponding policy iteration results confirms that the disturbance-aware extension layer is not tied to the Lyapunov evaluation structure of policy iteration.

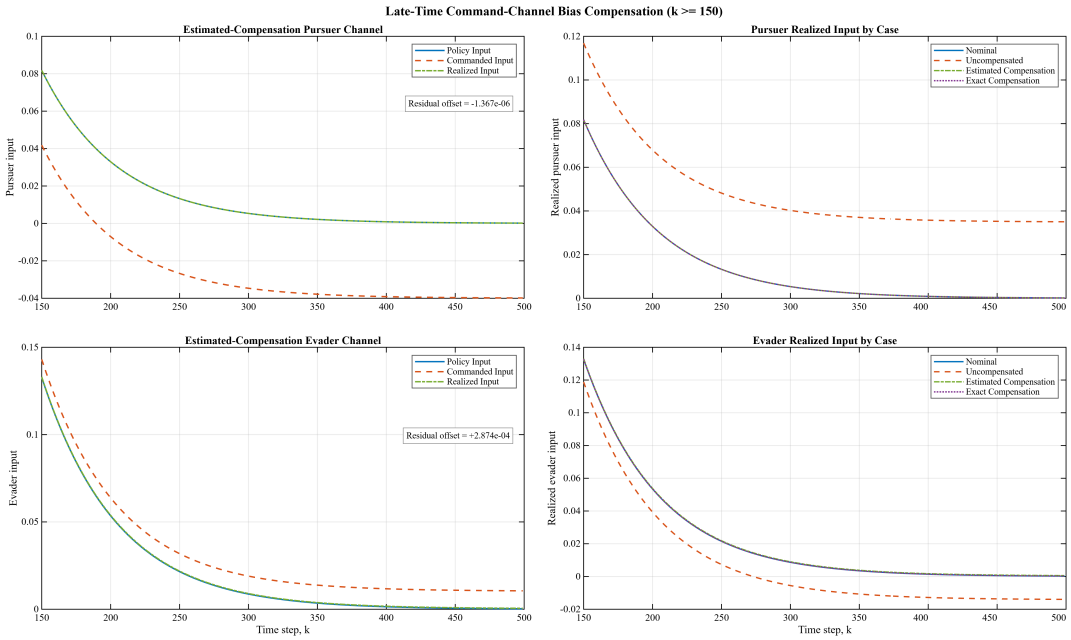

Fig. 6.8: GARE Value Iteration Command Channel Compensation

6.6. Model-Free Q-Learning Policy Iteration Results

Data Sufficiency and Nominal Policy Recovery: The Q-learning policy iteration algorithm used a fixed set of 500 off-policy transitions; moreover, the augmented state-action vector has dimension 5, so the symmetric Q-function matrix contains 15 unique coefficients. Both the raw and scaled feature matrices had a rank 15. The raw feature matrix condition number was approximately 412.79, the scaled condition number was approximately 308.87, and the final policy evaluation condition number was approximately 218.02.

The learned Q-function, gains, and value matrix agreed with the offline model-based reference to relative errors between approximately 10^{-14} and 10^{-12}. This is consistent with an exact quadratic representation and a sufficiently rich deterministic dataset.

TABLE XI: Nominal Numerical Validation for Model-Free Q-Learning Policy Iteration

Diagnostic	**Observed Value**
Converged	Yes
Outer Policy-Iteration Steps	5
Feature Rank	$15/15$
Scaled Feature Rank	$15/15$
Scaled Feature Condition Number	308.8705
Final Policy-Evaluation Condition Number	218.0190
Final Q-Function Change	9.5383×10^{-14}
Final Pursuer-Gain Change	2.3575×10^{-15}
Final Evader-Gain Change	2.1399×10^{-14}
Final Regression Residual	3.9898×10^{-15}
Fixed-Policy Bellman Residual	4.0166×10^{-15}
Closed-Loop Spectral Radius	0.9819707273
Game-Matrix Reciprocal Condition Number	0.56579379
Minimum Pursuer Curvature	1.761840343
Maximum Evader Curvature	-0.998509990

Figure 6.9 demonstrates the rapid decrease in the Q-function and gain changes over five outer-evaluation and improvement steps. The third update moves the iterates into the local convergence region, after which the fourth and fifth updates drive all changes close to numerical precision. The five-step count describes outer policy updates as each update still uses the complete 500 transition dataset.

Direct Command-Mismatch Estimation: The primary estimates were $\widehat{b}_P = 0.0399973010$ and $\widehat{b}_E = -0.0099939136$, with percentage errors of approximately 0.00675% and 0.0609%. Figure 6.10 demonstrates that both channels have nearly identical spread since they use the same realized-input measurement noise standard deviation as well as the same calibration length.

The theoretical densities track the empirical histograms, the Monte Carlo means lie close to the true offsets, and the primary estimates fall within the expected individual estimate intervals. The tighter evader estimate relative to the known-model case is an information structure offset; more specifically, the direct estimator observes each realized channel separately and avoids inversion through the joint plant input map.

Compensation and Cost Recovery: Without compensation, the learned policy produced the same maximum state deviation as the model-based methods, 4.12863×10^{-2}. Estimated compensation reduces this value to 4.16682×10^{-6}, corresponding to a reduction of approximately 99.9899%. Exact compensation reproduced the nominal trajectory to approximately 2.78×10^{-17}.

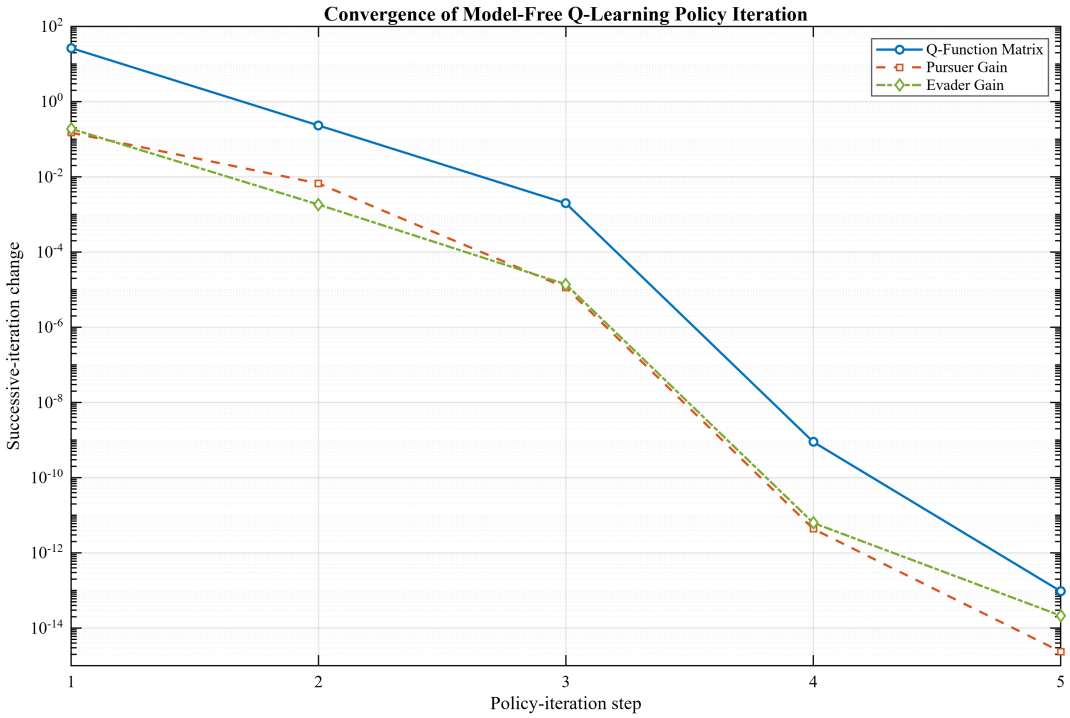

Fig. 6.9: Model-Free Q-Learning Policy Iteration Convergence

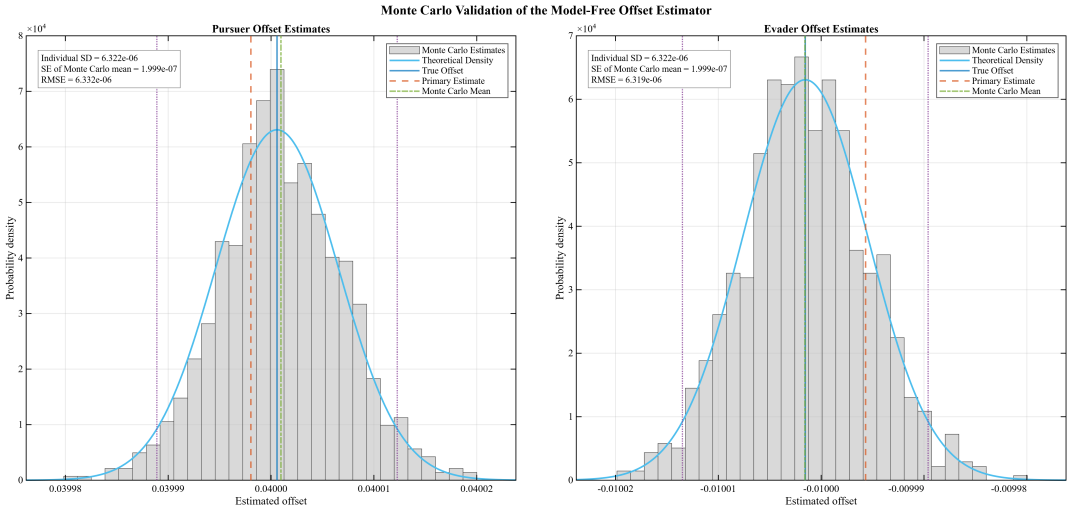

Fig. 6.10: Direct Offset-Estimator Monte Carlo Validation for Q-Learning Policy Iteration

Figure 6.11 demonstrates that the estimated compensation curve increases slightly from an initially zero deviation and the settles at a very small residual affine equilibrium. The effect is not a loss of stability; on the contrary, it is the expected accumulation of a microscale residual offset. The uncompensated curve, by contrast, approaches the same nonzero displacement seen in the model-based cases.

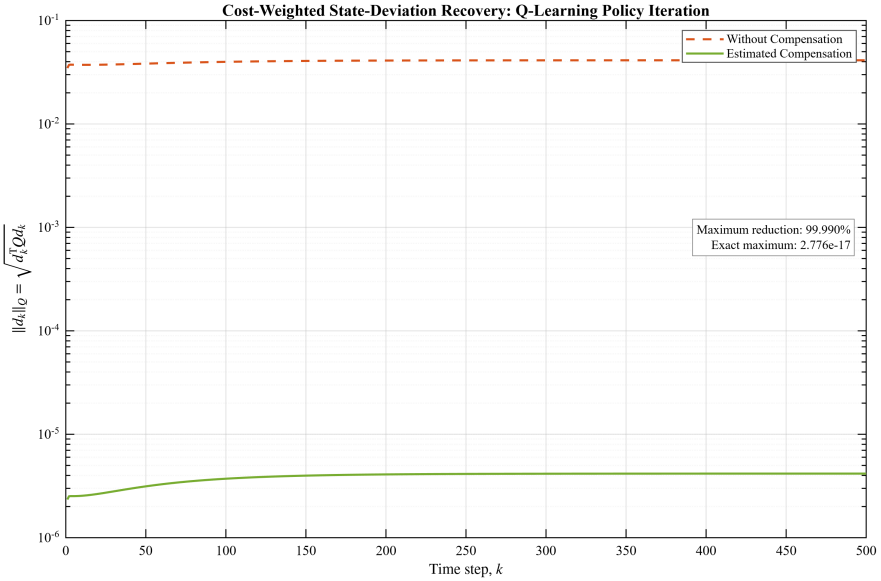

Fig. 6.11: Q-Learning Policy Iteration State Deviation Recovery

The analytical deviation recurrence agreed with simulation to approximately 3.74×10^{-15} without compensation and 7.69×10^{-15} with estimated compensation. The signed costs were $J_{\text{nom}} = 3169.85881235989$, $J_{\text{uncomp}} = 3171.20561554776$, $J_{\text{est}} = 3169.85881258328$, and $J_{\text{exact}} = 3169.85881235989$.

Command Channel Behavior: Figure 6.12 separates the policy input, compensated command, and realized input for the direct estimator. The command signals are displaced by the estimated offsets, after which the true channel offsets bring the realized inputs back into near coincidence with the nominal policy inputs. The remaining separation is on the microscale and is determined by the estimator residuals.

Therefore, Figures 6.9 through 6.12 demonstrate a complete model-free policy iteration result; moreover, the quadratic policy is recovered from sampled transitions, the direct mismatch estimator identifies both command offsets, and the execution-side correction restores the nominal behavior without inserting the analytical plant matrices into the Bellman regression.

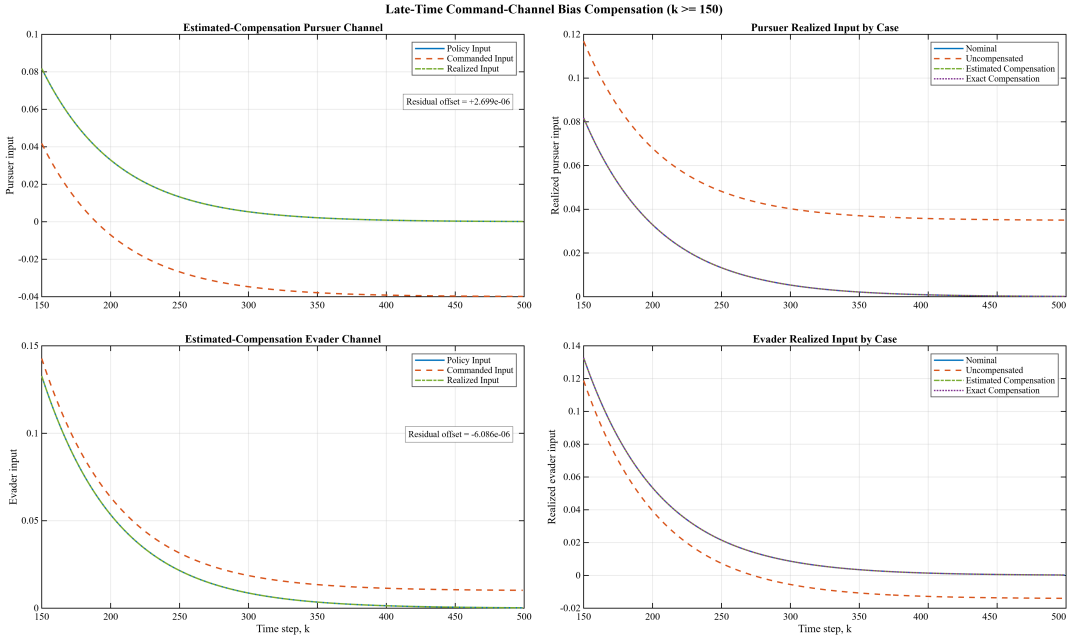

Fig. 6.12: Q-Learning Policy Iteration Command Channel Compensation

6.7. Model-Free Q-Learning Value Iteration Results

Nominal Learning Convergence: Q-learning value iteration used the same fixed 500 transition dataset and the same 15-parameter quadratic feature representation. The raw and scaled feature matrices again had full rank; moreover, the method converged in 633 value iteration steps, matching the outer step count of the model-based GARE value iteration under the selected tolerance. The learned Q-function, gain pair, and value matrix agreed with the model-based reference errors on the order of 10^{-10}, which is consistent with the stopping tolerance.

Figure 6.13 demonstrates the nearly geometric decrease of the Q-function and gain changes over the main portion of the 633 bootstrapped updates. The late movement in the evader gain curve occurs only near 10^{-12} and is not renewed divergence. In addition, it reflects floating-point sensitivity after the gain has already converged to the displayed precision.

Direct Command-Mismatch Estimation: The value iteration implementation utilizes the same direct command-mismatch measurement structure as Q-learning policy iteration. Figure 6.14 presents its own Monte Carlo output such that the figure sequence remains aligned with the value iteration method rather than referring backward to another algorithm's plot.

The pursuer and evader histograms have essentially the same width, as expected from the equal

TABLE XII: Nominal Numerical Validation for Model-Free Q-Learning Value Iteration

Diagnostic	**Observed value**
Converged	Yes
Value-Iteration Steps	633
Feature Rank	$15/15$
Scaled Feature Rank	$15/15$
Scaled Feature Condition Number	308.8705
Final Q-Function Change	9.7898×10^{-11}
Final Pursuer-Gain Change	4.6138×10^{-13}
Final Evader-Gain Change	1.9229×10^{-12}
Final Regression Residual	4.0531×10^{-16}
Fixed-Point Bellman Residual	3.3758×10^{-12}
Closed-Loop Spectral Radius	0.9819707273
Game-Matrix Reciprocal Condition Number	0.56579379
Minimum Pursuer Curvature	1.761840343
Maximum Evader Curvature	-0.998509990

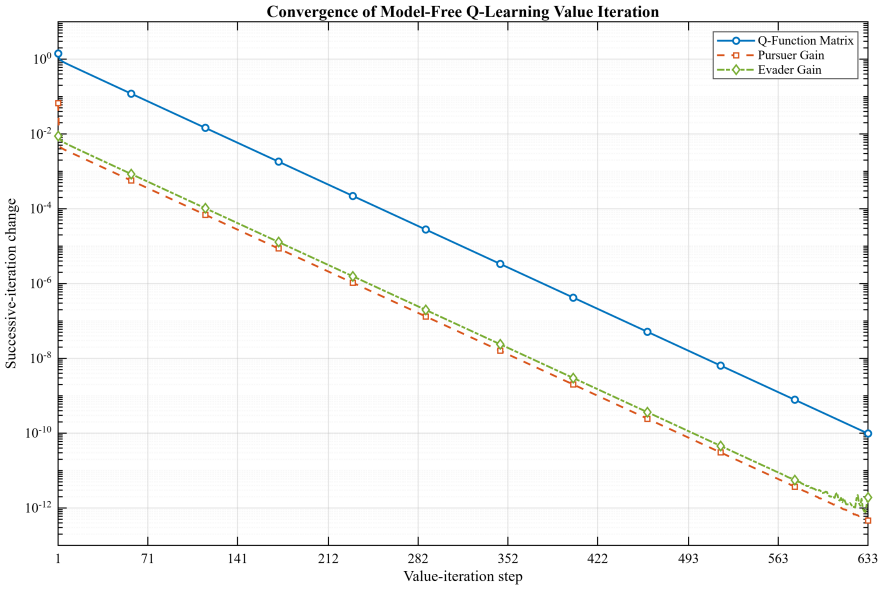

Fig. 6.13: Model-Free Q-Learning Value Iteration Convergence

channel noise levels and common calibration length. The theoretical densities agree with the empirical distributions, and the primary estimates lie within the expected sampling ranges. In addition, this confirms that keeping the disturbance information fixed across the two Q-learning methods produces the same estimator-scale behavior.

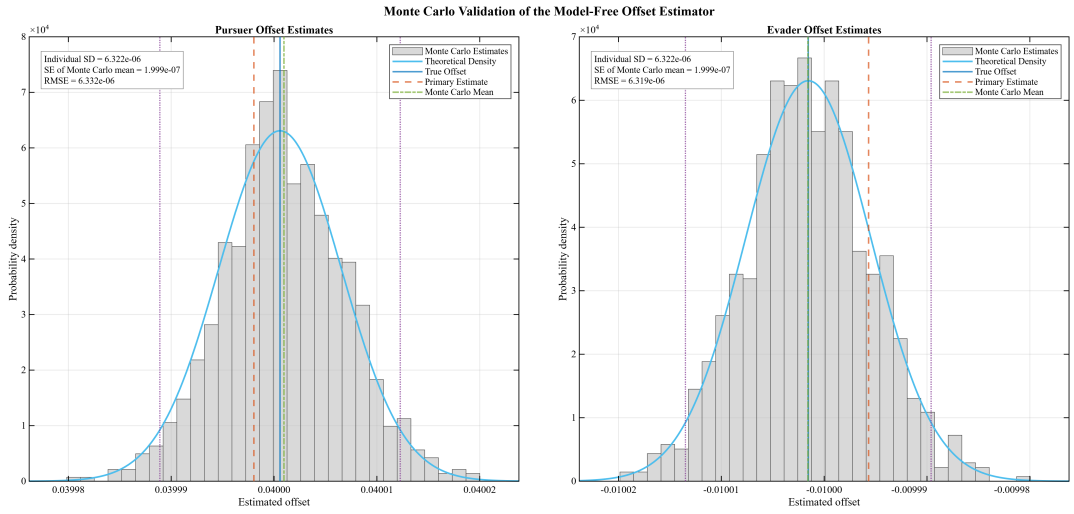

Fig. 6.14: Direct Offset Estimator Monte Carlo Validation for Q-Learning Value Iteration

Compensation and Cost Recovery: The uncompensated maximum deviation was 4.12863×10^{-2}, and estimated compensation reduced it to 4.16682×10^{-6}. The reduction was 99.9899%, exact compensation reproduced the nominal trajectory to approximately 2.78×10^{-17}, and the analytical mismatches were approximately 6.38×10^{-15} and 6.63×10^{-15} for the uncompensated and estimated cases.

Figure 6.15 demonstrates that the same residual affine equilibrium obtained with Q-learning policy iteration. This agreement is expected since both learning methods recover the same gain pair and use the same direct offset-information structure. Therefore, the figure validates the compatibility of the compensation layer with the bootstrapped value update rather than indicating a distinct nominal controller.

The signed costs were $J_{\text{nom}} = 3169.85881235989$, $J_{\text{uncomp}} = 3171.20561554652$, $J_{\text{est}} = 3169.85881258327$, and $J_{\text{exact}} = 3169.85881235989$. These values agree with the policy iteration results to the reported precision.

Command Channel Behavior: Figure 6.16 completes the value iteration execution record. The estimated compensation commands are translated by the recovered offsets, the realized inputs nearly overlap the nominal and exact compensation curves, and the uncompensated curves retain the persistent channel biases. The residual labels correspond to the same microscale estimation errors that determine the remaining state deviation.

Figure 6.13-6.16 completes the four method validation;moreover, they demonstrate that the disturbance-aware execution layer remains compatible with a model-free value iteration update and that the observed recovery is governed by estimator accuracy rather than by a change in the nominal saddle solution. The

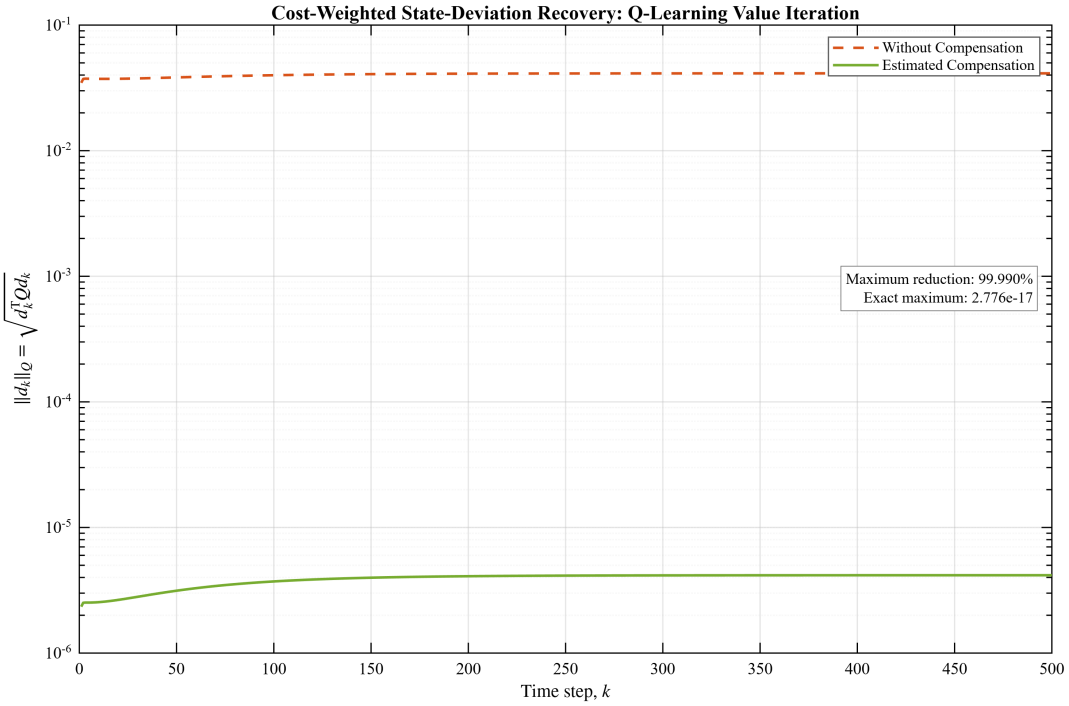

Fig. 6.15: Q-Learning Value Iteration State Deviation Recovery

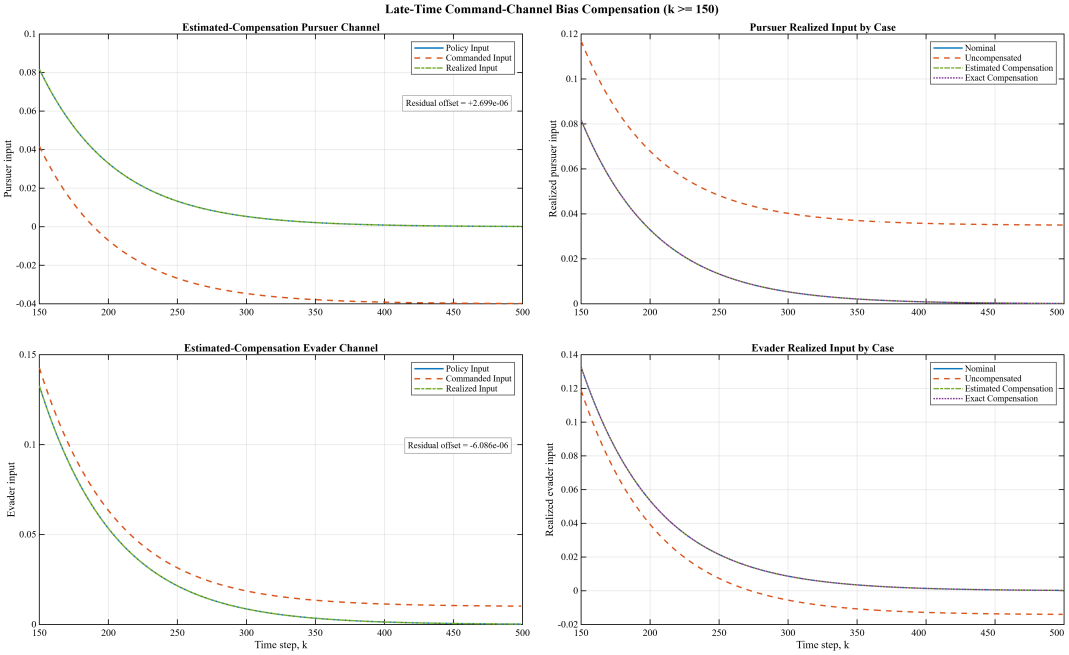

Fig. 6.16: Q-Learning Value Iteration Command Channel Compensation

next section compares the four methods directly under the matched benchmark.

6.8. Matched Four-Method Comparison

Nominal Solution Agreement: Table XIII compares the principal numerical results of the four nominal algorithms. All methods produce the same closed-loop spectral radius, game matrix conditioning, and saddle curvatures to the displayed precision. The model-free methods also matched the offline model-based reference with policy iteration reaching relative errors near 10^{-12} or smaller and value iteration reaching errors near 10^{-10}. These differences are consistent with the stopping histories and do not represent meaningfully different controllers in the present benchmark.

TABLE XIII: Matched Nominal convergence and Validity Comparison

Method	Outer Steps	Primary Final Change	Defining Residual	$\rho(A_{\mathrm{cl}})$	Game rcond	Curvature Pair
GARE Policy Iteration	4	$\approx 9.0 \times 10^{-10}$	6.40×10^{-15} GARE	0.9819707273	0.56579379	$(1.76184, -0.99851)$
GARE Value Iteration	633	9.44×10^{-11}	9.44×10^{-11} GARE	0.9819707273	0.56579379	$(1.76184, -0.99851)$
Q-Learning Policy Iteration	5	9.54×10^{-14}	4.02×10^{-15} Bellman	0.9819707273	0.56579379	$(1.76184, -0.99851)$
Q-Learning Value Iteration	633	9.79×10^{-11}	3.38×10^{-12} Bellman	0.9819707273	0.56579379	$(1.76184, -0.99851)$

The iteration counts must be interpreted according to algorithm structure; more specifically, each policy iteration update contains a complete fixed-policy evaluation, whereas value iteration advances the value representation through simpler optimality or bootstrapped updates. Therefore, the results establish a difference in outer convergence behavior, not a general wall-clock or sample complexity ranking. Such a ranking would require timed runs, solver-level accounting, repeated trials, and scaling across problem dimensions.

The nominal agreement is itself an important validation result, since all four methods are intended to solve the same linear-quadratic zero-sum game, a substantial disagreement would indicate an inconsistency in signs, action labeling, feature construction, policy recovery, or numerical implementation. Rather, the methods recover the same saddle solution under their respective information structures.

Estimator Information Structures: The two model-based methods use the same transition-residual estimator, and the two model-free methods use the same direct command mismatch estimator. Table XIV compares the resulting compensation performance.

All four nominal controllers are numerically equivalent; moreover, the smaller residual trajectory obtained by the model-free implementation thus reflects the estimator information structure rather than an improved Q-learning controller. The direct estimator observes each realized channel separately, whereas

TABLE XIV: Offset Estimation and Execution Recovery Comparison.

Method	Estimator Information	$\widehat{b}_P$	$\widehat{b}_E$	Max. est. Deviation	Reduction	$J_{\text{est}} - J_{\text{nom}}$
GARE Policy Iteration	Known-Model Transition Residual	0.040001367	−0.010287438	1.1350×10^{-4}	99.7251%	-4.90×10^{-5}
GARE Value Iteration	Known-Model Transition Residual	0.040001367	−0.010287438	1.1350×10^{-4}	99.7251%	-4.90×10^{-5}
Q-learning Policy Iteration	Direct Command Mismatch	0.039997301	−0.009993914	4.1668×10^{-6}	99.9899%	2.23×10^{-7}
Q-learning Value Iteration	Direct Command Mismatch	0.039997301	−0.009993914	4.1668×10^{-6}	99.9899%	2.23×10^{-7}

the model-based estimator reconstructs both offsets from their combined state-transition effect through a moderately conditioned input map. Under the specified noise model, direct realized-input measurement produces the more accurate estimate. That is, known-model residual estimation remains applicable when the realized player inputs cannot be measured separately.

Effect of Compensation Across All Methods: The uncompensated maximum deviation is approximately 4.1286×10^{-2} for every method. This common value confirms that the same plant, offsets, initial state, and nominal gain pair are used throughout the comparison. Estimated compensation reduces the deviation by more than 99.7% for the model-based estimator and by more than 99.98% for the direct model-free estimator. Exact compensation reproduces nominal behavior to between approximately 10^{-15} and 10^{-17}, depending on the numerical path used by the program.

The analytical affine-recursion mismatches are on the order of 10^{-15} for all four methods. This check is stronger than visual trajectory overlap since it confirms that the simulated deviations follow the derived residual bias model and are not caused by an undocumented policy change, state reset, or inconsistent action convention.

The signed-cost comparison leads to the same conclusion. The uncompensated case increases the finite-window cost by approximately 1.3468. Estimated compensation returns the cost to within approximately 4.9×10^{-5} for the model-based estimator and 2.3×10^{-7} for the direct estimator. Exact compensation reproduces the nominal cost to the displayed precision. The trajectory and cost metrics thus show that the remaining execution error is governed by the residual offset estimation error.

Policy Iteration versus Value Iteration: Within both information structures, the policy iteration methods require substantially fewer outer policy iterations than the corresponding value iteration methods require value iterations. GARE policy iteration converges in four outer policy iterations, while GARE value iteration in 633 value iterations. On the other hand, Q-learning policy iteration in 5 outer policy iterations, and Q-learning value iteration in 633 value iterations.

These counts describe different numerical paths to the same final control law. Policy iteration per-

forms complete fixed policy evaluation before improvement, whereas value iteration advances the value representation gradually. Since the work performed by one update differs across the two structures, then the counts do not establish that policy iteration is universally faster. They do demonstrate that policy evaluation and repeated optimality updates have visibly different convergence histories even when the final saddle solution is shared.

Model-Based versus Model-Free Computation: The model-based algorithms have direct access to A_d, B_P, and B_E during nominal computation. The model-free algorithms instead use sampled states, realized inputs, stage costs, and successor states to identify the quadratic Q-function. Full feature rank, acceptable conditioning, and small Bellman residuals demonstrate that the dataset contains sufficient quadratic information for this benchmark. Agreement with the GARE reference then verifies that the learned input blocks recover the same saddle gains.

Under the controlled deterministic benchmark and exact quadratic class, the model-free regressions thus recover the same nominal zero-sum solution without using the analytical transition matrices in the learning update. This agreement does not imply equivalent recovery for arbitrary nonlinear plants, noisy state measurements, insufficiently excited datasets, or misspecified function classes.

6.9. Evaluation of Thesis Questions

The first research question concerns whether the four nominal algorithms can be implemented while remaining traceable to their established zero-sum equations. The GARE, Lyapunov, Bellman, gain recovery, stability, and saddle curvature checks support an affirmative answer. More specifically, all four implementations recover the same nominal saddle solution to numerical tolerance.

Then, the second research question discusses the effect of persistent command offsets when the feedback gains remain unchanged. The offsets leave the nominal closed-loop eigenvalues unchanged but introduce a constant affine forcing term, producing a maximum Q-weighted state deviation of approximately 4.1286×10^{-2} and a finite-window cost increase of approximately 1.3468. The analytical affine recurrence agrees with simulation to approximately 10^{-15}.

Next, the third research question concerns offset estimation and compensation. The rank two model-based residual estimator and the direct command mismatch estimator recover both offsets under their stated information and measurement assumptions. Estimated compensation reduces the maximum devi-

ation by approximately 99.725% in the model-based cases and 99.990% in the model-free cases, while exact compensation reproduces the nominal trajectory and signed cost to floating-point precision.

The fourth research question concerns policy iteration and value iteration under matched conditions. The methods follow different computational paths and require different numbers of outer updates; however, they recover the same nominal saddle solution. The update counts alone do not establish a universal runtime advantage since the work that is performed by a policy iteration update differs from the work done by the value iteration update.

Finally, the fifth research question discusses a possible transfer to nonlinear vehicle, QLabs, or QCar studies. The present work provides a reproducible reduced-order mathematical and MATLAB foundation, which includes the common game, command-offset model, estimator assumptions, compensation cases, and numerical validity checks. Moreover, it does not establish a completed nonlinear vehicle controller nor hardware results so nonlinear dynamics, sensing, constraints, real-time effects, and platform testing are potential future research directions.

6.10. Limitations of the Numerical Study

The evidence that was provided in this chapter is intentionally controlled. More specifically, the plant is linear, time-invariant, fully-observed, and deterministic during nominal control computation. The persistent offsets are constant over the calibration and execution intervals. The calibration noise is zero mean and Gaussian in the MATLAB experiment. The model-free dataset is generated under sufficiently rich exploration and maintains full quadratic feature rank. These assumptions isolate the disturbance-aware effect, however they also define the limits of the claims.

First, the three-state benchmark is a reduced zero-sum engagement model as opposed to a complete nonlinear vehicle model. More specifically, planar position, heading, wheelbase, tire effects, acceleration dynamics, steering limits, communication delay, and actuator saturation are not included in the four algorithms that were evaluated here. Therefore, the results provide a theoretical and computational foundation for later QCar or QLabs work, but they do not constitute hardware validation.

Second, the offsets are persistent constants; more specifically, time-varying bias, abrupt faults, nonlinear dead-zones, hysteresis, drift, and colored measurement noise would require different estimators and possibly a dynamic compensation state. The present affine correction is not a universal fault-tolerant controller.

Third, the model-free results rely on a deterministic dataset whose feature matrix has full rank and moderate conditioning. The exact quadratic representation is appropriate for the linear-quadratic benchmark. Therefore, neural approximation, nonlinear basis selection, safety-constrained exploration, and sample-limited online learning are delegated to future research directions.

Fourth, the direct command mismatch estimator assumes that the realized pursuer and evader inputs can be measured separately. That assumption can be realistic in an instrumented command interface, however it is not equivalent to having no model and no actuator measurements. Similarly, the model-based residual estimator assumes that accurate nominal matrices and enough input-direction independence to separate the offsets.

Finally, the outer iteration counts are not a complete computational complexity study. Policy iteration steps contain matrix or regression evaluations that differ from value iteration steps. The results justify a qualitative comparison of convergence histories, however not a universal claim that one method is faster in all implementations or at larger problem dimensions.

6.11. Summary

The four implementations recover the same stable nominal saddle solution under the shared benchmark. More specifically, the model-based residuals, model-free Bellman residuals, feature rank and conditioning checks, saddle curvatures, and closed-loop spectral radii jointly support the validity of the nominal computations. Persistent commands offsets produce a common uncompensated deviation, while estimated compensation reduces that deviation by approximately 99.990% for the direct command mismatch estimator. Exact compensation reproduces the nominal trajectory and signed cost to floating-point precision, and the analytical affine deviation recurrence agrees with simulation to approximately 10^{-15}.

These results complete the technical progression established from Chapters 3 through 5. The common framework defines one game and one disturbance model, the model-based methods solve the nominal game from known matrices, and the model-free methods recover the same quadratic saddle structure from sampled transitions. The next chapter synthesizes these findings and identifies nonlinear vehicle modeling, QLabs implementation, and physical QCar validation as the next potential research directions.

7. Conclusions and Future Work

7.1. Summary

This thesis's objective was to investigate disturbance-aware zero-sum pursuit-evasion control through four closely related solution algorithms, which were model-based Game Algebraic Riccati Equation (GARE) policy and value iteration, as well as model-free quadratic Q-learning policy and value iteration. This thesis did not intend to replace these nominal algorithms. Rather, it placed them within one mathematical and experimental framework and extended their execution layer to address persistent command-channel offsets affecting the minimizing and maximizing players.

Chapters 2 through 5 established the literature basis, common zero-sum formulation, and the nominal model-based and model-free algorithms. The model-based algorithms used known plant matrices whereas the model-free algorithms recovered the corresponding quadratic action-value relationship from measured transitions. In both classes, the nominal algorithm was completed and verified before the command offsets were estimated and affine compensation was applied. As a result, this ordering preserved the published algorithmic structures and kept the thesis contribution at the execution and validation layers.

Chapter 6 evaluated all four algorithms under one benchmark, one pair of persistent offsets, one initial condition, one calibration length, and one execution horizon. The validation separated successive iteration convergence from equation consistency, saddle-point validity, estimator accuracy, command bookkeeping, and compensated execution. Each implementation was evaluated across multiple dimensions of performance and stability, including system residuals (GARE, Lyapunov, or Bellman), structural properties (closed-loop, spectral radius, saddle curvatures, and applicable rank/conditioning), and validation metrics like affine-deviation recurrences, Monte Carlo statistics, trajectory recovery, and finite-window signed costs.

The model-based policy iteration method converged in four outer policy iterations, and the model-free policy iteration method converged in five. Both value iteration methods require 633 value iterations. These counts distinguish the numerical paths, but they do not establish a universal runtime or sample complexity advantage since the work that is performed by a policy iteration update differs from the work performed by a value iteration update. All four methods produced the same nominal closed-loop spectral radius expressed by

$$\rho(A_d + B_P L + B_E K) = 0.9819707273 < 1, \tag{7.1}$$

and the same verified saddle curvatures to the displayed precision. The learned Q-functions and gains also agreed with the model-based reference at the levels implied by their stopping tolerances. The four implementations thus recover the same nominal linear-quadratic saddle solution under the stated numerical conditions.

Without compensation, the common maximum cost-weighted state deviation was

$$\max_k \|d_k\|_Q \approx 4.1286 \times 10^{-2}. \tag{7.2}$$

Known-model transition residual estimation reduced the maximum deviation to approximately 1.1350×10^{-4}, or around 99.725%. Direct command mismatch estimation reduced it to approximately 4.1668×10^{-6}, or around 99.990%. The smaller model-free residual does not indicate a better nominal Q-learning controller. The nominal controllers are numerically equivalent; moreover, the differences arises since the direct estimator observes each realized channel separately, whereas the model-based estimator reconstructs both offsets from their combined state-transition effect through $\begin{bmatrix} B_P & B_E \end{bmatrix}$.

The signed cost results support the same interpretation; that is, the uncompensated offset increased the finite-window cost by approximately 1.3468 relative to nominal. Estimated compensation returned the model-based cases to within approximately 4.9×10^{-5} of the nominal cost and the model-free cases to within approximately 2.3×10^{-7}. Exact compensation reproduced the nominal trajectory and signed cost to the reported precision. With the nominal gain pair fixed, then the disturbed state deviation followed

$$\begin{aligned} d_{k+1} = &\left(A_d + B_P L + B_E K\right) d_k \\ &+ B_P \left(b_P - \widehat{b}_P\right) + B_E \left(b_E - \widehat{b}_E\right). \end{aligned} \tag{7.3}$$

The simulated and independently propagated deviations agreed to approximately 10^{-15} in all four programs. The compensated trajectories thus approach nominal behavior since the estimated offsets reduce the affine forcing by the residual estimation errors predicted by the model.

Table XV summarizes the solutions to the central research questions and limits each conclusion to the completed theory as well as MATLAB evidence. Therefore, the thesis establishes a bounded result; more specifically, that a persistent command mismatch is able to be estimated and compensated across four established zero-sum solution methods while the nominal game remains fixed and traceable to its source algorithms. However, it does not demonstrate that model-free control universally dominates model-based

control, that value iteration is always slower than policy iteration, nor that constant bias compensation resolves general actuator faults.

TABLE XV: Research Question Conclusions

Research Question	Conclusion	Supporting Evidence
Can the four nominal algorithms be implemented so that their equations remain traceable to the established zero-sum algorithms from which they are derived?	Yes, for the controlled benchmark and stated numerical conditions.	Each method retained its Riccati, Lyapunov, Bellman, or coupled gain-recovery structure and produced the same stable nominal saddle gains, spectral radius, curvature values, trajectory, and cost to numerical tolerance.
How do constant pursuer and evader command offsets alter the nominal response when the feedback gains are unchanged?	They introduce a persistent affine deviation and change the finite-window game cost without changing the nominal closed-loop eigenvalues.	All four uncompensated cases produced a maximum state deviation of approximately 4.1286×10^{-2} and a cost increase of approximately 1.3468; the analytical recurrence matched simulation to approximately 10^{-15}.
Can the offsets be estimated using the information available to the model-based and model-free methods, and can affine compensation reduce the deviation?	Yes, under the stated identifiability and measurement assumptions.	The rank-two model-based residual estimator and the direct command-mismatch estimator recovered both offsets. Estimated compensation reduced maximum deviation by approximately 99.725% or 99.990%, while exact compensation reproduced nominal behavior to floating-point precision.
How do policy iteration and value iteration compare within the model-based and model-free classes under matched conditions?	They follow different computational paths but recover the same nominal game solution.	The policy-iteration methods converged in four or five outer policy iterations, while both value-iteration methods required 633 value iterations. Final gains and closed-loop behavior agreed across all four methods.
To what extent does the framework provide a defensible foundation for later nonlinear vehicle, QLabs, or physical QCar studies?	It provides a verified reduced-order theoretical and MATLAB foundation, not completed vehicle or hardware validation.	The common game, command-offset model, estimator assumptions, compensation cases, and numerical checks are reproducible and transferable, while nonlinear dynamics, sensing, constraints, real-time effects, and platform testing remain future work.

7.2. Contributions

The first contribution that thesis provides is a unified disturbance-aware zero-sum framework that keeps the physical problem fixed across model-based and model-free computation. The same state equation, minimizing and maximizing channels, quadratic objective, feedback convention, offset model, as well as numerical metrics are used throughout the thesis. This common structure make cross-method agreement and disagreement interpretable since every method is evaluated against the same mathematical object.

Then, the second contribution is the extension of the model-based GARE policy iteration and value iteration algorithm implementations with persistent offset calibration and affine compensation. The nominal policy evaluation, policy improvement, and value update structures remain the established model-based algorithms. Moreover, the added components estimate the offsets from known-model transition residuals, compensate the commands during execution, and quantify the deviation that is produced by residual estimation error. This estimator does not require direct measurement of each realized input; however, it relies on the accuracy and conditioning of the nominal input map.

Next, the third contribution is the corresponding extension of the model-free quadratic Q-learning implementations. The policy iteration and value iteration methods retain their sampled Bellman regressions and gain recovery structures. The disturbance-aware execution layer adds command-versus-realized input accounting, direct command mismatch estimation, disturbance consistent action labeling, and compensated closed-loop simulation. The nominal Bellman update remains model-free since the analytical plant matrices are not inserted into the regression nor policy recovery equations. The complete implementation is not information free; more specifically, it still needs measured states, realized actions, costs, successor states, and command interface measurements.

Hence, the fourth contribution is a shared numerical validation protocol as each method is evaluated under nominal operation, uncompensated offsets, estimated compensation, and exact compensation. The MATLAB programs generate convergence histories, equation or Bellman residuals, stability and saddle checks, conditioning information, calibration estimates, Monte Carlo distributions, command histories, trajectories, costs, and independent analytical deviation comparisons from the same stored arrays. Thus, the protocol distinguishes an update that has stopped changing from an equation that is satisfied, a stable closed-loop from a valid saddle point, and visual trajectory overlap from verified command and affine recursion bookkeeping.

The fifth and final contribution is explicit separation between nominal algorithm validity and disturbance-

compensation performance. The GARE and Q-learning equations are not rewritten to absorb an arbitrary disturbance term. Each established nominal algorithm is completed first, then it follows that the command channel is modeled as the interface at which a persistent offset changes the realized input. Calibration estimates that mismatch, and affine compensation translates the policy command before it enters the biased channel. As a result, this separation preserves scholarly accuracy while identifying precisely what another researcher will be able to reproduce, test, or extend.

Collectively, these contributions establish a matched four method comparison in which computational path, information structure, and execution-side disturbance effects are separated. Moreover, the methods recover the same nominal saddle solution, while the disturbance-aware extensions show how persistent command mismatch can be estimated and compensated under method appropriate assumptions. Therefore, the resulting contribution is a reproducible MATLAB-validated foundation rather than a claim of universal algorithmic superiority or completed vehicle implementation.

7.3. Limitations

The conclusions of this thesis are bound first by the plant model; moreover, the three-state benchmark is a reduced-order linear zero-sum engagement model. It provides a mathematically transparent setting for Riccati and quadratic Q-learning analysis, however it does not contain the complete planar position, heading, speed, wheelbase, tire, actuator, and contact dynamics that are associated with two autonomous ground vehicles. Therefore, pursuit-evasion is the game interpretation of the minimizing and maximizing channels rather than a claim that the current state vector is a complete geometric vehicle-capture model.

In addition, the plant is also time-invariant, fully-observed, and deterministic during nominal policy computation. The model-based solution assumes that the matrices supplied to the GARE algorithms are accurate, while the model-free solution assumes that the measured transitions are sufficiently informative and that a quadratic Q-function exactly represents the nominal game. As a result, these assumptions are appropriate for isolating the algorithmic and disturbance-aware relationships that are studied here, however they limit generalization to nonlinear, partially-observed, or strongly stochastic systems.

Next, a second limitation is the disturbance model; more specifically, the offsets b_P and b_E are constant over the calibration and execution intervals and enter through the same channels as the player commands. Real actuators may exhibit time-varying drift, dead-zones, hysteresis, gain errors, etc. Some of these effects can be approximated locally as offsets, but they are not equivalent to the constant matched

disturbance used in this thesis. Unmatched disturbances that do not lie in the player input directions cannot generally be removed by subtracting a command bias.

A third limitation concerns estimator assumptions. The model-based assumptions transition-residual estimator requires sufficiently independent input directions, accurate nominal matrices, and state transition measurements from which the residual can be formed. As a result, a poorly conditioned or rank-deficient matrix given by $\begin{bmatrix} B_P & B_E \end{bmatrix}$ would make separate offset recovery either unreliable or impossible. Hence, the model-free direct estimator avoids this inverse problem but assumes separately measured realized pursuer and evader inputs. On a physical platform, those measurements could require actuator sensing, motor-current interpretation, steering calibration, or low-level command telemetry. Therefore, the two estimators trade different types of information rather than representing universally superior and inferior choices.

The Monte Carlo studies utilized zero-mean Gaussian calibration noise with a fixed standard deviation and independent trials. As a result, this supports a clear comparison of estimator spread, however it does not cover colored noise, bias in the measurements themselves, heavy-tailed disturbances, correlated channels, quantization, or state-estimation error. Hence, the reported standard deviations and confidence markers are properties of the selected MATLAB noise model as opposed to platform independent uncertainty guarantees.

The model-free results rely on a deliberately rich offline dataset. The quadratic feature matrix has full rank, and scaling keeps the least-squares problem numerically manageable. In higher-dimensional problems, the number of unique quadratic coefficients grows rapidly. Exploration may become costly or unsafe, feature conditioning may deteriorate, and exact quadratic representation may no longer be available. The current results show structured recovery in a favorable benchmark; more specifically, they do not establish finite-sample guarantees for arbitrary data distributions nor approximation architectures.

In addition, this thesis does not provide a complete computational complexity comparison. The outer iteration counts make the policy iteration and value iteration paths easy to distinguish, however their updates do not have equal computational cost. As a result, the MATLAB programs were designed to verify equations and produce reproducible outputs rather than to benchmark optimized runtimes. Claims about speed, memory, or sample efficiency would require controlled timing experiments, solver-level accounting, repeated runs, and scaling studies across state and input dimensions.

The execution studies omit actuator saturation, state and input constraints, and safety filtering. The

nominal saddle policies are unconstrained linear feedback laws. As a result, a physical pursuit-evasion implementation would need to enforce speed, acceleration, steering, rate, workspace, and collision safety limits. Simply clipping an unconstrained policy can change the game and invalidate the exact linear-quadratic value relationship. Therefore, constrained dynamic games, model predictive control, barrier functions, or safety filters may be required before platform deployment.

Finally, this thesis does not include QLabs nor physical QCar experiments. Rather, the MATLAB results establish that the four algorithms, estimator assumptions, affine compensation equations, and validation metrics are internally consistent. They do not establish that the same performance will be obtained under real-time scheduling, sensor latency, network delay, command quantization, calibration error, actuator dynamics, or physical contact constraints. Hardware validation remains the next implementation stage rather than evidence that is required to support the current theoretical and computational contribution.

7.4. Further Research

Nonlinear Vehicle and Platform Validation: The most direct continuation of this thesis is to replace the reduced-order benchmark with a nonlinear two-vehicle model. A kinematic bicycle formulation would depict each agent through planar position, heading, speed, wheelbase, acceleration, and steering, while pursuit-evasion would still give the local game interpretation, nominal linearization, estimator organization, compensation cases, and validation logic.

As a result, this extension should proceed first in MATLAB or QLabs while maintaining the current benchmark as a reference. Tests across initial geometrics, speeds, wheelbase values, and capture radii would demonstrate where a local saddle policy remains useful and where repeated linearization, gain scheduling, or nonlinear dynamic programming becomes necessary. Then, QLabs should be used to verify command mapping, state extraction, timing, steering calibration, and multi-vehicle coordination before physical QCar deployment.

A QCar-style implementation would also need vector-valued command offsets since each vehicle can have separate longitudinal and steering commands. Thus, the scalar offsets can be generalized as follows

$$b_P = \begin{bmatrix} b_{P,a} & b_{P,\delta} \end{bmatrix}^\top, \quad b_E = \begin{bmatrix} b_{E,a} & b_{E,\delta} \end{bmatrix}^\top, \tag{7.4}$$

where the components denote acceleration or throttle bias and steering bias. The resulting identifiability problem would rely on operating point, input-direction independence, and the different noise and

bandwidth characteristics of the longitudinal and steering commands.

Physical QCar experiments should follow only after the QLabs interface is verified; moreover, hardware testing would require synchronized state estimation, repeatable reset procedures, bounded commands, emergency stopping, workspace constraints, and clear separation between calibration and evaluation data. The initial objective should be to validate the command-channel model as well as to determine whether persistent offsets are able to be measured, whether affine compensation reduces execution error, and whether nominal and exact compensation bookkeeping behaves as predicted.

Estimation, Sensing, and Execution Constraints: The current calibration is completed before the compensated execution run. Online estimation could instead use recursive least-squares, a Kalman filter bias state, an unknown-input observer, or a moving window estimator to update $\widehat{b}_P$ and $\widehat{b}_E$ while the game is in progress. As a result, the analysis would then need to account for estimator transients, interaction between policy and excitation, slowly varying drift, and abrupt faults.

Output Feedback implementation would extend the framework beyond exact state measurement. A practical vehicle platform may provide noisy position, orientation, encoder, inertial, and camera measurements. Thus, state estimation and input bias estimation would become coupled, motivating either an augmented state-and-input observer or an output-feedback Q-learning formulation in which the controller operates on estimated states.

Actuator saturation, rate limits, dead-zones, and command relay need separate treatment since they cannot be represented globally as constant offsets. Therefore, candidate approaches include saturation-aware policy iteration, constrained zero-sum model predictive control, reference governor or safety-filter layers, and delay augmented dynamic games. The command bookkeeping developed in this thesis remains applicable since each approach needs to distinguish policy output, command input, and realized input.

Learning Robustness, Safety, and Scaling: Further model-free studies should vary sample count, exploration amplitude, state distribution, feature conditioning, measurement noise, and model mismatch across repeated trials. Such experiments would support empirical confidence and sample complexity analyses and would specify when regularization, instrumental variables, recursive estimation, or robust regression becomes necessary.

For nonlinear systems, an exact global quadratic Q-function will generally be unavailable. A staged progression from local quadratic approximations and multiple operating regions to neural critics or policies would preserve interpretability for as long as it is feasible. The GARE solution and the nominal, uncom-

pensated, estimated compensation, and exact compensation cases should remain as reference conditions such that changes can be attributed to the approximation architecture rather than to an undocumented change within the game.

Safety-constrained pursuit-evasion is another important distinction. More specifically, a practical vehicle experiment needs to avoid unintended collisions, workspace violations, excessive steering, and unsafe speeds even when the evader acts adversarially. Then, control barrier functions, reachability methods, constrained dynamic games, or safety filters could be evaluated using capture and cost together with constraint violations, intervention frequency, and minimum safety margin.

Moreover, the two-player benchmark can also be expanded to multiple pursuers or evaders, introducing larger state and action spaces, communication, coordination, role assignment, and potentially nonzero-sum objectives. A parallel computational study should time policy evaluation, Riccati mapping, feature construction, regression, gain recovery, calibration, and simulation while varying problem dimension and sample count. This would determine when the lower outer iteration count of policy iteration translates into lower runtime and when the simpler updates of value iteration become advantageous.

These directions extend a completed foundation; moreover, this research fixes the nominal game, establishes model-based and model-free solution paths, defines the persistent command offset problem, derives the compensation effect, and then verifies the result in MATLAB. Therefore, nonlinear vehicles, online estimation, constraints, QLabs, hardware, and multi-agent systems are the next research directions rather than missing components of the current contribution.

In conclusion, the disturbance-aware execution is able to be added consistently to both model-based and structured model-free zero-sum control without obscuring the nominal game solution. When the offsets are identifiable or directly measurable, affine compensation then recovers the intended saddle policy behavior to the level set by estimator accuracy. The four algorithms differ in information and computation path, however they concur on the nominal game as well as on the mathematical effect of residual bias. As a result, that agreement provides the theoretical and numerical foundation that will be required for future pursuit-evasion implementation on more realistic autonomous vehicle models and platforms.

REFERENCES

[1] Tamer Basar and Geert Jan Olsder. *Dynamic Noncooperative Game Theory*. SIAM, Philadelphia, PA, 2 edition, 1999.

[2] Isaac E. Weintraub, Meir Pachter, and Eloy Garcia. An Introduction to Pursuit-Evasion Differential Games. In *2020 American Control Conference (ACC)*, Denver, CO, USA, 2020.

[3] Meir Pachter and K. D. Pham. Discrete-Time Linear-Quadratic Dynamic Games. *Journal of Optimization Theory and Applications*, 146(1):151–179, 2010.

[4] Asma Al-Tamimi, Frank L. Lewis, and Murad Abu-Khalaf. Model-Free Q-Learning Designs for Linear Discrete-Time Zero-Sum Games With Application to H-Infinity Control. *Automatica*, 43(3):473–481, 2007.

[5] Syed Ali Asad Rizvi and Zongli Lin. *Output Feedback Reinforcement Learning Control for Linear Systems*. Birkhäuser, Cham, Switzerland, 2023.

[6] Biao Luo, Yin Yang, and Derong Liu. Policy Iteration Q-Learning for Data-Based Two-Player Zero-Sum Game of Linear Discrete-Time Systems. *IEEE Transactions on Cybernetics*, 51(7):3630–3640, 2021.

[7] Jianguo Zhao, Chunyu Yang, Weinan Gao, and Ju H. Park. Novel Single-Loop Policy Iteration for Linear Zero-Sum Games. *Automatica*, 163:111551, 2024.

[8] Liangyuan Guo, Bing-Chang Wang, and Ji-Feng Zhang. On-Policy and Off-Policy Value Iteration Algorithms for Stochastic Zero-Sum Dynamic Games. *Journal of Systems Science and Complexity*, 38(1):421–435, 2025.

[9] Jiduan Wu, Anas Barakat, Ilyas Fatkhullin, and Niao He. Learning Zero-Sum Linear Quadratic Games with Improved Sample Complexity and Last-Iterate Convergence. *SIAM Journal on Control and Optimization*, 63(5):3244–3271, 2025.

[10] M. Kankashvar, S. Rafiee, and H. Bolandi. Fault-tolerant q-learning for discrete-time linear systems with actuator and sensor faults using input-output measured data. *Franklin Open*, 6:100259, 2025.

[11] Kun Yang, Ao Shen, Nengwei Xu, Fang Deng, Maobin Lu, and Chen Chen. A Review of Reinforcement Learning Approaches for Pursuit-Evasion Games. *Chinese Journal of Aeronautics*, 39(6):103940, 2026.

[12] Zewei Wang et al. Reinforcement Learning in Pursuit-Evasion Differential Game: Safety, Stability and Robustness, 2025. arXiv preprint.

[13] Burak M. Gonultas and Volkan Isler. Learning to Play Pursuit-Evasion with Dynamic and Sensor Constraints, 2024. arXiv preprint.

[14] S. A. A. Rizvi and Z. Lin. Output feedback Q-learning for discrete-time linear zero-sum games with application to the H_∞ control. *Automatica*, 95:213–221, 2018.

[15] Kenneth R. Muske and Thomas A. Badgwell. Disturbance Modeling for Offset-Free Linear Model Predictive Control. *Journal of Process Control*, 12(5):617–632, 2002.

[16] Gabriele Pannocchia and James B. Rawlings. Disturbance Models for Offset-Free Model-Predictive Control. *AIChE Journal*, 49(2):426–437, 2003.

[17] Steven Gillijns and Bart De Moor. Unbiased Minimum-Variance Input and State Estimation for Linear Discrete-Time Systems. *Automatica*, 2007.

[18] Sze Zheng Yong, Minghui Zhu, and Emilio Frazzoli. A Unified Filter for Simultaneous Input and State Estimation of Linear Discrete-Time Stochastic Systems. *Automatica*, 63:321–329, 2016.

[19] Rufus Isaacs. *Differential Games: A Mathematical Theory with Applications to Warfare and Pursuit, Control and Optimization*. John Wiley & Sons, New York, NY, 1965.

[20] Yu-Chi Ho. A Note on Linear-Quadratic Pursuit-Evasion Differential Games. *Journal of Optimization Theory and Applications*, 5(6):449–451, 1970.

[21] Arunabha Bagchi and Geert Jan Olsder. Linear Quadratic Stochastic Pursuit-Evasion Games. *Applied Mathematics and Optimization*, 7:95–123, 1981.

[22] Josef Shinar. Solvability of Linear-Quadratic Differential Games Associated with Pursuit-Evasion Problems. *International Game Theory Review*, 10(4):619–636, 2008.

[23] Jingrui Sun and Jiongmin Yong. Linear Quadratic Stochastic Differential Games: Open-Loop and Closed-Loop Saddle Points. *SIAM Journal on Control and Optimization*, 52(6):4082–4121, 2014.

[24] Jingrui Sun. Two-Person Zero-Sum Stochastic Linear-Quadratic Differential Games. *SIAM Journal on Control and Optimization*, 59(3):1804–1829, 2021.

[25] Andrea Monti, Benita Nortmann, Thulasi Mylvaganam, and Mario Sassano. Feedback and Open-Loop Nash Equilibria for Linear Quadratic Infinite-Horizon Discrete-Time Dynamic Games. *SIAM Journal on Control and Optimization*, 2024.

[26] Jingrui Sun and Jiongmin Yong. Long-Time Behavior of Zero-Sum Linear-Quadratic Stochastic Differential Games. *SIAM Journal on Control and Optimization*, 63(6):3961–3989, 2025.

[27] David L. Kleinman. On an Iterative Technique for Riccati Equation Computations. *IEEE Transactions on Automatic Control*, 13(1):114–115, 1968.

[28] Gary A. Hewer. An Iterative Technique for the Computation of the Steady-State Gains for the Discrete Optimal Regulator. *IEEE Transactions on Automatic Control*, 16(4):382–384, 1971.

[29] E. F. Mageirou. Iterative Techniques for Riccati Game Equations. *Journal of Optimization Theory and Applications*, 1977.

[30] Murad Abu-Khalaf, Frank L. Lewis, and Jie Huang. Policy Iterations on the Hamilton–Jacobi–Isaacs Equation for H-Infinity State-Feedback Control with Input Saturation. *IEEE Transactions on Automatic Control*, 53(6):1483–1488, 2008.

[31] Huaguang Wu and Biao Luo. Simultaneous Policy Update Algorithms for Learning the Solution of Linear Continuous-Time H-Infinity State-Feedback Control. *Information Sciences*, 2013.

[32] Jie Li et al. Relaxed Policy Iteration Algorithm for Nonlinear Zero-Sum Games with Application to H-Infinity Control. *IEEE Transactions on Automatic Control*, 69(1):426–433, 2024.

[33] Benita Nortmann, Andrea Monti, Mario Sassano, and Thulasi Mylvaganam. Nash Equilibria for Linear Quadratic Discrete-Time Dynamic Games via Iterative and Data-Driven Algorithms. *IEEE Transactions on Automatic Control*, 69(10):6561–6575, 2024.

[34] Dimitri P. Bertsekas. *Dynamic Programming and Optimal Control, Volume I*. Athena Scientific, Belmont, MA, 4 edition, 2017.

[35] Christopher J. C. H. Watkins and Peter Dayan. Q-Learning. *Machine Learning*, 8(3–4):279–292, 1992.

[36] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.

[37] Steven J. Bradtke and Andrew G. Barto. Linear least-squares algorithms for temporal difference learning. *Machine Learning*, 22:33–57, 1996.

[38] Michail G. Lagoudakis and Ronald Parr. Least-Squares Policy Iteration. *Journal of Machine Learning Research*, 4:1107–1149, 2003.

[39] Asma Al-Tamimi, Murad Abu-Khalaf, and Frank L. Lewis. Adaptive Critic Designs for Discrete-Time Zero-Sum Games

With Application to H-Infinity Control. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 37(1):240–247, 2007.

[40] Frank L. Lewis and Draguna Vrabie. Reinforcement Learning and Adaptive Dynamic Programming for Feedback Control. *IEEE Circuits and Systems Magazine*, 9(3):32–50, 2009.

[41] M. Liu, Q. Cai, W. Meng, D. Li, and M. Fu. Finite-horizon Q-learning for discrete-time zero-sum games with application to H_∞ control. *Asian Journal of Control*, 25(4):3160–3172, 2023.

[42] Mingxiang Liu, Qianqian Cai, Dandan Li, Wei Meng, and Minyue Fu. Output Feedback Q-Learning for Discrete-Time Finite-Horizon Zero-Sum Games with Application to H-Infinity Control. *Neurocomputing*, 529:48–55, 2023.

[43] Bosen Lian, Wenqian Xue, Yijing Xie, Frank L. Lewis, and Ali Davoudi. Off-Policy Inverse Q-Learning for Discrete-Time Antagonistic Unknown Systems. *Automatica*, 155:111171, 2023.

[44] Liangyuan Guo, Bing-Chang Wang, and Hailing Dong. Q-Learning Design for Discrete-Time Stochastic Zero-Sum Games. In *2023 China Automation Congress*, pages 6423–6426, 2024.

[45] Yuan Wang, Ding Wang, Mingming Zhao, Nan Liu, and Junfei Qiao. Neural Q-Learning for Discrete-Time Nonlinear Zero-Sum Games with Adjustable Convergence Rate. *Neural Networks*, 175:106274, 2024.

[46] Yun Wang, Tian Fang, Qingkai Kong, and Feng Li. Zero-Sum Game-Based Optimal Control for Discrete-Time Markov Jump Systems: A Parallel Off-Policy Q-Learning Method. *Applied Mathematics and Computation*, 467:128462, 2024.

[47] Z. Jiang et al. Data-Based Discrete-Time Two-Player Zero-Sum Delayed Games via Q-Learning. *Neurocomputing*, 2025.

[48] Shihan Liu, Zhi Chen, and Dongxu Gao. Safe Reinforcement Learning for Discrete-Time Nonlinear Zero-Sum Games with Unknown State Constraints and Asymmetric Input Constraints. *Neurocomputing*, 651:131026, 2025.

[49] Kaiqing Zhang, Zhuoran Yang, and Tamer Basar. Policy Optimization Provably Converges to Nash Equilibria in Zero-Sum Linear Quadratic Games, 2019.

[50] Urban Maeder, Francesco Borrelli, and Manfred Morari. Linear Offset-Free Model Predictive Control. *Automatica*, 45(10):2214–2222, 2009.

[51] Isah A. Jimoh, Ibrahim B. Kucukdemiral, Geraint Bevan, and Patience E. Orukpe. Offset-Free Model Predictive Control: A Study of Different Formulations with Further Results. In *2020 28th Mediterranean Conference on Control and Automation*, pages 671–676, 2020.

[52] Julian Berberich, Johannes Köhler, Matthias A. Müller, and Frank Allgöwer. Linear Tracking MPC for Nonlinear Systems—Part II: The Data-Driven Case. *IEEE Transactions on Automatic Control*, 67(9):4406–4421, 2022.

[53] Steven J. Kuntz and James B. Rawlings. Maximum Likelihood Identification of Linear Models with Integrating Disturbances for Offset-Free Control. *IEEE Transactions on Automatic Control*, 70(9):5675–5689, 2025.

[54] Steven J. Kuntz and James B. Rawlings. Offset-Free Model Predictive Control: Stability Under Plant-Model Mismatch. *IEEE Transactions on Automatic Control*, 71(5):3136–3151, 2026.

[55] Baibin Yang et al. ℓ_1-ATSXKF-Based State and Bias Estimation for Nonlinear Systems with Non-Gaussian Process Noise. *IET Control Theory & Applications*, 2025.

[56] Shaunak D. Bopardikar, Francesco Bullo, and João P. Hespanha. On Discrete-Time Pursuit-Evasion Games with Sensing Limitations. *IEEE Transactions on Robotics*, 24(6):1429–1439, 2008.

[57] Zifeng Gong, Bing He, Gang Liu, and Xiaobo Zhang. Solution for Pursuit-Evasion Game of Agents by Adaptive Dynamic Programming. *Electronics*, 12(12):2595, 2023.

[58] Zhixuan Zhang, Kun Zhang, Xiangpeng Xie, and Jiayue Sun. Fixed-Time Zero-Sum Pursuit-Evasion Game Control of Multisatellite via Adaptive Dynamic Programming. *IEEE Transactions on Aerospace and Electronic Systems*, 60(2):2224–2235, 2024.

[59] Xinfeng Xu, Chun Liu, Liang Xu, Qiang Wang, and Yizhen Meng. Integral Reinforcement Learning-Based Optimal Pursuit-Evasion Control for Multi-QUAVs: A Zero-Sum Game Approach. *Journal of the Franklin Institute*, 362:107685, 2025.

[60] Bo Li, Ziqi Yang, et al. Adaptive Dynamic Programming-Based Optimal Pursuit-Evasion Control for Quadrotor Unmanned Aerial Vehicles with Obstacle Avoidance. *Neurocomputing*, page 130483, 2025.

[61] Nick-Marios T. Kokolakis and Kyriakos G. Vamvoudakis. Safety-Aware Pursuit-Evasion Games in Unknown Environments Using Gaussian Processes and Finite-Time Convergent Reinforcement Learning. *IEEE Transactions on Neural Networks and Learning Systems*, 35(3):3130–3143, 2024.

[62] Xin Wang, Qing-Lai Wei, Tao Li, and Jie Zhang. Optimal Strategy for Aircraft Pursuit-Evasion Games via Self-Play Iteration. *Machine Intelligence Research*, 21:585–596, 2024.

[63] Jiawei Chen et al. Online Planning for Multi-UAV Pursuit-Evasion in Unknown Environments Using Deep Reinforcement Learning, 2025. arXiv preprint.

[64] Rajesh Rajamani. *Vehicle Dynamics and Control*. Springer, New York, NY, 2 edition, 2012.

[65] Cristian Tiriolo and Walter Lucia. A Feedback Linearized Model Predictive Control Strategy for Input-Constrained Self-Driving Cars, 2024.

[66] Quanser. *QCar*. Quanser, 2026. Accessed: 2026-05-24.

[67] Quanser. *QCar 2*. Quanser, 2026. Accessed: 2026-05-25.

[68] Quanser. *QCar 2 Object Library*. Quanser Interactive Labs API Documentation, 2026. Accessed: 2026-05-25.

[69] Quanser. *QLabs Virtual QCar 2*. Quanser, 2026. Accessed: 2026-05-24.

[70] Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. Cambridge University Press, Cambridge, UK, 2 edition, 2012.

VITA

Blaine Swieder was born in Springfield, Tennessee, United States of America. Blaine received his Bachelor of Science in Mathematics and Bachelor of Arts in Foreign Languages, Spanish from Tennessee Technological University in 2023. In 2023, he enrolled at Tennessee Tech to pursue his master's degree in Electrical and Computer Engineering. During his master's degree, he conducted research in autonomous systems, reinforcement learning, and control theory under the direction of Dr. Syed Ali Asad Rizvi.

Blaine was actively involved in academic and research projects, contributed to peer-reviewed publications, served as a Graduate Teaching Assistant in the Department of Computer Science, and as a Graduate Research Assistant in the Department of Electrical and Computer Engineering. He earned his Master of Science in Electrical and Computer Engineering in August 2026. Blaine plans to pursue his PhD in Mechanical Engineering at Vanderbilt University starting Fall 2026 under the direction of Dr. Alvin Strauss.